POLARIS 2026

Autonomous Underwater Ice-Thickness Sensing

Final Report

June 2026

Noel Bühler Louis Caflisch Ridh Choudhury Laure Gonner
Linus Kuster Jonas Oertli Gleb Platonov Richard Rüdin
Maximilian von Loesch Paul Zambelli

Supervision
Prof. Dr. Melanie Zeilinger
Dr. Andrea Carron
Alex Sheridan
Moritz Tuchschmid

ETH zürich IDSC

Project Website: `https://aris-space.ch/polaris/`
Project GitHub: `https://github.com/aris-space/project-polaris`

Abstract

Measuring alpine lake ice thickness is critical for climate monitoring and hazard assessment, yet current methods require manual drilling and lack spatial precision. To automate high-resolution data collection, the POLARIS focus project developed a 1.7 m tethered autonomous underwater vehicle (AUV) in collaboration with the ETH Department of Earth and Planetary Sciences (D-EAPS).

The vehicle's architecture was designed to meet four core objectives: a ±1 cm ice-thickness measurement accuracy, a 10 m spatial resolution over a 10 000 m^2 survey area, a horizontal localisation error bounded within ±2 m, and autonomous mission execution.

The 6-DOF AUV utilises a contact-based sensing methodology, deriving ice thickness from hydrostatic pressure via Archimedes' principle. Subsurface navigation is executed through a sensor-fusion framework that pairs inertial and acoustic dead reckoning with absolute updates from Global Navigation Satellite Systems (GNSS) and short-baseline (SBL) acoustic positioning.

The integrated system was validated through an iterative testing campaign advancing from open water to sub-ice deployments at Lake St. Moritz and Schwarzsee in Zermatt. Experimental results confirmed compliance with the ±1 cm thickness accuracy over the 13 cm to 23 cm ice range tested, and the ±2 m localisation constraint. Notably, field data verified that through-ice GNSS signals can successfully bound long-term sensor drift. While individual sensing and navigation loops were fully verified, full-grid autonomy was only partially validated on a scaled-down survey profile due to field constraints.

Acknowledgements

We, the POLARIS team, thank Prof. Dr. Melanie Zeilinger and Dr. Andrea Carron for enabling this Focus Project and for their continuous guidance and trust throughout. Dr. Cédric Schmelzbach and Dr. Simon Christian Stähler deserve equal gratitude for their geophysical expertise in shaping POLARIS's scientific direction, as well as Luca Ebner for sharing his technical experience from Tethys Robotics. Our project coaches, Alex Sheridan and Moritz Tuchschmid, supported us with valuable guidance throughout the project.

The Academic Space Initiative Switzerland (ARIS) kindly hosted POLARIS. The Innovation Park Zurich, particularly Thierry Tardent, Andrej Kern, and Michael Walter, provided workspace and technical assistance. Six volunteer contributors, Maxim Baumann, Yanick Blaser, Merve Kaya, Semira Kir, Emil Schoch, and Cherry Woo, supported us throughout the project.

Financial support came from the SCOR Foundation, the Institute for Dynamic Systems and Control (IDSC) at ETH Zurich, the Sustainability Fund of the Department of Mechanical and Process Engineering, and the Guido Feger Foundation. Our sponsors Panasonic Connect, Chalet Ulysse, WaterLinked, Tethys Robotics, Xsens, Foxglove, eisskulpturen.ch, Electrolux, IcePanel, Keller Pressure, Swaytronic, Holybro, Traco Power, Novacavi, CorrosionX and Feey enabled us to develop and test the autonomous underwater vehicle (AUV) from POLARIS.

Our field campaigns were made possible through the support of the Municipality of St. Moritz and its municipal staff, as well as the Municipality of Zermatt and Zermatt Bergbahnen itself, who facilitated access to Schwarzsee.

Finally, the entire POLARIS team deserves our deepest gratitude for their extraordinary dedication: Project Manager Maximilian von Loesch, Technical Lead Linus Kuster, Sponsoring Head Richard Rüdin, System Team Leads Noel Bühler and Jonas Oertli, and team members Louis Caflisch, Ridh Choudhury, Laure Gonner, Gleb Platonov and Paul Zambelli. This project is the result of the dedication, collaboration and passion of everyone involved.

Contents

Abstract i

Acknowledgements ii

Nomenclature v

1 Introduction 1
- 1.1 Motivation and Background . . . 1
- 1.2 Mission Objectives . . . 2
- 1.3 Requirements . . . 2
  - 1.3.1 Mission-Level Requirements (MR) . . . 2
  - 1.3.2 System-Level Requirements (SR) . . . 3

2 Field Testing and System Overview 4
- 2.1 Testing Sites . . . 4
- 2.2 Operational Setup . . . 5

3 Robotics Platform 7
- 3.1 Mainframe — Structures and Propulsion . . . 7
  - 3.1.1 Hull . . . 7
  - 3.1.2 Weight and Buoyancy System . . . 7
  - 3.1.3 Thruster Layout and Component Distribution . . . 8
  - 3.1.4 Tether and Strain Relief . . . 8
- 3.2 Electronics Tube and Watertight Interfaces . . . 9
  - 3.2.1 Pressure Vessel and Watertight Interfaces . . . 9
  - 3.2.2 Internal Electronics Layout . . . 9
- 3.3 Sensing . . . 10
- 3.4 Limitations and Future Work . . . 11

4 Software Platform 12
- 4.1 Architecture . . . 12
  - 4.1.1 Onboard Computer . . . 13
  - 4.1.2 Flight Controller . . . 13
- 4.2 Limitations and Future Work . . . 14

5 Localisation 15
- 5.1 Architecture . . . 15
- 5.2 Setup and Calibration . . . 16
- 5.3 Estimator Validation in Open Water . . . 16
  - 5.3.1 Static Sensor-Noise Characterisation . . . 17
  - 5.3.2 Dead-Reckoning Drift and Global Correction . . . 18
- 5.4 Sensor Behaviour Under Ice . . . 20
  - 5.4.1 GNSS Through Ice and Slush . . . 20
  - 5.4.2 Acoustic SBL . . . 20
  - 5.4.3 DVL Bottom-Lock . . . 21
- 5.5 Limitations and Future Work . . . 22

6 Path Planning and Control 23

6.1 Architecture . . . 23
6.2 Simulation Environment . . . 24
6.3 Tests . . . 24
6.3.1 First Simulation Cycle . . . 24
6.3.2 Low-Level Controller Tuning in Lake Zurich . . . 26
6.3.3 Second Simulation Cycle . . . 28
6.3.4 Autonomous Grid in Lake Zurich . . . 28
6.4 Limitations and Future Work . . . 30

7 Ice Measurement **32**
7.1 Measurement Principle and Method . . . 32
7.2 Tests . . . 33
7.2.1 St. Moritz System Demonstration . . . 33
7.2.2 St. Moritz Measurement Method Validation . . . 34
7.2.3 Zermatt Full-System Demonstration . . . 36
7.3 Limitations and Future Work . . . 39

8 Conclusion **41**

A General Appendix **42**

B Requirements Appendix **51**

C Robotics Platform Appendix **58**

D Software Platform Appendix **67**

E Localisation Appendix **79**

F Path Planning and Control Appendix **94**

G Ice Measurement Appendix **112**

Nomenclature

Acronyms & Nomenclature

Acronym	Definition
AUV	Autonomous Underwater Vehicle
DVL	Doppler Velocity Log
D-EAPS	ETH Department of Earth and Planetary Sciences
EKF	Extended Kalman Filter
GNSS	Global Navigation Satellite System
IMU	Inertial Measurement Unit
MAVLink	Micro Air Vehicle Link
NTP	Network Time Protocol
ROS 2	Robot Operating System 2
SBL	Short Baseline (acoustic positioning)
USBL	Ultra-Short Baseline (acoustic positioning)
PETG HF	Polyethylene Terephthalate Glycol High Flow (3D printing filament)
CoM	Centre of Mass
CoB	Centre of Buoyancy
MCAP	Modular Container for Autonomous Platforms
SITL	Software-in-the-Loop
PPC	Pure Pursuit Controller
MPPC	Modified Pure Pursuit Controller
PID	Proportional–Integral–Derivative (controller)
PD	Proportional–Derivative (controller)
BT	Behaviour Tree
LAD	Look-Ahead Distance
RMSE	Root-Mean-Square Error
PWM	Pulse-Width Modulation
RTK	Real-Time Kinematic (GNSS positioning)
WGS84	World Geodetic System 1984 (coordinate system)
ENU	East–North–Up (reference frame)
FLU	Forward–Left–Up (body-fixed frame)
LAN	Local Area Network
CPU	Central Processing Unit
RAM	Random-Access Memory
LTS	Long-Term Support (Ubuntu release)
QGC	QGroundControl
NIS	Normalised Innovation Squared (filter-consistency statistic)
UKF	Unscented Kalman Filter
ESKF	Error-State Kalman Filter
IEKF	Invariant Extended Kalman Filter
MAP	Maximum A Posteriori (estimate)
RSSI	Received Signal Strength Indicator (DVL acoustic return)
DOF	Degree of Freedom
ESC	Electronic Speed Controller
C/N_0	Carrier-to-Noise-Density Ratio (GNSS signal strength)

Nomenclature	Definition
Python	General-purpose programming language known for its readability and ease of development, though slower than compiled languages. [1]
C++	General-purpose compiled programming language prized for its high efficiency and execution speed. [1]
Docker	Docker is a software platform that allows you to build, test, and deploy software stacks quickly.
Rosbag	ROS 2 specific file format, used for logging.
ArduSub	Open source software designed for underwater vehicles. [2]
nav2	ROS 2 navigation framework used for Behaviour Tree execution and path management. [3]
orca4	Open-source ROS 2 / ArduSub repository used as basis for the simulation and autonomy stack. [4]
Gazebo	Robotics simulation environment used to approximate the vehicle hydrodynamics. [5]
QGroundControl	Open-source ground control station used to read and write the flight-controller parameters. [6]
Foxglove	Visualisation tool used to inspect commanded and measured signals live during tuning. [7]
Pixhawk (6C)	Open-hardware flight controller running the ArduSub firmware. [8]
Behaviour Tree	Hierarchical structure determining which mission phase / plug-in is active.
Pure Pursuit	Geometric path-tracking guidance law. [9]
Gauss–Markov process	First-order stochastic process used to model the simulated water current. [10]
Dead reckoning	Position estimation by integrating odometry in the absence of absolute fixes.
map frame	World-fixed global reference frame.
odom frame	World-fixed local reference frame.
base_link	Body-fixed reference frame of the vehicle.
robot_localization	ROS 2 sensor-fusion package providing the EKF used for both filter instances. [11]
Allan deviation	Time-domain method separating a sensor's white noise from its slowly varying bias. [12]
Bottom-lock	Condition in which the DVL acoustically tracks the lake bed and reports velocity over ground.

Symbols

Symbol	Unit	Description
Ice and buoyancy		
F_g	N	Gravity force
h_{ice}	m	Local ice thickness
h_{draft}	m	Submerged ice draft below waterline
ρ_{water}	kg m^{-3}	Water density
ρ_{ice}	kg m^{-3}	Ice density
p_{CoM}	m	Centre of mass position
p_{CoB}	m	Centre of buoyancy position
Frequencies and timing		
$f_{\text{controller}}$	Hz	Guidance Controller update frequency
$f_{\text{replanning}}$	Hz	Behaviour-tree replanning rate
$T_{\text{completion}}$	s	Mission completion time
τ	s	Time-correlation constant of the simulated water current process
Distances, errors and tolerances		

Symbol	Unit	Description
e_{track}	m	Cross-track (tracking) error from the AUV to the reference path
$e_{\text{overshoot}}$	m	Corner overshoot
e_{φ}	rad	Yaw / heading error w.r.t. the reference path
d_{safe}	m	Safe-distance / replanning threshold
d_{goal}	m	Goal tolerance
d_{offset}	m	Initial lateral offset of the AUV from the reference path for a step response
RMSE	m	Root-mean-square cross-track error
Pure Pursuit geometry and path		
L, LAD	m	Look-ahead distance (geometry)
η	m	Forward (body-frame)
ε	m	Lateral (body-frame)
R	m	Arc / turning radius, $R = 1/\lvert\kappa\rvert$
$R_{\min}$	m	Minimum turning radius (curvature scaling)
κ	m^{-1}	Commanded arc curvature
θ_c	rad	Heading towards the look-ahead point
$\mathcal{P}$	–	Discretised reference path (ordered set of positions) in map frame
$\Delta x,\ \Delta y,\ \Delta z$	m	Per-axis spacing between consecutive path poses
Velocities and accelerations		
$\mathbf{v}_{\text{cmd}}$	m s^{-1} / rad s^{-1}	Commanded velocity vector (FLU frame)
$\bar{v}_x$	m s^{-1}	Commanded surge velocity
$\bar{v}_z$	m s^{-1}	Commanded heave velocity
$\bar{\omega}_z$	rad s^{-1}	Commanded yaw rate
v_x	m s^{-1}	Estimated surge velocity (EKF)
v_z	m s^{-1}	Estimated heave velocity (EKF)
ω_z	rad s^{-1}	Estimated yaw rate (EKF)
$v_{x,\text{cruise}}$	m s^{-1}	Desired cruising speed in surge direction
v_{bias}	m s^{-1}	Deterministic bias of the simulated current
v_{current}	m s^{-1}	Water-current speed
$\mathbf{v}$	m s^{-1}	Estimated velocity vector (EKF)
σ	m s^{-1}	Standard deviation of the current white noise
$a_x^{\max}$	m s^{-2}	Maximum (trackable) surge acceleration
$a_z^{\max}$	m s^{-2}	Maximum (trackable) heave acceleration
$\dot{\omega}_z^{\max}$	rad s^{-2}	Maximum (trackable) yaw acceleration
State estimation and localisation		
$\hat{x}$	–	EKF state estimate (15 states: position, orientation, and their rates)
P	–	State-estimate covariance
$f,\ F$	–	Process (motion) model and its Jacobian $F = \partial f/\partial x$
$h,\ H$	–	Measurement model and its Jacobian (constant selection matrix)
z_k	–	Sensor measurement at step k
K_k	–	Kalman gain
Q	–	Process-noise covariance (vehicle dynamics)
R	–	Measurement-noise covariance, per sensor (cf. turning radius above)
ν	–	Innovation, $\nu = z - h(\hat{x}^-)$
S	–	Innovation covariance, $S = R + HP^-H^\top$
$\hat{\rho}(k)$	–	Innovation autocorrelation at lag k
χ_1^2	–	Chi-squared distribution with one degree of freedom
$\phi,\ \theta,\ \psi$	rad	Roll, pitch, yaw (Euler angles)
N	$\text{m}/\sqrt{\text{s}}$	Allan white-noise (velocity random-walk) coefficient (cf. pose count above)

Symbol	Unit	Description
B	$\mathrm{m\,s^{-1}}$	Allan bias-instability floor
K	–	Allan rate-random-walk coefficient
σ	–	Standard deviation, $\sigma = \sqrt{R}$
h_{acc}	m	GNSS receiver self-reported horizontal accuracy
std_m	m	SBL driver-reported position standard deviation
L	m	Travelled path length, dead-reckoning (cf. look-ahead distance above)
$L_{\max}$	m	Maximum dead-reckoning range before the 2 m error bound
$\varepsilon(L)$	m	Dead-reckoning position error vs. path (cf. lateral path axis above)
m	–	Drift slope (position error per unit path)
b	m	Drift-fit initial offset
η	m	Residual scatter about the drift fit (cf. forward path axis above)
z	–	One-sided confidence factor (e.g. 1.645 at 95 %)
s	–	DVL scale-factor error
α	rad	Constant heading-bias angle
H_{meas}, H_{Earth}	–	Measured / true geomagnetic field vector
A	–	Soft-iron calibration matrix (with hard-iron offset b) in $H_{\mathrm{meas}} = A\,H_{\mathrm{Earth}} + b$

CHAPTER 1

Introduction

1.1 Motivation and Background

Lake-ice thickness and seasonal and inter-annual variations serve as indicators of cryospheric response to global climate change [13, 14]. In alpine ecosystems, high-altitude frozen lakes react sensitively to mid-winter warming events [14]. Beyond long-term environmental modelling, accurate, high-resolution ice-thickness data is needed for safety assessments in winter tourism and alpine logistics. Satellite remote sensing effectively maps surface ice area, but measuring thickness from orbit introduces errors of several tens of centimetres due to snow insulation, surface roughness, and complex ice morphology [15, 16]. These errors fail to meet the 1 to 2 cm precision benchmark set by the Global Climate Observing System (GCOS) for lake ice as an Essential Climate Variable (ECV) [17], leaving alpine lakes under-represented in current climate frameworks [18].

Consequently, local profiling campaigns still rely on manual core drilling or surface-deployed Ground Penetrating Radar (GPR) [19]. These traditional top-down methodologies introduce clear operational and physical constraints. Because these methods require physical access to the ice, data collection becomes impossible when the ice is too thin, especially during the early-winter freezing phase and the spring melting season. This requirement leads to a systematic loss of primary data. Furthermore, varying ambient temperatures can create an unstable, snow-ice layer along the shoreline, introducing significant safety hazards for personnel attempting to access the ice. Additionally, surface-bound methods require extra logistical equipment or infrastructure to safely access unsupported ice shelves that are not connected to land.

Operating an underwater vehicle beneath the ice provides a viable alternative to overcome these surface-bound limitations. While aerial drones are restricted by flight conditions and limited payload capacities, an underwater vehicle enables all-weather data collection and establishes a versatile payload baseline, offering the long-term potential to integrate additional sensor suites for broader sub-ice monitoring.

Project POLARIS was developed in close collaboration with scientific partners from ETH Zurich's Department of Earth and Planetary Sciences (D-EAPS) to address this data gap. By designing a tethered AUV optimised for sub-ice environments, the project establishes a safe, repeatable, and automated profiling methodology.

1.2 Mission Objectives

The mission goal of POLARIS is to develop and deploy a reliable autonomous system capable of generating accurate ice-thickness maps, even in areas where on-ice access is unsafe.
In collaboration with D-EAPS, this high-level goal was translated into four concrete, verifiable objectives that guide the system design:

MO-01: Develop a system capable of ice-thickness measurements with an accuracy of ±1 cm.
Reasoning: This precision matches the Global Climate Observing System (GCOS) benchmark [17]. Maintaining a low uncertainty is essential because variations in early-winter ice formation occur on a centimetre scale, making high accuracy critical for valid structural and safety assessments.

MO-02: Cover an area of 10 000 m^2 to enable datapoint acquisition with a resolution of 10 m.
Reasoning: A 100 m × 100 m grid allows the system to completely map smaller alpine lakes or capture a representative section of larger basins like Lake St. Moritz (a primary focus for D-EAPS), while remaining operationally manageable within physical tether length limitations.

MO-03: Localise system in a global coordinate frame to map datapoints to absolute locations with an average accuracy of ±2 m.
Reasoning: Reliable global georeferencing is necessary to pinpoint structural weak points in real-world coordinates and to enable longitudinal studies where repeated survey grids are executed across the entire ice season. For the scientific frameworks utilised by D-EAPS, an average accuracy of ±2 m is sufficient to correlate these datasets.

MO-04: Autonomous mission completion, including navigation, data acquisition, and safe return to the recovery location.
Reasoning: Manual tracking of a dense measurement grid under ice is impractical. External forces cause the vehicle to drift during static measurements, and navigating the narrow water column between the lake bed and the ice ceiling requires active stabilisation; coupling position- and depth-hold into a fully autonomous system guarantees the spatial repeatability required for scientific modelling.

1.3 Requirements

To bridge the gap between the high-level Mission Objectives and the physical development of the vehicle, the requirements follow a top-down hierarchy. The four Mission Objectives act as our ultimate design constraints, from which the two technical layers are derived.
For the scope of these requirements, the *system* comprises all physical and digital components necessary to complete the mission. This includes the hardware platform, the topside base station, and all software frameworks executing for mission completion.

1.3.1 Mission-Level Requirements (MR)

Mission-Level Requirements translate the Mission Objectives into operational boundaries without prescribing the physical vehicle architecture:

- **Ice Thickness Mapping:** The system shall consecutively profile ice thickness with an absolute accuracy of ±1 cm between 0 cm and 100 cm, and a relative accuracy of ±1 % for thicknesses exceeding 100 cm, generating a georeferenced thickness map across the deployment area (MR-01, MR-02, MR-06).
  Reasoning: Centimetre-scale precision during early freezing phases is critical for safety assessments and climate models. For thicker layers, relative tolerance satisfies scientific modelling constraints. Fusing consecutive data points yields complete spatial coverage.
- **Localisation and State Estimation:** The system shall determine the position of all data points within an absolute global accuracy of ±2 m while maintaining an onboard estimate of vehicle pose and motion (MR-04, MR-05).
  Reasoning: Global referencing is required to construct spatial map, and to compare survey grids across different seasons. High-frequency onboard estimation is functionally required to provide the path tracking algorithms with real-time feedback.
- **Mission Survivability and Recovery:** The system shall sustain all critical functions for the entire mission duration including an operational safety margin under ice, and maintain physical recovery compatibility under all conditions (MR-03, MR-08).

Reasoning: The runtime must fully cover the spatial survey track with stops and an additional operational margin. If a total power failure or system fault occurs beneath the ice, the vehicle must allow direct physical retrieval via the tether.

- **Autonomous Navigation:** The system shall autonomously complete the mission, including navigation, data acquisition, and safe recovery (MR-07).
  Reasoning: Manual tracking and navigation of dense measurement grid under ice is impractical. Automated control also helps with spatial repeatability.

1.3.2 System-Level Requirements (SR)

System-Level Requirements define the quantitative technical targets assigned to each engineering domain to satisfy the mission-level boundaries. Having selected an AUV as platform, these specifications define the performance constraints for each subsystem:

- **Ice Measurement** (SR-ICE-01 to SR-ICE-03): Constrain the spatial and vertical measurement boundaries of the profiling instruments. They define the mandatory accuracy limits for ice thickness mapping. Detailed in Chapter 7.
- **Localisation** (SR-LOC-01 to SR-LOC-05): Dictate the accuracy and frequency thresholds required for reliable vehicle state estimation. They define horizontal and vertical station-keeping tolerances to prevent data errors during sampling, set global tracking limits, and mandate absolute position updates to bound dead-reckoning drift. Detailed in Chapter 5.
- **Hardware Platform** (SR-HW-01 to SR-HW-09): Establish the physical, environmental, and structural constraints of the vehicle. They bound the operational speed, minimum depth rating, mass limits, and environmental endurance needed to survive under-ice missions, while mandating passive righting moments and a load-bearing recovery interface for surface retrieval. Detailed in Chapter 3.
- **Software Platform & Autonomy** (SR-SW-01 to SR-SW-15): Outline the architectural frameworks, communication baselines, and path-following tolerances of the software stack. They constrain the system to a modular, containerised architecture with deterministic safety disarm loops, while setting the maximum allowable cross-track, depth, and ascent velocity errors under dynamic fluid disturbances. Detailed in Chapter 4 and Chapter 6.

Detailed individual requirements and engineering reasoning are provided in Section B.2 (mission-level requirements) and Section B.3 (system-level requirements by subsystem), while their empirical fulfilment is analysed within the respective technical chapters.

CHAPTER 2

Field Testing and System Overview

2.1 Testing Sites

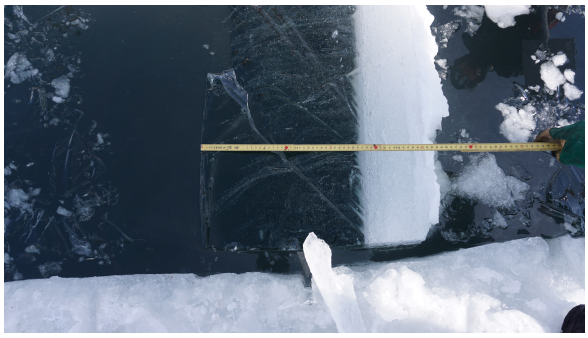
The developed system was validated across four distinct field locations, each selected to progressively verify system performance under increasingly demanding environmental conditions.

Pool: Initial commissioning of the integrated vehicle was carried out in an outdoor pool under calm, clear water conditions with no current, allowing direct visual inspection of the vehicle at all times. This allowed direct verification of enclosure seal integrity, hydrostatic buoyancy, motor direction.

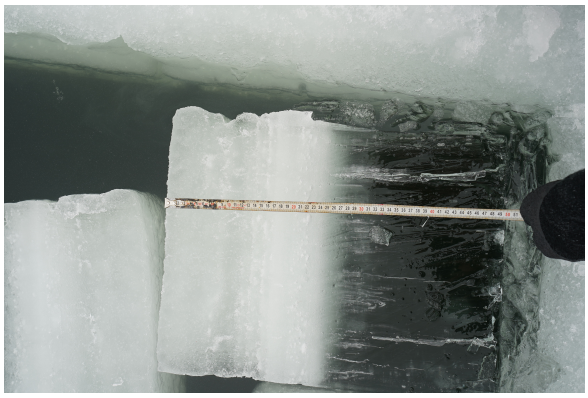
Lake Zurich: Prior to any under-ice deployment, the fully integrated system was validated under real-world open-water conditions on Lake Zurich. Tests were conducted in ice-free, open freshwater with natural current and wave disturbances. The Extended Kalman Filter tuning, emergency stop system, and end-to-end autonomous mission execution were verified.

Lake St. Moritz: The first under-ice campaign was conducted beneath a homogeneous, structurally uniform freshwater ice cover of representative thickness, providing ideal baseline conditions for sensor calibration and the validation of the measurement procedures. Water conditions were clear and near-freezing, with sub-zero ambient air temperatures at the surface. The acoustic SBL localisation link through ice, GNSS fix acquisition and the Archimedes-principle-based ice-thickness measurement pipeline were validated for the first time under operational conditions.

Schwarzsee, Zermatt: Full mission capability was demonstrated under the most demanding and representative conditions of the project. In contrast to Lake St. Moritz, the ice cover at Schwarzsee exhibited a complex multi-layered structure, comprising alternating strata of solid ice and partially refrozen meltwater layers, posing a significantly greater challenge for the ice-thickness measurement pipeline. Tests were conducted at an altitude of approximately 2580 m, with persistent ice cover, low ambient temperatures, and remote logistics. All primary mission objectives were subject to definitive validation at this site, with the exception of ice thickness measurement. The latter had been validated during previous field campaigns and was deemed logistically impractical to verify under the current operational conditions.



(a) Homogeneous ice at Lake St. Moritz.



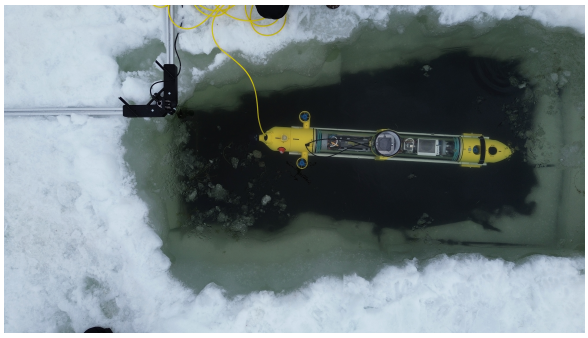
(b) Stratified ice layers at Schwarzsee, Zermatt.

Figure 2.1 Morphological comparison of ice conditions: The structural discontinuity at Schwarzsee (right) highlights the complexity faced by the measurement pipeline compared to the homogeneous conditions at Lake St. Moritz (left).

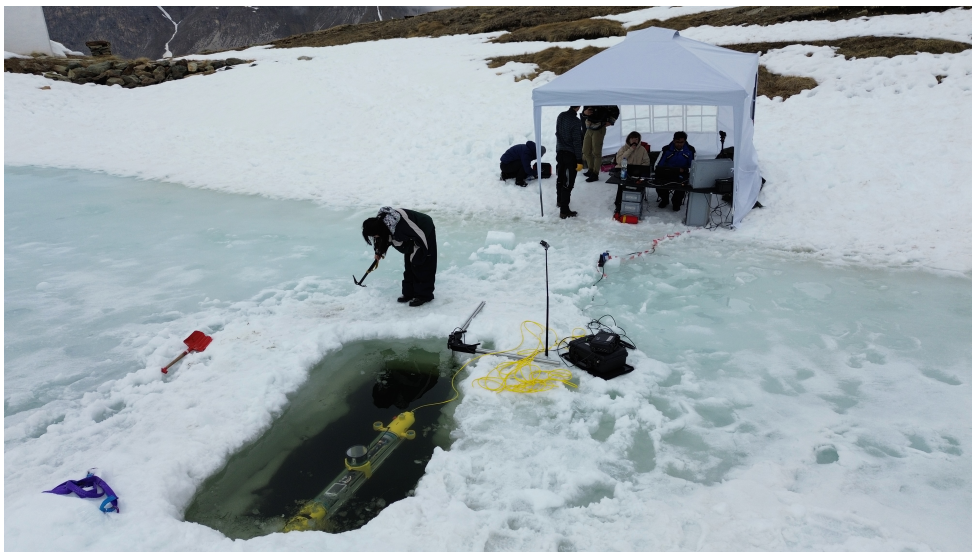
2.2 Operational Setup

The field deployment setup separates the command-and-control equipment from the water. Positioning the base station on a stable surface away from the ice access hole protects the electronics and provides a controlled environment for mission management (Figure 2.2a).
The vehicle deployment process is illustrated in Figure 2.2b. The AUV is launched through an opening in the ice. To ensure accurate underwater tracking, the SBL transceiver is mounted on a pole directly at the edge of this access hole. This configuration maintains an optimal acoustic line-of-sight to the AUV, which is necessary as radio signals do not penetrate ice or water.
A neutrally buoyant tether connects the AUV to the surface interface modules. This tether serves as a physical retrieval link, providing a reliable means to locate and recover the vehicle in the event of a system failure or mission anomaly. It is secured at the surface to prevent mechanical strain on the electrical connectors and to minimise the risk of entanglement beneath the ice.
The operational site layout is shown in Figure 2.2c. This overview displays the operator tent, the base station, and the deployment area. Operational procedures involve personnel stationed within the tent to monitor telemetry and manage system commands, while additional team members are positioned at the ice access hole to oversee vehicle deployment and ensure the integrity of the setup during operation.

(a) Base station featuring the mission control computer and the acoustic localisation master unit for acoustic telemetry.



(b) AUV deployment through the ice access hole with the SBL transceiver mounted on a pole at the hole edge.



(c) Site overview showing the operator tent, central command table, and the vehicle deployment area.

Figure 2.2 Operational setup during the under-ice field campaigns: topside base station (a), vehicle deployment procedure (b), and overall site layout (c).

CHAPTER 3

Robotics Platform

Developing a physical platform capable of enduring the underwater world presents severe engineering challenges: from combating pressure and preventing water ingress to ensuring optimal weight distribution and thermal management, the robotics platform serves as the foundation for overall mission success. The robotics platform is the part where all separate components are integrated to form a complete system. The platform is driven by system requirements that include an ingress protection rating up to a depth of 10 m (SR-HW-02), maintaining slight positive buoyancy (SR-HW-03), a total platform weight limit of 30 kg (SR-HW-06), and providing continuous system power for a minimum of 180 min (SR-HW-08). The answer is a dual-enclosure structural layout: a primary pressure vessel that houses the core electronics, power distribution and internal thermal management systems, and a flooded fairing that isolates external sensors, thrusters and acoustic transducers.

3.1 Mainframe — Structures and Propulsion

3.1.1 Hull

For the structural frame, 3D printing was selected to realise the complex geometries of the skeleton. Polyethylene Terephthalate Glycol High Flow (PETG HF) was chosen for its mechanical properties, environmental resistance, cost-efficiency, and good printability, which enabled rapid and precise design iterations. The material provides a tensile strength of around 35 MPa, a stiffness of 1800 MPa, and prevents embrittlement in cold operating environments down to −20°C [20, 21]. Additionally, its low water absorption rate (0.15 to 0.3%) ensures dimensional and chemical stability during wet operation [22]. To prevent water ingress through layer lines, submersion tests were conducted (see Appendix C). The results showed that an increase in perimeter thickness seals the internal structure. Therefore, a slicing strategy with 8 wall loops was implemented to minimise water penetration into the printed parts. Standard modules utilised a 30% infill density to optimise weight and printing duration, while high-stress components were reinforced with 50% infill. Due to the high manufacturing precision of the Bambu Lab X1 Carbon printers, post-processing was limited to the removal of the support structures.

3.1.2 Weight and Buoyancy System

To control the spatial relationship between the CoM and the CoB, a pre-mission adjustable ballast system was implemented. The system consists of modular concrete weights free to slide on the aluminium profiles located at the lowest axis of the skeleton. Concrete was selected due to its cost-efficiency and ease of modular fabrication. Each concrete weight is secured via bolts, allowing continuous repositioning along the rails for static trim adjustments. The final operational configuration utilises three primary central weights and two smaller lateral side weights. The trim strategy focused on vertical separation between CoM and CoB to establish a passive restoring moment against roll and pitch displacements. Due to the concentration of the concrete ballast at the lowest position, the CoM is located geometrically below the CoB. This vertical offset provides inherent passive roll and pitch stability. The ballast components were repositioned iteratively along the aluminium rails until horizontal equilibrium was achieved. Tuning was finalised to a state of near neutral buoyancy, where the vehicle remains submerged with only the uppermost 1 cm of the structure breaching the water surface.

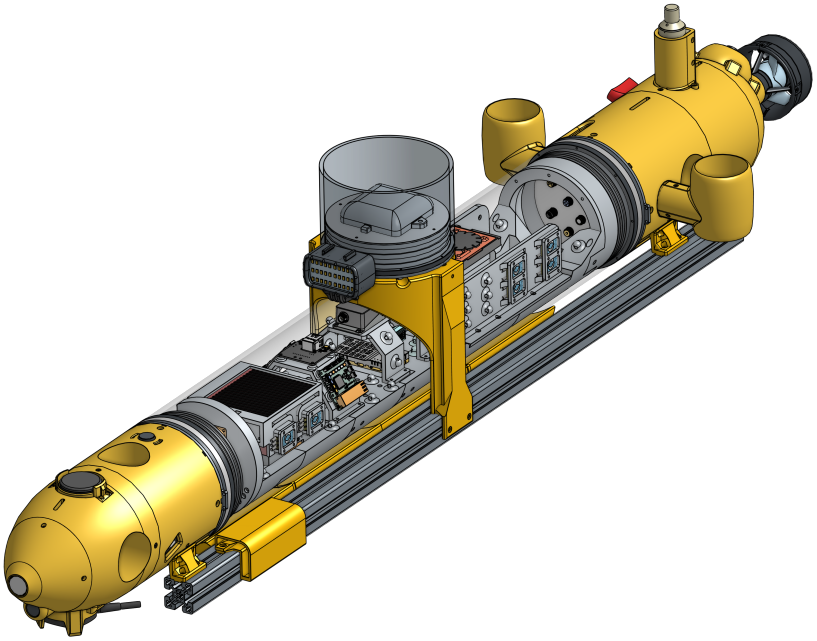


Figure 3.1 Computer-aided design rendering showing the latest AUV design.

3.1.3 Thruster Layout and Component Distribution

To achieve full independent 6-DoF manoeuvrability, a six-motor configuration is utilised: three vertical thrusters for roll, pitch, and heave control, and three horizontal thrusters for surge, sway, and yaw. Figure 3.1 visually summarises this thruster arrangement. This layout provides agility whilst retaining a streamlined, tubular chassis design that minimises hydrodynamic resistance. To address corrosion issues, all exposed connectors are sealed with epoxy.

3.1.4 Tether and Strain Relief

For the link between the AUV and the top station, the system supports two interchangeable tether configurations: a 25 m Ethernet cable for short-range operations and 200 m fibre optic cable for long-range missions. To protect the sensitive, waterproof electrical enclosures from mechanical tension, a strain relief mechanism is integrated at the rear of the vehicle. This system utilises a friction based ball clamp secured directly onto the cable, ensuring that operational pulling forces are absorbed by the vehicle's chassis rather than the connectors.

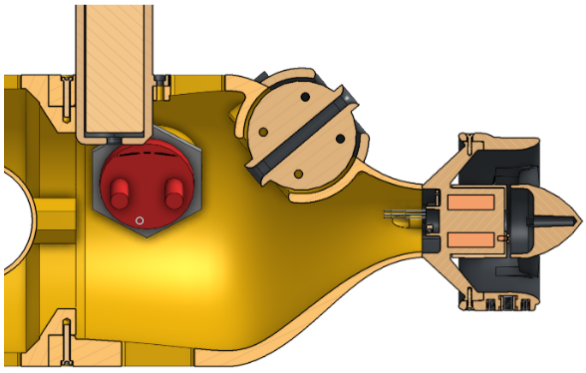


Figure 3.2 Section view of the vehicle's back module showing the tether strain relief mechanism.

3.2 Electronics Tube and Watertight Interfaces

3.2.1 Pressure Vessel and Watertight Interfaces

The primary pressure vessel is a cast polymethylmethacrylate (PMMA) cylinder with an inner diameter of 150 mm. PMMA was selected over aluminium for its optical transparency: the internal electronics assembly can be visually inspected for condensation, arcing, or loose connections without opening the enclosure.

The pressure vessel is closed at both ends by aluminium flanges inserted into the PMMA cylinder. Each flange incorporates two radial O-ring grooves, providing independent sealing stages between the flange and the cylinder wall. An external aluminium connector plate is attached to the outer face of each flange. A dedicated O-ring between the flange face and connector plate seals this interface. Bluetrail watertight connectors are mounted in M10 clearance holes machined into the connector plates, providing feedthroughs for electrical power and communication signals across the pressure boundary. This arrangement allows connectors to be mated and demated from outside the vehicle without disturbing the primary pressure-vessel seals.

The rear connector plate accommodates twelve connectors, carrying thruster power lines, the main power-switch harness, and external sensor cabling, together with a dedicated vacuum port. The front connector plate accommodates eight connectors for forward-facing sensors and communication interfaces.

3.2.2 Internal Electronics Layout

The internal electronics assembly is mounted on a sled attached to the rear flange, allowing all components to be extracted as a single unit. The main components carried by the sled are summarised in Table 3.1, arranged from front to rear by function.

Thermal management is entirely passive. At the sub-zero ambient temperatures encountered during under-ice operations, the heat generated by the compute hardware, power converters and ESCs is sufficient to maintain the internal air above the minimum operating temperature of each component. No active cooling or heating is required, reducing system complexity and eliminating parasitic power consumption.

Table 3.1 Components of the internal electronics sled.

Subsystem	Component	Function
Compute & Control		
Compute	NVIDIA Jetson AGX Orin	Onboard compute
Sensing	XSens Sirius IMU	Inertial sensing
Control	Pixhawk 6C	Flight control
Propulsion		
Distribution	PWM distribution board	Command distribution
Actuation	Six ESCs	Motor control
Power		
Conversion	Two DC/DC buck converters	Power conversion
Management	Battery Management System (BMS)	Battery management
Storage	Two 22 A h 4S LiPo cells	Energy storage
Communications & Interfaces		
Networking	GigaBlox SFP six-port Ethernet switch	Network switch
Serial	RS-232-to-USB and RS-485-to-USB converters	Legacy interfaces
UART	Multi-port UART hub	UART expansion
Timing		
Timing	Battery-backed real-time clock	Timekeeping

3.3 Sensing

To support autonomous under-ice operations, the AUV integrates a heterogeneous sensor suite providing the primary data layer for flight control, state estimation, and ice mapping. This section establishes the physical baseline, mounting configurations, and raw data streams, serving as the direct foundation for the localisation framework developed in Chapter 5 and the ice-thickness calculation detailed in Chapter 7.

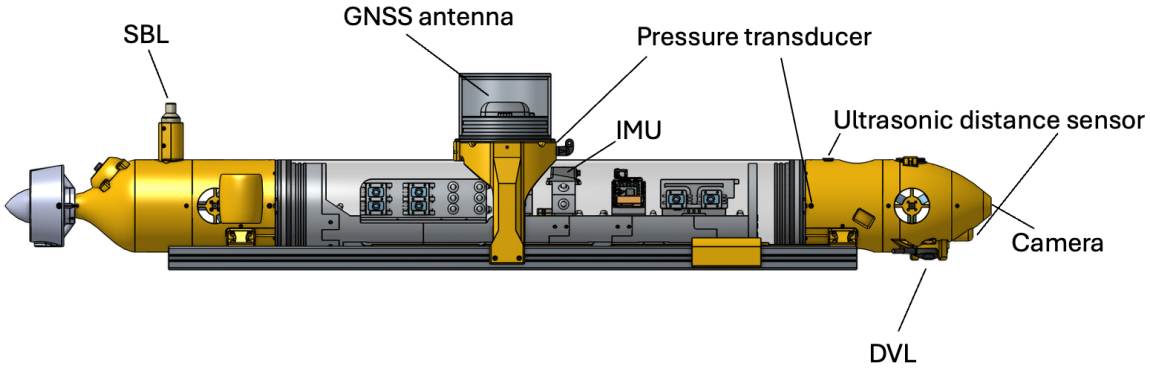


Figure 3.3 Physical sensor layout on the AUV.

The sensory hardware is structured into global positioning systems and dead-reckoning or boundary-tracking instruments.

Global Positioning Systems

- **Short Baseline Acoustic Positioning System (SBL):** A Water Linked Locator U1 transmitter is mounted on the upper aft hull, emitting acoustic pulses to a stationary surface receiver array. The topside station processes these signals to compute global coordinates, transmitting them back via the tether link. While the system features a horizontal range accuracy of <0.2 %, its horizontal angular uncertainty of <1° dominates the tracking geometry [23]. This causes the total position error to scale at approximately 1–2% of the slant range, acting as a low-frequency absolute reference (Chapter 5). However, adverse acoustic boundaries and geometric degradation near ice holes or shallow zones can still induce transient coordinate jumps of several metres.
- **Global Navigation Satellite System (GNSS):** A u-blox RTK GNSS receiver patch antenna is housed inside an air-filled, vertical cylindrical tower module. This internal air volume prevents frequency detuning caused by the high relative permittivity of water. During open-water surface phases, the system achieves nominal centimetre-level accuracy and a stable RTK lock, even when the antenna element itself is below the external ambient waterline. When operating beneath the ice sheet, performance is primarily determined by the composition of the overhead layers. Field trials under clear ice demonstrate that the system successfully resolves usable sub-metre absolute coordinates, providing a more accurate absolute reference than the acoustic SBL. However, performance degrades severely under slush ice, which is a water-ice mixture beneath or within the sheet [24], often blocking the satellite signal entirely or increasing positioning errors to several metres.
- **Hydrostatic Pressure Transducers:** Two separate sensors decouple navigation loops from scientific mapping to eliminate vertical lever-arm estimation errors:
  - *Navigation Depth:* A Blue Robotics Bar30 sensor is mounted on the electronics tube end cap. Zeroing the sensor in the air before the dive removes atmospheric pressure offsets. While prone to thermal drift, its high resolution of 0.2 mbar (2 mm) [25] provides the fast, low-latency feedback needed for the depth control loops (Chapter 5).
  - *Profiling Depth:* A scientific-grade Keller PR-26X piezoresistive level probe is mounted at the tower base, aligned with the vertical contact axis. Featuring a nominal accuracy of 0.1 % Full Scale Output (FSO) and integrated mathematical temperature compensation [26], this instrument mitigates thermal drift to deliver the stable, sub-centimetre vertical reference required for the ice-thickness calculations in Chapter 7.

Dead Reckoning and Boundary Tracking

- **Inertial Measurement Unit (IMU):** An XSens Sirius MEMS IMU is mounted near the vehicle's centre of mass. The sensor data stream is sampled at 100 Hz [27] to drive the high-frequency EKF state prediction step (Chapter 5). The internal magnetometer is disabled due to severe magnetic interference.
- **Doppler Velocity Log (DVL):** A Water Linked DVL-A50 performance is flush-mounted downward within the forward lower hull. It tracks body-frame linear velocities relative to the lake bed with a specified long-term accuracy of ±0.1 % and an adaptive ping rate of 4 Hz to 15 Hz [28], supplying relative velocity updates for the state filters. Excessive vehicle pitch or roll can temporarily break acoustic lock by tilting the transceiver geometry away from the lake bed.
- **Ultrasonic Distance Sensors:** Two acoustic distance transducers track localised boundaries. One sensor faces vertically upward to measure the distance to the overhead ice sheet for ceiling-proximity tracking and non-contact profile mapping (Chapter 7). The second sensor faces directly forward to support real-world obstacle detection and reactive collision avoidance (Chapter 6).

3.4 Limitations and Future Work

The current robotics platform carries several structural, mechanical, and electrical constraints. Structurally, while the ballast mass is essential to counteract positive buoyancy, the modular aluminium weight rails introduce unnecessary structural bulk and volume. Mechanically, the plastic endcap connector remains vulnerable to shear failure when pulled at an angle, and tight internal routing paths risk cable pinching during vehicle assembly. Electrically, standard buck converters expose the system to voltage fluctuations as the batteries discharge. Finally, the platform remains sensor-limited: the upward camera faces blind spots behind the central sensor tower, and the fibre optic interfaces remain highly susceptible to mechanical signal interference.

Several clear directions exist for short-to-medium term development, alongside longer-term optimisation goals. In the near term, the standard remedy for power delivery constraints is to integrate Power buck-boost regulators onto a custom PCB to guarantee stable 5 V and 14.8 V rails, paired with high-capacity 22 A h Swaytronic batteries and a front I2C header block to eliminate cable pinching. The plastic endcap should be redesigned for multi-angle loads and a the upward-facing camera place at a different position for un-obscured ice tracking. Long-term improvements could focus on removing the modular rails for a downsized, fixed-mass setup, and engineering an active buoyancy control unit to maximise energy efficiency. Future iterations could also utilise a hot-swappable battery compartment, transition away from noisy USB protocols, and adopt a specialised tube composite to mitigate solar thermal loading on deck.

CHAPTER 4

Software Platform

The software stack is what gives the vehicle its autonomy, handles sensor data and enables remote operability and safety features. Key requirements driving its design include modular, maintainable code via ROS 2 nodes (SR-SW-07), failsafe behaviour on communication loss (SR-SW-04), and time-synchronised data logging for post-mission analysis (SR-SW-05). Docker deployment ensures reproducibility across development and field environments (SR-SW-08), while a tether to a topside computer provides live data streams and the option of manual override (SR-SW-01), resulting in a stack that prioritises safety, modularity, and easy maintenance.

4.1 Architecture

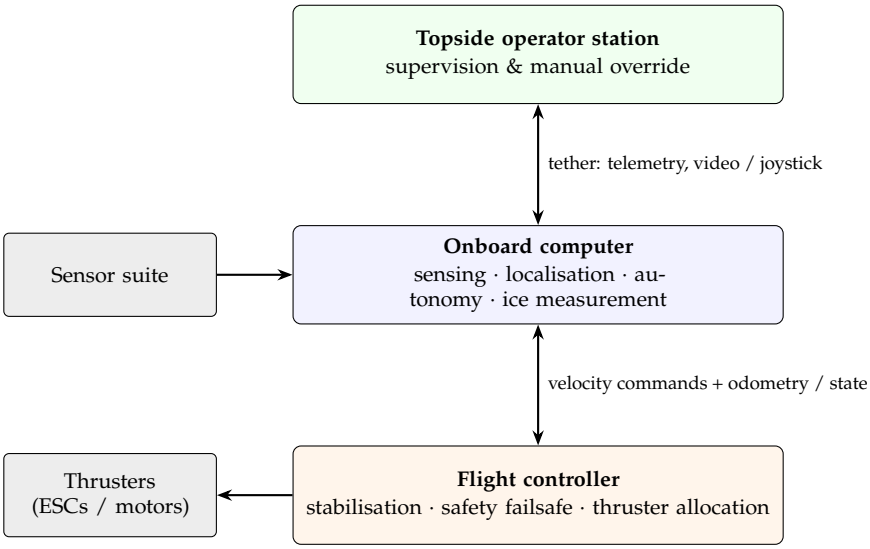


Figure 4.1 Three-tier software architecture. The system partitions high-level planning, real-time control, and supervisory stacks across distinct tiers. The onboard computer processes data from the sensor suite, while the flight controller dispatches commands to the thrusters and ESCs.

The software stack is structured to separate computation across an onboard computer and a flight controller, isolating critical software loops. This ensures that the position and attitude-rate control loops, running at up to several hundred Hz on the flight controller, are not affected by CPU load spikes or non-deterministic scheduling on the onboard computer. The flight controller is also capable of maintaining vehicle orientation and triggering failsafes (Section 4.1.2) should the onboard computer fail. In turn, the onboard computer provides sufficient computational power and ports to service the entire sensor suite and support higher-level tasks such as state estimation and mission planning.
As the vehicle cannot be observed by the operator once submerged below an ice sheet, telemetry and camera feeds are streamed over the tether to the topside computer, forming a third software tier. There, a Foxglove-based visualisation (Section D.7) enables human monitoring of safety-critical quantities such as battery state and estimated vehicle position. The topside computer is additionally used to relay manual

joystick inputs to the onboard computer, completing the architecture illustrated in Figure 4.1.
A Docker container is used to handle all dependencies and ensure a consistent environment from development to deployment on the onboard computer. Bringup is driven by a single master launch file, so that the full stack is started from a fixed launch configuration on every deployment. Together, the container and the launch file guarantee a reproducible starting state. The container configuration and the full bringup sequence are detailed in Section D.2 and Section D.3 respectively.
All sensor and telemetry data, with the exception of camera streams, is logged in Rosbag files stored on the onboard computer. These can be replayed to reproduce the original message flow and timing, allowing for debugging and continued development without access to the vehicle.

4.1.1 Onboard Computer

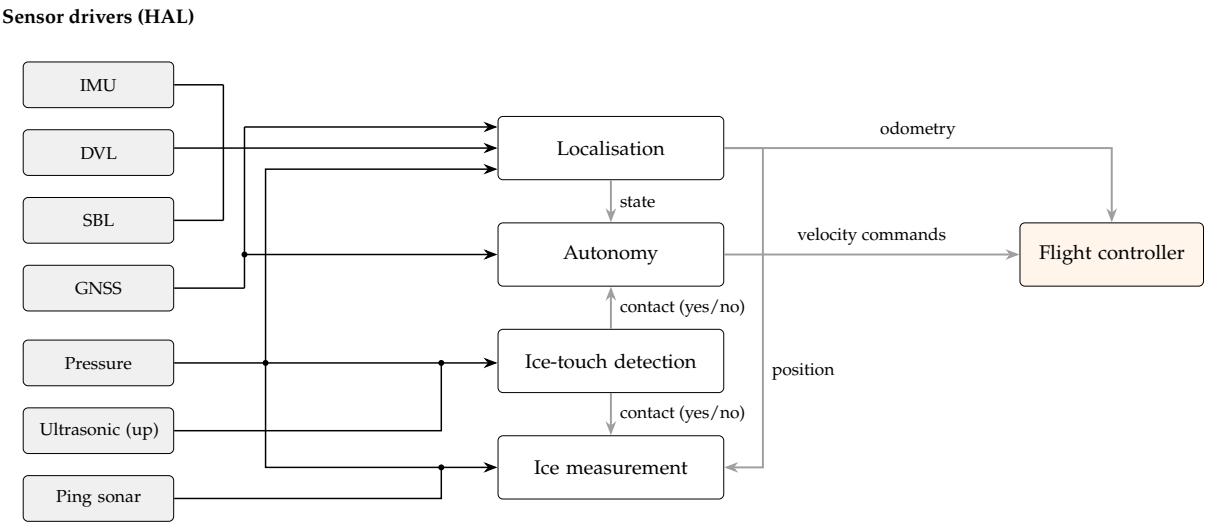


Figure 4.2 Onboard ROS 2 node groups and sensor routing. Sensor drivers (left) feed the functional groups that consume them, which in turn pass velocity commands and odometry to the flight controller. The localisation estimate drives the autonomy stack and feeds ice measurement alongside the ice-touch detection contact state (Section 7.1). At each measurement point, the GNSS fix additionally governs how long the vehicle dwells before continuing (Section F.1).

The onboard computer is an NVIDIA Jetson AGX Orin running a ROS 2 software stack covering sensor drivers, state estimation, autonomy, and ice measurement processing. The stack's modular publisher/subscriber model decouples the individual functional components, facilitating parallel development (SR-SW-07). ROS 2 also provides a mature robotics ecosystem, allowing the integration and adaptation of established open-source packages such as `robot_localization` for state estimation (Chapter 5) and `nav2` for path planning and control (Chapter 6).
Each sensor is integrated through a dedicated ROS 2 driver. Downstream nodes consume sensor data purely through topic and message type, forming a uniform hardware abstraction layer across the sensor suite. All nodes share a common time base, synchronised via the topside network (NTP) over the tether. The resulting node structure and sensor routing are shown in Figure 4.2, while the complete set of packages is catalogued in Section D.1.

4.1.2 Flight Controller

The flight controller is a Pixhawk 6C running ArduSub [2], an open-source firmware purpose-built for underwater vehicles that provides stabilisation loops and a failsafe system out of the box. Within the flight controller, commanded references are converted into body-frame forces and torques. These are then mapped to individual thruster commands via a custom mixing matrix, accounting for the vehicle's thruster geometry (Section D.4). The resultant commands are finally output as PWM signals to the ESCs. Commands are exchanged between the onboard computer and the flight controller via the MAVLink protocol [29]. The translation from ROS 2 commands to MAVLink is handled by a dedicated bridge node, detailed in Section D.5. To achieve better reference tracking, odometry from the onboard localisation stack is also forwarded to the flight controller.

The flight controller supports several operating modes. In `MANUAL` mode, the attitude stabilisation loops are bypassed. This grants direct manual control of the vehicle through a physical PS5 controller (SR-SW-06). Assisted modes and fully autonomous operation are also supported, detailed in Section D.6 and Chapter 6 respectively.
An independent safety layer is provided through the separate failsafe behaviour, one of the key motivations for the control split. During regular operation, the flight controller receives heartbeats at 1 Hz from the onboard computer. If these are missed for 5 s due to an onboard computer failure, the ArduSub failsafe is triggered and the flight controller disarms automatically (SR-SW-04). As a secondary measure, a ROS 2 shutdown also disarms the flight controller. In the case of a flight controller failure, thrusters remain unactuated and the system can be recovered via the tether.
To ease development, ArduSub provides a software-in-the-loop (SITL) environment. Within it, the flight controller's behaviour was simulated and the autonomy stack validated (Section 6.2) prior to field testing.

4.2 Limitations and Future Work

The current software stack carries several constraints inherited from its reliance on third-party packages. The `robot_localization` ROS 2 package, though well-established, offers limited customisability for the state estimation requirements, as further elaborated in Chapter 5. The codebase also grew in complexity over the project duration, and in its current state introduces unnecessary computational overhead and reduces readability for new contributors. A further limitation lies in the logging infrastructure: rosbags and ArduSub logs currently exist as separate data streams, complicating post-mission analysis and debugging workflows. Furthermore, the logs are currently stored solely on the onboard computer's local storage, with no redundant offload, meaning a catastrophic failure could result in total log loss.

Several clear directions exist for future development. A unified logging pipeline, integrating ArduSub logs into rosbags and synchronising the stored data live to the topside computer, would significantly lower debugging efforts and mitigate the possibility of losing important logfiles in a worst-case scenario of severe water damage.
Longer term, a full ROS 2 simulation environment integrated directly into the main codebase would accelerate development and further reduce dependence on field testing for software validation.
Broader goals include a general productisation effort: simplifying the codebase, transitioning to C++, releasing as open source, and improving usability across the operator and developer interfaces.

CHAPTER 5

Localisation

The AUV can only georeference its ice-thickness measurements and navigate between survey points if it knows where it is, and under the ice this is the central difficulty. Once the vehicle submerges, satellite positioning is unavailable (Section 3.3), so a globally consistent estimate must be held from noisy onboard sensors with only intermittent absolute fixes. The subsystem must satisfy two mission requirements: MR-04, that every measurement point be located to within ±2 m, and MR-05, that it provides the state autonomous operation needs. These flow down to system requirements that include a ±2 m horizontal accuracy (SR-LOC-02) and periodic global fixes to bound the accumulated dead-reckoning drift (SR-LOC-04), with the full set given in Chapter B. The response is a pair of extended Kalman filters: a local filter runs continuously to supply the high-rate position, velocity and orientation that the controller and autonomy stack of Chapter 6 build on, while a global filter additionally fuses the intermittent GNSS and acoustic-SBL fixes to keep the estimate georeferenced.
Between fixes the local estimate dead-reckons at about 0.42 m per 100 m, leaving roughly 400 m of travel before a fix is needed to stay within the ±2 m budget (Section E.3). The global filter supplies that reset and keeps the georeferenced error bounded over the mission. Section 5.1 sets out the estimator architecture, Section 5.3 its experimental evaluation, Section 5.4 the behaviour of the individual sensors under ice, and Section 5.5 the limitations and future work.

5.1 Architecture

The AUV estimates its state online with an extended Kalman filter (EKF). It fuses the vehicle's sensors, which differ widely in rate and noise (Section 3.3), into a single recursive estimate. That estimate is cheap enough to run continuously on the onboard computer, and its tuning and failure modes are well understood. The filters are built on the `robot_localization` package rather than a custom implementation, which supplies a validated EKF with a configurable sensor-fusion interface. A fuller justification of the EKF choice, the package, and the EKF formulation is given in Sections E.1 and E.2.
The two instances, shown in Figure 5.1, differ in their reference frame, their rate, and the aiding available to each. The local instance works in the `odom` frame at 30 Hz, above the 20 Hz odometry rate required by SR-LOC-03, fusing the IMU orientation and angular rate, the DVL velocity and the depth. Its linear acceleration is not fused, since the DVL supplies velocity directly and the accelerometer is markedly noisier. The global instance works in the georeferenced `map` frame at 10 Hz from the same measurements, and additionally fuses the intermittent absolute fix: a GNSS fix at the surface of the entry hole or at the underside of the ice, or an acoustic SBL fix when submerged. The first accepted fix defines the origin of the `map` frame, and thereafter the estimate is corrected only when a fix arrives. The two instances run independently, with the local estimate not fed into the global filter, for reasons detailed in Section E.2.3.
The frames follow the standard ROS 2 convention (REP-105): a fixed `map` frame, a continuous `odom` frame, and the body frame `base_link`. The local instance publishes the `odom` → `base_link` transform and the global instance the `map` → `odom` transform, so the georeferenced pose is read as the composition `map` → `odom` → `base_link`. Each measurement is fused relative to its own timestamp rather than its arrival time (Section E.2.4).

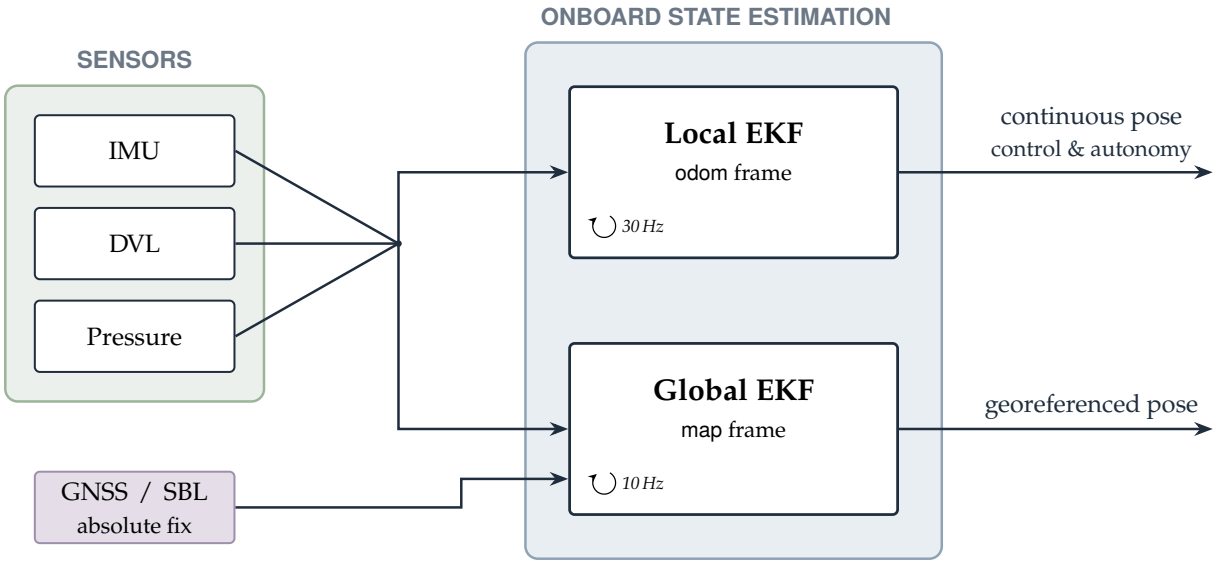


Figure 5.1 High-level two-instance EKF architecture. The proprioceptive sensors (IMU, DVL, pressure) feed both filters, which run in parallel. The local filter (`odom` frame, 30 Hz) supplies the continuous pose used by the controller and the autonomy stack; the global filter (`map` frame, 10 Hz) additionally fuses the intermittent absolute fix (GNSS at the surface, acoustic SBL when submerged) to keep the estimate georeferenced. The local estimate is not fed into the global filter (Section E.2.3).

5.2 Setup and Calibration

Both filters obtain their orientation from the IMU, which runs in gyro-only mode because its magnetometer is disabled (Section 3.3). The evidence for rejecting the magnetometer as a heading reference on this vehicle is given in Section E.6. Heading is therefore carried on the gyro alone, which drifts slowly with manoeuvring and has no absolute reference of its own (its yaw is arbitrary at power-on).

Because the gyro yaw is referenced to an arbitrary power-on direction, it is tied to an absolute bearing by a one-off calibration before the estimate is georeferenced. The vehicle is driven in a straight line at the surface, and the bearing between two GNSS fixes along that run gives the true heading of travel. The offset between that bearing and the IMU yaw over the same interval is applied as a fixed yaw correction, and the corrected orientation stream feeds both filters. This step matters because heading error couples directly into the dead-reckoned position: a 1° yaw error rotates a 1 km path by about 17 m. That is large against the ±2 m budget of Section E.3, so even a small constant heading bias would dominate the error. A dual-antenna GNSS installation that measures heading directly would make this manual calibration obsolete (Section 5.5).

Depth is referenced to the surface: the navigation pressure sensor (Section 3.3) is zeroed against the surface pressure at deployment, so the fused vertical state is measured from the water surface. It is specified to ±2 cm accuracy, which meets the 2 cm vertical tolerance the vehicle must hold station to (SR-LOC-01). Its short-term noise is far finer (Section 5.3.1). The acoustic sensors require a further step: the speed of sound used by the DVL and the SBL is set through their configuration interface before the mission, since both convert acoustic travel time into velocity and range using that value.

5.3 Estimator Validation in Open Water

This section evaluates the localisation stack against independent references. The integrated evaluation of dead-reckoning drift and global correction (Section 5.3.2) requires a continuous absolute reference as ground truth, which only surface GNSS provides. It is therefore conducted in open water rather than under ice, where no such reference exists, while the under-ice behaviour of the individual sensors is characterised separately in Section 5.4. Both are preceded by a characterisation of the sensor noise (Section 5.3.1) that sets the filter's measurement model. Each test follows the same structure: introduction, materials and methods, results and discussion, and conclusion.

5.3.1 Static Sensor-Noise Characterisation

Introduction

The EKF weights each measurement by its measurement-noise covariance R. This is a single diagonal matrix in which each sensor channel occupies its own entry. The off-diagonal terms are zero, so the channels are assumed mutually independent and each is set by that sensor's own noise. This test measures the noise of the IMU, DVL, and pressure sensor to fix those entries. The deployed values are kept deliberately conservative, set at or slightly above the measured noise, which favours filter stability over the fastest possible tracking.

Materials and Methods

To capture sensor noise rather than platform motion, the DVL was removed from the vehicle and fixed rigidly in the water, where it kept continuous bottom lock, while the IMU was logged on the AUV resting on land. Both were held still for 64 minutes. The pressure sensor was instead recorded while floating freely, and the slow water motion this introduces is separated from its sensor noise in the analysis. The magnitude and character of each channel's noise, white sensor noise as opposed to slowly varying bias, are obtained from its Allan deviation. The full curves, per-channel parameters, and resulting R values are given in Section E.7.

Results and Discussion

All three sensors are quiet enough for the filter to trust. The DVL, the filter's primary velocity input, scatters by only a few millimetres per second (one standard deviation) on each axis. The pressure-derived depth scatters by well under a millimetre, and the IMU heading by about a degree, with roll and pitch a few times finer. Each of these noise levels becomes one diagonal entry of the measurement-noise covariance R: the larger a channel's variance, the less the filter trusts that measurement, and the smaller the variance, the more it trusts it. The full set of values, and how they were obtained, is given in Section E.7. What these noise levels do not capture is that each sensor also drifts slowly. The drift is negligible from one measurement to the next, but it accumulates over a mission, and it is this slow drift, not the measurement noise, that limits how far the vehicle can dead-reckon between fixes (Sections 5.3.2 and 5.5).

Conclusion

The DVL supplies white, zero-mean velocity noise at the millimetre-per-second level, matching the Gaussian white-noise model the EKF assumes. The IMU contributes orientation and angular rate, and the pressure-derived depth has sub-millimetre noise. Together these entries fix the measurement-noise covariance R. The process noise Q, which represents vehicle dynamics rather than sensor noise, is tuned separately from manoeuvring data and documented in Section E.8.

5.3.2 Dead-Reckoning Drift and Global Correction

Introduction

The local and global filters answer two questions about the same trajectory. The local estimate dead-reckons between absolute fixes, so the first question is how fast its error grows, which sets how often a fix is needed (SR-LOC-04). The global estimate fuses those fixes, so the second is how accurately it then holds absolute position against the ±2 m requirement (MR-04, SR-LOC-02). Evaluating both on one mission makes the uncorrected drift and the corrected accuracy directly comparable.

Materials and Methods

On the `grid_02` run (Lake Zurich, 2026-05-07), the AUV was piloted manually along a nine-waypoint survey grid, a scaled-down emulation of an ice-survey mission, traced as a 134 m closed loop at the surface with RTK-fixed GNSS available throughout. A mild surface current was present during the run, an unmodelled disturbance the constant-velocity motion model does not represent. That GNSS fix is the centimetre-level ground-truth reference against which both estimates are compared. Two estimates are taken from the same recording. The local dead-reckoning solution, anchored to the first fix and then propagated on IMU, DVL, and pressure alone, isolates the uncorrected drift. Its error is first-point aligned, and the geometry-independent rate is recovered by excluding the loop's closing leg (Sections E.4 and E.5). The global EKF, fusing the GNSS fix throughout, gives the corrected georeferenced solution and is compared directly against the fix. The global filter was iterated by offline replay of the recorded sensor stream.

Results and Discussion

The dead-reckoning error grows roughly linearly with distance travelled (Figure 5.2), at an open-trajectory rate of 0.42 m per 100 m (Section E.5). Over the 134 m loop the uncorrected estimate stays within 1.82 m of truth (mean 0.59 m). Extrapolating this rate, the ±2 m budget is exhausted after about 400 m of travel (Section E.3). Beyond that an absolute fix is required, which is the basis of SR-LOC-04.

With the GNSS fix fused throughout, the global estimate stays locked to the reference. Its error against the RTK truth over the same loop is mean 0.11 m (median 0.06 m, maximum 1.64 m), against the local solution's 0.59 m mean drift, and the two tracks are compared in Figure 5.3. Since the fix is itself fused, this is a check that the filter tracks it without lag or divergence rather than an independent accuracy measure. Its value is the contrast with the local solution, where dead-reckoning alone drifts away. That bound does not grow with distance, since each fix resets the accumulated drift, so it holds for an arbitrarily long mission and stays within the ±2 m of MR-04 and SR-LOC-02.

The dead-reckoning drift comes from systematic sensor effects, not from random measurement noise. Two are constant biases: the DVL scale factor (the dominant term) and a residual heading bias. Being constant, they cancel over a closed loop and are recovered only by the open-loop fit (Section E.5). A third, the gyro's slow yaw drift, is time-varying and does not cancel. None of the three is affected by process-noise tuning (Section E.8). Reducing the drift means attacking these sources rather than retuning the filter, which the remedies of Section 5.5 address. Operationally the drift does not need to be eliminated, only kept bounded, and the periodic absolute fix already does this.

Conclusion

On the 134 m Lake Zurich grid the local estimate dead-reckons at 0.42 m per 100 m, enough for about 400 m before the ±2 m budget is exhausted. This drift is set by systematic sensor effects (DVL scale factor, residual heading bias, and gyro yaw drift), and could be cut by calibration and online bias estimation (Section 5.5) rather than by filter retuning (Section E.8). Fusing the intermittent GNSS fix keeps the global estimate locked to the reference and bounds the error regardless of distance, meeting MR-04. Under ice the same mechanism relies on the absolute fixes of Section 5.4: an acoustic SBL fix when submerged, or a GNSS fix taken by docking against the underside of the ice.

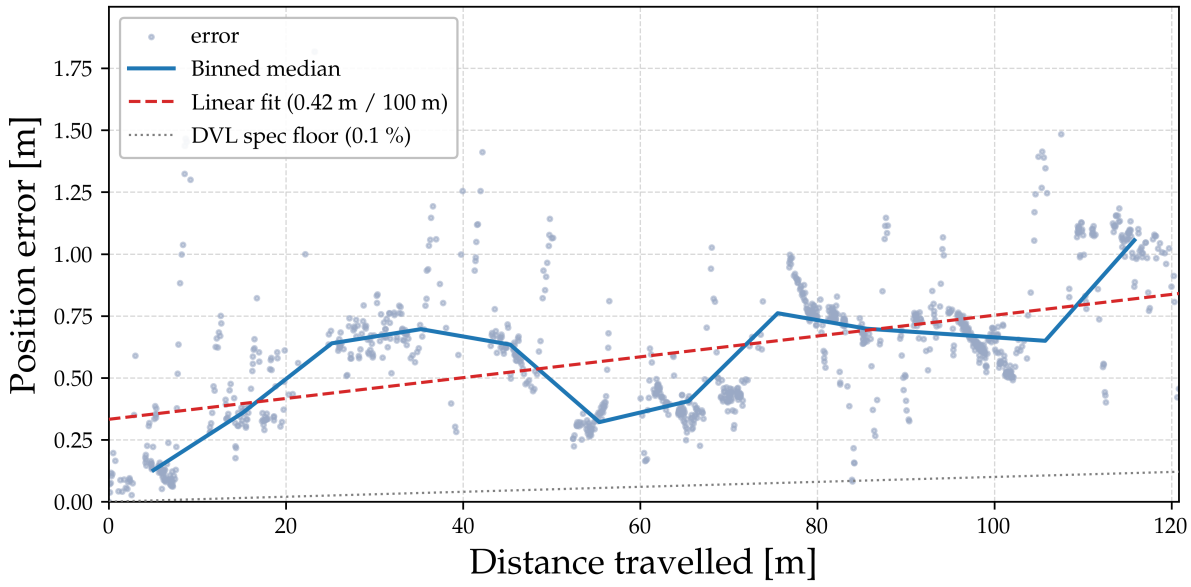


Figure 5.2 Dead-reckoning position error against distance travelled on the `grid_02` loop (local estimate vs gated RTK GNSS, first-point aligned). The linear fit gives an open-trajectory drift rate of 0.42 m per 100 m; the dotted line is the DVL 0.1 % specification floor.

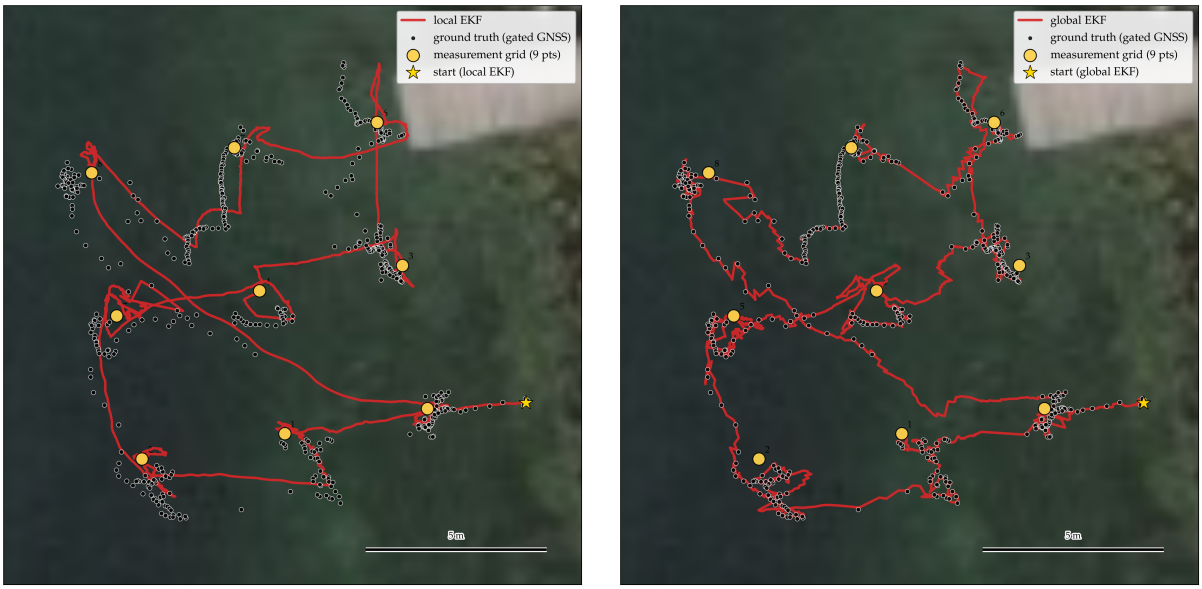


Figure 5.3 Estimated track (red) against gated RTK GNSS ground truth (white dots) on the `grid_02` nine-waypoint survey loop (yellow), Lake Zurich, 2026-05-07. *Left:* the local dead-reckoning solution, anchored to the first fix only, follows the truth but drifts (mean 0.59 m). *Right:* the global EKF, fusing the GNSS fix throughout, stays locked to the reference (mean 0.11 m); since the fix is fused, this is a tracking check rather than an independent accuracy measure.

5.4 Sensor Behaviour Under Ice

The integrated estimate of Section 5.3.2 cannot be ground-truthed under ice, where the surface GNSS reference is unavailable. This section therefore characterises the individual sensor inputs in the deployment environment instead. A frozen Alpine lake bounds the estimate at two surfaces: the *ice above*, which governs GNSS reception and the acoustic fix taken near the entry hole, and the *lake bed below*, which governs DVL bottom-lock.

5.4.1 GNSS Through Ice and Slush

Receiver-reported horizontal accuracy (h_{acc}) and depth were extracted from the St. Moritz clear-ice campaign (ice about 50 cm thick) and the Zermatt slush campaign (ice about 70 cm thick). These were cross-checked against a controlled test that varied only the sub-ice layer: natural ice with slush against the same ice with the slush removed (Section E.9). At the surface or in an open hole the receiver holds an RTK fix to about 1 cm. Liquid water, by contrast, extinguishes the GNSS signal within a few centimetres (Section 3.3), so the question under ice is whether the ice itself passes enough signal. Held flat against the underside of the clear ice, the antenna still resolved a usable sub-metre fix, best 0.58 m, with the first such fix arriving about 72 s after reaching the ice (Figure 5.4). Under slush the same manoeuvre usually missed the ±2 m requirement: only about a fifth of fixes were usable, and the typical one was several metres.
The controlled test isolates the cause: removing the slush beneath otherwise-identical ice restored centimetre accuracy, so slush, not ice thickness, is the limiting factor, attenuating the signal by 25 dB to 30 dB (Section E.9). GNSS therefore supplies the absolute reset that bounds dead-reckoning drift (Section 5.3.2, SR-LOC-04) over open water, an open hole, or clear ice. Under slush it cannot, and the vehicle falls back on the acoustic fix.

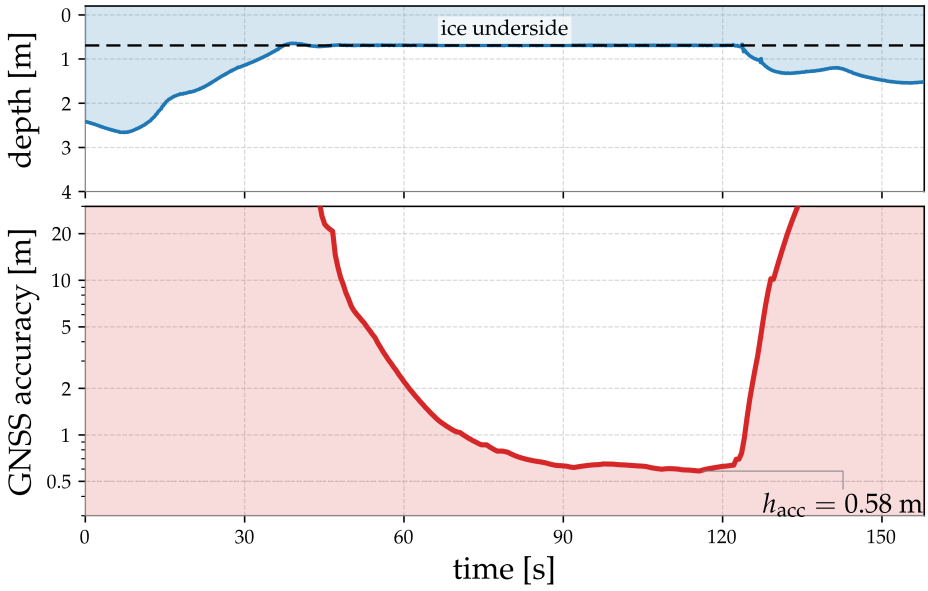


Figure 5.4 Under-ice GNSS lock, St. Moritz 2026-04-01. While the AUV held flat against the underside of about 50 cm of clear ice (shaded), the receiver-reported horizontal accuracy converged to a best of 0.58 m, the first sub-metre fix about 72 s into ice contact.

5.4.2 Acoustic SBL

When submerged the vehicle takes its absolute fix from the short-baseline acoustic system (SBL; Section 3.3). With no GNSS truth under ice, its quality is judged from its own reported covariance and its short-window position jitter (Section E.10). Matched at depth against submerged open-water runs on Lake Zurich, ice does not degrade it: the reported covariance is slightly lower under ice at every range, and the jitter is comparable, about 0.06 m in open water against 0.09 m under ice.
The one ice-specific effect is local. Near an ice hole the acoustic geometry degrades and the SBL position jumps by a few metres. On a 10 m under-ice survey grid the SBL stayed within about 1 m of the dead-reckoned track over the open grid, but jumped to roughly 6 m at the diagonal corner, where a

second hole had been cut (Figure 5.5). The reported covariance did not flag it. The jump is confined to the immediate vicinity of a hole. A fix taken there would carry the error into the global estimate, so SBL near a hole must be treated with caution. Away from holes the SBL gives a metre-level absolute fix underwater, coarser than GNSS but enough to bound drift between fixes.

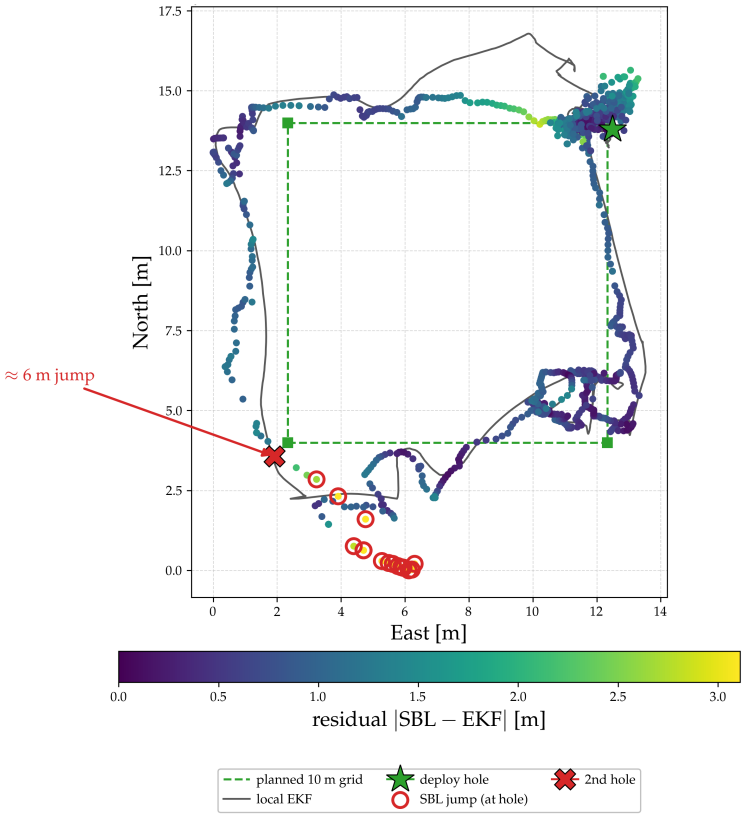


Figure 5.5 SBL position over a 10 m under-ice grid (Zermatt `rectangle_01`), coloured by its residual against the smoothed local EKF. The SBL tracks the EKF across the grid but jumps about 6 m near the ice holes (deploy hole, star; a second, non-surfaced hole, cross), where the acoustic geometry degrades; the filter does not follow the jumps.

5.4.3 DVL Bottom-Lock

The DVL measures velocity over the lake bed and, pointing downward, is shielded from the ice above (Section 3.3). No stationary under-ice recording exists to measure its noise directly, so it was compared against open water through quantities that do not depend on the vehicle's motion: the bottom-lock fraction, the number of valid beams, and the strength of the acoustic return (Section E.11). Bottom-lock stayed healthy in every under-ice campaign, holding above 98 % of the time with all four beams valid on the deepest, cleanest run (Figure 5.6). The return is a few decibels weaker under ice, but that gap traces to the lake bed rather than the ice, and even the weakest returns leave a wide lock margin (Section E.11). Only two conditions cost beams, and neither is the ice. Tilting the vehicle swings its angled beams off the bed, and thrusting to hold station against the ice aerates the water and scatters the downward beams. Both are transient, and a longer loss triggers a fallback that substitutes a propulsion-based velocity estimate until lock returns. The DVL is therefore the dependable velocity input under the ice, which is where it matters most: with no GNSS to reset position, the dead-reckoning rests on it directly.

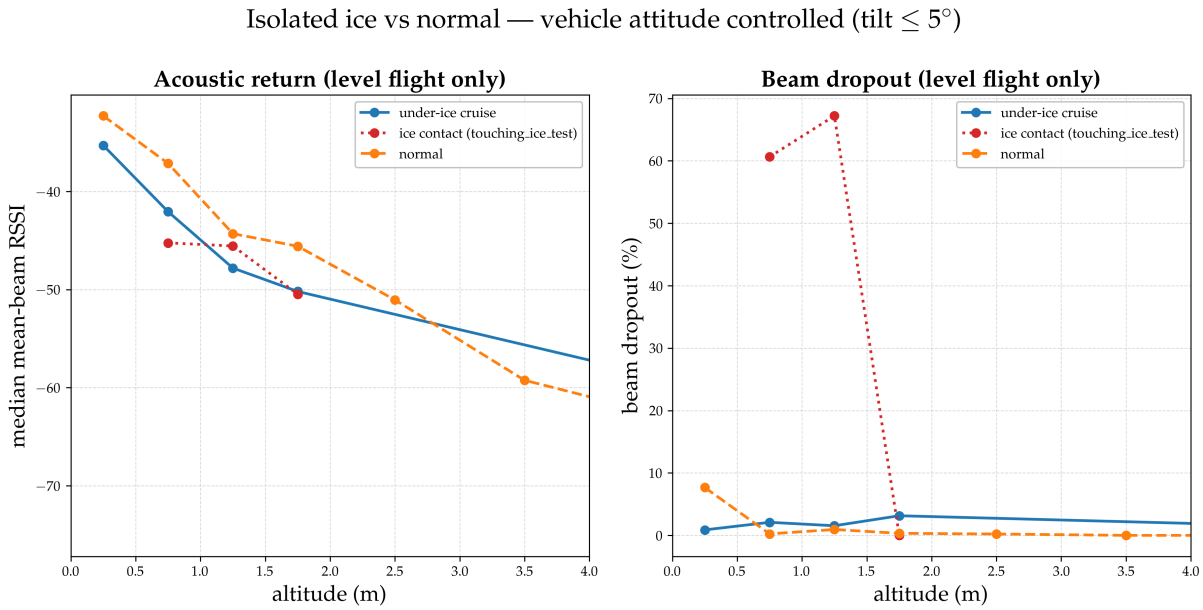


Figure 5.6 DVL acoustic return (left) and beam dropout (right) against altitude, with vehicle tilt held below 5° so range and attitude are matched. Under-ice cruise (blue) matches normal open-water operation (orange), with beam dropout of a few percent; only active station-holding at the ice (red) drives dropout into the 60 % to 67 % range, an aeration artefact of thruster wash rather than an ice effect.

5.5 Limitations and Future Work

The system has several limitations. The most consequential is its dependence on the DVL: between fixes the DVL is the only source of horizontal velocity, so a sustained loss of bottom-lock collapses the dead-reckoning, and the propulsion-based fallback (Section 5.4.3) covers only short interruptions (Section E.3). Heading is a second weak point, with no absolute reference of its own. The gyro yaw is arbitrary at power-on and drifts slowly with manoeuvring, so it relies on the one-off GNSS straight-line calibration of Section 5.2, and the small DVL transport delay (Section E.2.4) is left uncorrected. The filter is structurally limited as well: `robot_localization` is a general-purpose package built for ground and aerial robots, where neither water currents nor the loss of a primary velocity sensor are central concerns. Its fixed 15-state model estimates no sensor biases, so the gyro bias, the DVL's bias instability, and a sustained water current are not tracked online, and instead surface as the residual dead-reckoning drift and the time-correlated innovations of Section E.8. Finally, some measurements remain open: no stationary under-ice recording exists to document the ice's effect on the DVL noise, which is for now carried over from open water (Section 5.4), and the SBL's depth and range dependence has not been characterised.

Several improvements follow directly. A two-antenna GNSS that measures heading directly would remove the manual straight-line calibration and, under ice, let the slow gyro drift be reset by surfacing at a hole rather than re-driving a calibration line. A short straight-line run against RTK GNSS would measure the constant biases behind the drift, the DVL scale factor and the residual yaw bias, by comparing the integrated DVL track to the GNSS displacement. Both could then be compensated at the driver (Section E.5), and the DVL transport delay could be trimmed at its source in the same way. For the structural limitation the standard remedy is a custom estimator augmented with bias and water-current states. Finally, the missing under-ice DVL recording and dedicated SBL depth- and range-tests would close the remaining gaps. An offline smoothing pass could also refine the georeferenced map after the dive: unlike the causal online EKF, it lets each fix correct the trajectory that came before it.

CHAPTER 6

Path Planning and Control

As stated in Section 1.2 as **MO-04**, the vehicle shall autonomously navigate to predefined measurement locations and perform ice-thickness measurements. This requires the localisation stack to provide a continuous state estimate — position, velocity, orientation, and angular velocity — at a quality and computing rate sufficient for closed-loop path tracking (see Section 5.1). Given this state estimate, the autonomy stack developed in this chapter consists of a straight-line planner, a modified Pure Pursuit guidance controller, and a Behaviour Tree that coordinates waypoint sequencing, re-planning, and safety monitoring.
The autonomy stack has been successfully validated in the Lake Zurich testing area at the water surface. The AUV independently completed a full $20\,\text{m} \times 20\,\text{m}$ grid and the majority of a $40\,\text{m} \times 40\,\text{m}$ grid. It navigated to all predefined waypoints via 90° turns and returned to the starting position (see Section 6.3.4). Throughout all operations the AUV remained tethered, and the operator could disarm the vehicle at any moment via a single button press. Importantly, the operating environment was assumed to be obstacle-free, which justified the use of straight-line reference paths without collision checking.
The implementation builds upon the `nav2` framework [3], with specific plug-ins adapted from the `orca4` repository [4]. `orca4` also provides the basis for the simulation environment.
Section 6.1 describes the architecture, Section 6.2 the simulation environment, Section 6.3 the iterative tuning and field validation, and Section 6.4 the limitations with future work.

6.1 Architecture

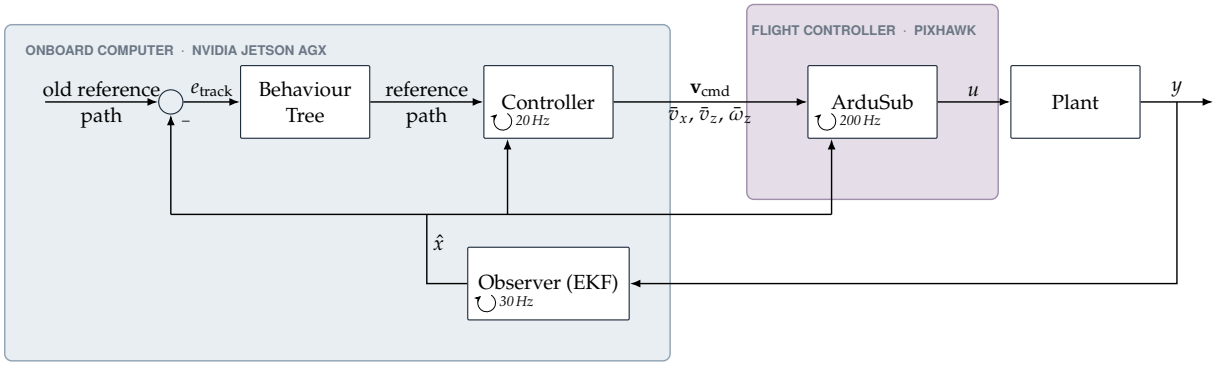


Figure 6.1 Block diagram of the cascaded autonomy stack's feedback loops, excluding the planner and mission start-up. Some boxes are annotated with their update rates. The Plant represents the real AUV. $|e_{\text{track}}|$ is the cross-track error to the reference path, $\hat{x}$ is the EKF state estimate, $\mathbf{v}_{\text{cmd}} = \begin{bmatrix} \bar{v}_x & \bar{v}_z & \bar{\omega}_z \end{bmatrix}^\top$ the commanded velocity, u the thruster PWM signals, and y the true vehicle outputs, of which only a noisy subset is fed back to the Observer.

The mission waypoints are specified in WGS84 coordinates [30] prior to deployment. Upon launching the autonomy stack, each waypoint is transformed into the map ENU frame using the same GNSS fix that defines the map frame origin (see Chapter 5, Section 3.3). Once the mission has started, the `planner` computes a straight-line reference path from the current estimated position to the first waypoint in the map frame (see Section F.2).
Figure 6.1 provides an overview of the architecture. The tracking error e_{track} is computed as the distance

between the estimated position and the closest point on the reference path. The behaviour tree evaluates this error at 2 Hz (see Section F.1): if the horizontal deviation exceeds a predefined threshold $|e_{\text{track}}| > d_{\text{safe}}$, a new reference path is generated from the current position towards the active waypoint. This re-planning prevents the cross-track error from growing unbounded.
The reference path — whether the previous or the re-planned one — is passed to the modified Pure Pursuit controller (MPPC) operating at $f_{\text{controller}} = 20$ Hz. Given the current position and the lookahead distance LAD, the controller computes a commanded velocity $\mathbf{v}_{\text{cmd}}$ in the body-fixed FLU frame [29], consisting of a surge component $\bar{v}_x$, a heave component $\bar{v}_z$, and a yaw rate $\bar{\omega}_z$. To handle the sharp 90° turns required by the rectangular survey grid, the controller includes a hysteresis mechanism [31] that switches between path-tracking and turning modes. The velocity commands are then set as twist references by the cascaded low-level PID controllers running on the Pixhawk flight controller (see Section 4.1.2) with the ArduSub firmware.
The base Pure Pursuit controller follows the orca4 implementation [4]. The decision for using Pure Pursuit as the baseline guidance controller can be found in Section F.3. The final MPPC is described in detail in Section F.3.

6.2 Simulation Environment

To validate the architecture prior to field deployment, a simulation environment [1] was established using Gazebo [5] and the ArduSub software-in-the-loop (SITL) capabilities [32]. Gazebo simulates hydrodynamic effects, such as added mass, buoyancy, and drag based on [33]. Importantly, the SITL instance runs the same ArduSub firmware as the physical Pixhawk, meaning the entire autonomy stack — from the behaviour tree down to the velocity commands — executes identically in simulation as on hardware. The only significant difference is the source of the state estimate: in simulation it is provided by Gazebo, whereas on the real vehicle it comes from the observer described in Chapter 5. Furthermore, a simulated water current was implemented to evaluate the controller performance under external disturbances. The current is modelled as a first-order Gauss–Markov stochastic process following [10]. The deterministic bias component v_{bias}, which can take various forms such as a constant or a sinusoidal signal, is combined with white noise of standard deviation σ, filtered through a first-order process with time correlation constant τ. The water current can be applied to any axis or combination of axes.
This design tries to minimises the sim-to-real gap at the software level. It does not however, capture all real-world effects: sensor noise on the pose estimate is absent (Gazebo provides ground truth), tether forces are neglected, and the hydrodynamics are simplified. Nevertheless, the simulation serves as a suitable environment for tuning the guidance controller before field deployment.

6.3 Tests

Since access to the Lake Zurich testing site was limited, an iterative sim-to-real approach was adopted to make efficient use of the available time. Physical constraints, such as maximum velocities and accelerations, were first estimated from early tests and used to validate the architecture and tune the guidance parameters in simulation. Subsequently, the low-level controllers were tuned on the hardware and the architecture was validated at the Lake Zurich testing site. The measured maximum trackable accelerations $a_x^{\max}$, $a_z^{\max}$, and $\dot{\omega}_z^{\max}$ obtained from this low-level tuning were then fed back into the simulation for a second tuning cycle (Section F.4), before the complete stack was redeployed at the lake Zurich for the final field tests (Section 6.3.4).

6.3.1 First Simulation Cycle

Introduction

The goal of this first simulation cycle was to validate the autonomy architecture before field deployment, to identify necessary modifications early, and to tune the guidance controller (MPPC).

[1] The simulation was executed on a DELL-PC with an Intel Core i7-10700 CPU (8 cores / 16 threads, 2.90 GHz) and 16 GB RAM.

Materials and Methods

The maximum accelerations used were estimated by commanding full-throttle inputs via the controller in manual mode and observing the vehicle response. Table 6.1 lists the assumed values.

Table 6.1 Assumed physical constraints for the first simulation cycle. The maximum velocities were set beforehand as mission constraints.

Parameter	Value	Unit
Surge velocity $\bar{v}_x$	0.4	$\mathrm{m\,s^{-1}}$
Surge acceleration $a_x^{\max}$	0.2	$\mathrm{m\,s^{-2}}$
Heave velocity $\bar{v}_z$	0.25	$\mathrm{m\,s^{-1}}$
Heave acceleration $a_z^{\max}$	0.2	$\mathrm{m\,s^{-2}}$
Yaw velocity $\bar{\omega}_z$	0.3	$\mathrm{rad\,s^{-1}}$
Yaw acceleration $\dot{\omega}_z^{\max}$	0.35	$\mathrm{rad\,s^{-2}}$

The lookahead distance LAD of the standard Pure Pursuit controller was tuned via step responses in straight-line tracking simulations with an initial lateral offset of $d_{\text{offset}} = 1.2\,\mathrm{m}$, using overshoot and settling time as performance metrics. During this process, it was observed that a standard Pure Pursuit controller was insufficient for the sharp 90° turns required by the survey grid. A hysteresis mechanism was therefore introduced, resulting in a modified Pure Pursuit controller (MPPC) that switches between path-tracking and turning modes (see Section F.3 for the full tuning procedure).
The architecture was then tested on a square path with a side length of 11.5 m at constant depth, driven once without current and once with a simulated current of $v_{\text{bias}} = -0.08\,\mathrm{m\,s^{-1}}$ in the horizontal plane, noise standard deviation $\sigma = 0.1\,\mathrm{m\,s^{-1}}$, and time constant $\tau = 2.5\,\mathrm{s}$.

Results and Discussion

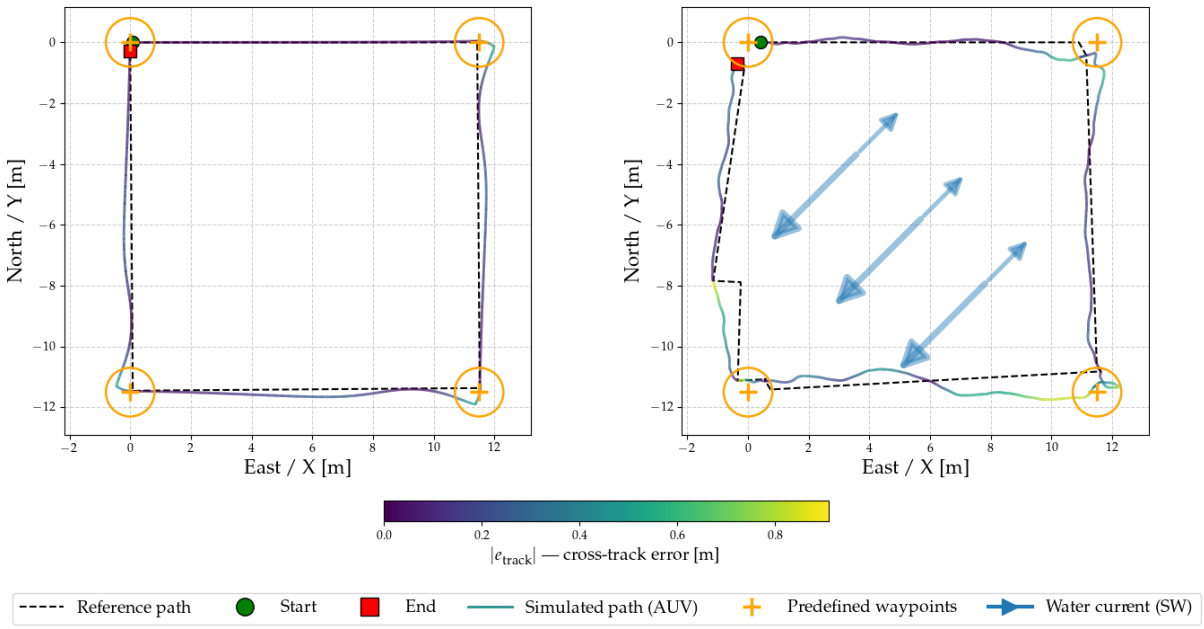


Figure 6.2 Simulated AUV path for the square mission with edge lengths of 11.5 m, coloured by the cross-track error $|e_{\text{track}}|$, defined as the distance from the AUV to the closest point on the reference path. The predefined waypoints are marked, and the start and end points are indicated. A lookahead distance of LAD = 2.5 m is used.
Subfigure a) (left) shows the result without water current, while Subfigure b) (right) shows the result with a simulated diagonal water current.

Figure 6.2 shows the AUV navigating sequentially to each of the four waypoints. For each leg, the planner computed a straight-line reference path from the current position to the next waypoint. When the AUV entered the goal tolerance $d_{\text{goal}} = 0.8\,\mathrm{m}$, the goal checker returned success and the stack proceeded to the next, repeating this process until the mission was completed.

Figure 6.2.a, visualises the result without simulated water current. The AUV tracked the first leg with negligible cross-track error. At the first corner, the surge velocity had not yet decayed to zero when the new reference path was computed, causing the AUV to enter the hysteresis turning mode while still moving forward, leading to an overshoot of approximately $e_{\text{overshoot}} \approx 0.4\,\text{m}$. After the turn the vehicle converged back onto the reference path with minor oscillations and completed the remaining legs successfully, though overshooting at each corner.
Figure 6.2.b, shows the same trajectory with a simulated water current. Since the noise component can momentarily exceed the bias $v_{\text{bias}} = -0.08\,\text{m s}^{-1}$, resulting in occasional positive current values, a smaller arrow in the positive XY-direction is included in the plot. On the first leg, small oscillations already appeared. At the end of the first leg, as the vehicle decelerated, the current-to-speed ratio increased and the AUV drifted by approximately 0.5 m before overshooting into the turn. The vehicle still completed the turn and converged back onto the reference path. On the fourth leg the cross-track error exceeded the safe-distance threshold $d_{\text{safe}} = 0.9\,\text{m}$, triggering the planner to generate a new reference path from the current position. The AUV tracked this new path and completed the mission.
The AUV completed the square in $T_{\text{completion}} = 3\,\text{min}\,29\,\text{s}$ without current and in $T_{\text{completion}} = 3\,\text{min}\,21\,\text{s}$ with current. The root-mean-square cross-track error increased from RMSE = 0.23 m without current to RMSE = 0.38 m with current.
An additional test showed that a constant south-west current of $v_{\text{current}} = 0.25\,\text{m s}^{-1}$ without added noise represents the upper limit the system can withstand while still completing this square mission.

Conclusion

The simulation results confirmed that the autonomy architecture is capable of completing a rectangular survey mission both with and without simulated water current. The hysteresis mechanism handled the sharp 90° turns, and the re-planning logic successfully recovered the vehicle when the cross-track error exceeded the safe-distance threshold $d_{\text{safe}} = 0.9\,\text{m}$. However, the corner overshoots indicated that the braking behaviour could be improved, for instance by initiating deceleration earlier. Based on these results, the autonomy architecture was considered ready for an first deployment at the Lake Zurich testing site.

6.3.2 Low-Level Controller Tuning in Lake Zurich

Introduction

As described in Section 6.1, the modified Pure Pursuit controller sends a (partial) target Twist $\mathbf{v}_{\text{cmd}}$ to the Pixhawk flight controller, which maps via the cascaded PID loops PWM signals to each thruster (see Section 4.1.2). These low-level controller required separate tuning. Additionally, the trackable maximum acceleration limits for surge and yaw rate needed to be determined experimentally to replace the assumptions used in Section 6.3.1 and listed in Table 6.1. These acceleration limits effectively act as a reference filter that shapes the velocity profile before it reaches the low-level controllers. The heave axis had already been tuned for the depth-hold mode (see Section D.6).

Materials and Methods

The tuning was performed at the Lake Zurich testing area with the AUV tethered via a LAN cable. Surge and yaw were tuned independently using ramp inputs with variable maximum speed and ramp-up time. Ramp commands were chosen over step inputs because the MPPC produces continuous velocity profiles. For surge, the objective was to track the commanded velocity, especially the desired cruising speed $v_{x,\,\text{cruise}} = 0.4\,\text{m s}^{-1}$, without steady-state decay over time. The settling time was of secondary importance. For yaw, the focus was on accurate ramp-up tracking up to a maximum yaw rate of $\bar{\omega}_z = 0.3\,\text{rad s}^{-1}$, since commanded yaw rates are typically short-lived during path corrections.
After tuning each axis independently, both were commanded simultaneously — a surge ramp to $v_{x,\,\text{cruise}} = 0.4\,\text{m s}^{-1}$ combined with a yaw ramp up to $\bar{\omega}_z = 0.3\,\text{rad s}^{-1}$ followed by a reversal to $\bar{\omega}_z = -0.3\,\text{rad s}^{-1}$ — approximating the oscillating commands produced by the MPPC during real tracking. The detailed tuning procedure, response plots of each axis, and the controller gains are provided in Section F.5.

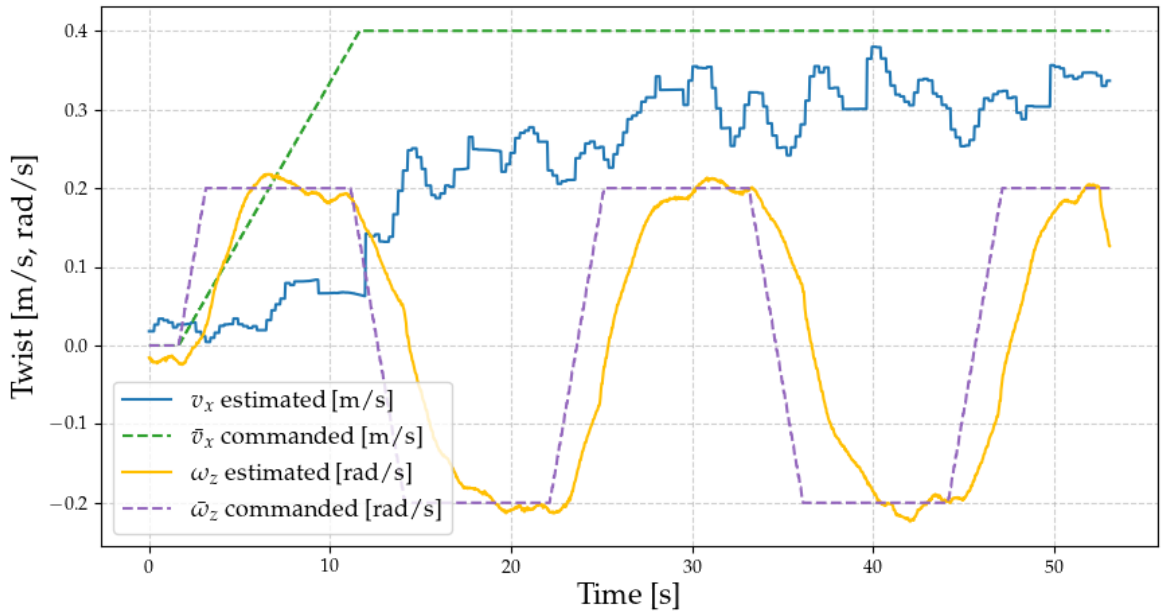


Figure 6.3 Combined surge and yaw-rate response (Twist) under simultaneous commands. The commanded surge velocity $\bar{v}_x$ is plotted as a dashed green line, while the yaw rate $\bar{\omega}_z$ as a dashed purple line. The estimated surge velocity v_x is shown in a blue solid line, while the yaw rate ω_z in a yellow solid one.

Results and Discussion

The surge controller was tuned to a PID configuration with a relatively high integral gain and low proportional gain (see Table F.1), since the integral term was needed to compensate for non-linearities and drag while maintaining zero steady-state error. The derivative gain was kept low, since the velocity estimate from the observer was still noisy and differentiation would have further degraded the signal-to-noise ratio. See Section F.5 for the detailed surge tuning.
The yaw-rate controller, by contrast, required primarily proportional action with a small derivative component (see Table F.2), as fast tracking was prioritised and minor overshoots were assumed to be acceptable. See Section F.5 for a separate discussion of the yaw-rate tuning results.
From these responses, the maximum trackable accelerations were computed (see Section F.5) and are listed in Table 6.2 in the column "**Actual Value**".
Figure 6.3 shows the final result of the "zig-zag" validation with the "**Final Value**" accelerations. The surge loop ramped up slowly to the cruise speed $v_{x,\text{cruise}} = 0.4\,\text{m}\,\text{s}^{-1}$ with the same over-damped response observed during individual tuning, and the yaw-rate loop continued to follow each reversal of the "zig-zag" profile with the same lag observed during individual tuning.
Due to the non-linear hydrodynamic effects, a trade-off between ramp-up tracking and overshoot avoidance was necessary. The measured accelerations were increased (Table 6.2, column "**Final Value**") to provide the guidance controller with sufficient headroom for transient manoeuvres while avoiding artificially slow trajectories.

Table 6.2 Maximum accelerations calculated from the ramp responses. The "**Actual Value**" column lists the measured maximum trackable accelerations, while the "**Final Value**" column gives those actually used for the second simulation cycle. Heave values were neglected.

Parameter	Initial Value	Actual Value	Final Value	Unit
Surge acceleration $a_x^{\max}$	0.2	0.01	0.04	$\text{m}\,\text{s}^{-2}$
Yaw acceleration $\dot{\omega}_z^{\max}$	0.35	0.02	0.2	$\text{rad}\,\text{s}^{-2}$

Conclusion

The combined test confirmed that both loops retained the tracking behaviour seen in isolation, with no significant coupling between the surge and yaw-rate channels. Yet, the yaw-rate tracking of the ramp-up exhibited a persistent lag and a unbounded steady-state error, which could have been mitigated (theoretically) by adding integral action to the yaw-rate PD. This was deliberately omitted out of caution, as integral action in the yaw-rate loop risked introducing cumulative heading-rate drift. The tuned gains were nonetheless considered suitable for deployment.

Full Autonomy Stack Deployed

After the low-level controller tuning, the full autonomy stack was deployed for the first time with the configuration of Table 6.1. The results (see Section F.6) confirmed that the stack successfully navigated multiple waypoints at the water surface. Figure F.11 shows the $X - Y$ view of the estimated AUV path and reference path computed online. Nevertheless, vertical motion proved unreliable, with the vehicle exhibiting strong pitch oscillations during depth changes. Consequently, a simplified autonomy architecture was introduced in which the heave command was held at $\bar{v}_z = 0.0\,\mathrm{m\,s^{-1}}$, restricting the AUV to horizontal motion in the $X - Y$-plane at a constant depth. This architecture was used for the tests described in Section F.4 and Section 6.3.4.

6.3.3 Second Simulation Cycle

With the trackable acceleration limits now measured on hardware (Table 6.2), the guidance controller was re-tuned in a second simulation cycle, following the same step-response tuning and square-mission validation procedure as in Section 6.3.1. The tighter acceleration limits reduced the optimal lookahead distance to LAD = 1.7 m (down from 2.5 m) and the safe-distance threshold to $d_{\mathrm{safe}} = 0.7\,\mathrm{m}$, while earlier and stronger braking lowered the corner overshoot. These parameters were carried forward to the final field tests. The full tuning procedure, trajectories, and discussion are provided in Section F.4.

6.3.4 Autonomous Grid in Lake Zurich

Introduction

This test served as the final validation of the autonomy stack developed over the course of this project. The objective was to verify that the AUV could autonomously navigate a grid of waypoints with 90° turns and return to the starting position using the parameter set from the second simulation cycle (Section F.4) and the tuned low-level controllers (Section 6.3.2). The mission was conducted at the water surface, enabling GNSS and dead-reckoning for state estimation without relying on the SBL system.

Materials and Methods

Before the mission, the waypoints were defined and the AUV was connected via the fibre-optic tether. The testing site was Lake Zurich under calm conditions with a northward current of $v_{\mathrm{current}} \approx 0.1\,\mathrm{m\,s^{-1}}$. Two grids were tested: first a $20\,\mathrm{m} \times 20\,\mathrm{m}$ grid (Section 6.3.4), followed by a $40\,\mathrm{m} \times 40\,\mathrm{m}$ grid (Section F.7). A waiting time of $T_{\mathrm{wait}} = 1.5\,\mathrm{s}$ was enforced at each waypoint before proceeding to the next, to approximate a real ice mission involving descent, ascent, and position holding at the ice surface.
The heave command was held constant at $\bar{v}_z = 0.0\,\mathrm{m\,s^{-1}}$ to maintain the starting depth. For evaluation, the same metrics as in the simulations were used: the top-down $X - Y$-plane trajectory view, a comparison of commanded versus estimated velocities, and the cross-track RMSE.

Results and Discussion

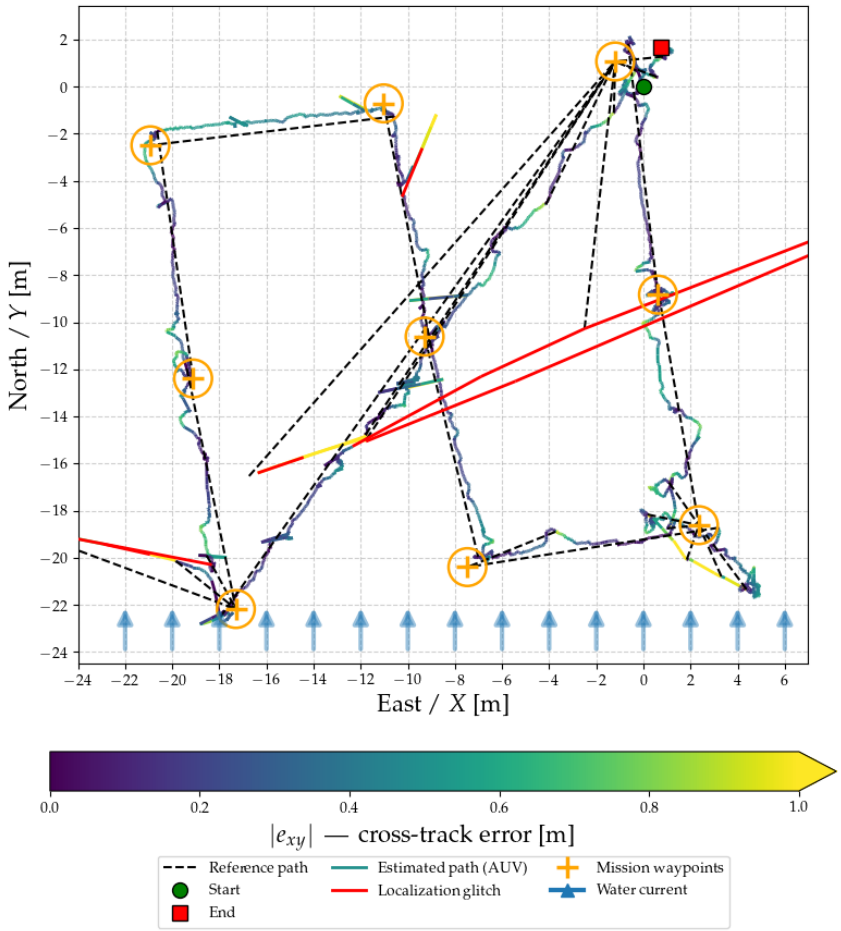


Figure 6.4 Top-down $X - Y$ view of the 20 m × 20 m grid mission at Lake Zurich. The estimated AUV path (from the EKF), coloured by the cross-track error $|e_{\text{track}}|$, is shown against the online-computed reference paths (dashed) and the predefined mission waypoints. The start and end points are marked, and the red segments mark localisation glitches. The blue arrows indicate the direction of the northward water current only and are not scaled to its magnitude.

Figure 6.4 shows the estimated path from the EKF against the online-computed reference paths and the predefined waypoints, colour-coded by cross-track error for the 20 m × 20 m grid. The AUV started near ($X_{\text{start/end}} = -1$ m; $Y_{\text{start/end}} = 1$ m) and proceeded southward. At the second waypoint ($X_{\text{w2}} = 2$ m; $Y_{\text{w2}} = -19$ m), the vehicle did not enter the goal tolerance $d_{\text{goal}} = 0.8$ m but passed just outside it. Since the goal was not accepted, the AUV continued along the active reference path, drifting further from the waypoint. As the cross-track error to the reference path grew, the safe-distance threshold $d_{\text{safe}} = 0.7$ m was exceeded and a new reference path was computed from the current position back towards the missed waypoint. This re-planning activated the hysteresis and the AUV re-orientated itself towards the waypoint, giving itself an opportunity to reach the goal tolerance for a second time. The vehicle then followed the new path and successfully reached the goal. In some cases the vehicle missed a waypoint on the first approach and required a re-planned path to reach the goal tolerance, yet the vehicle recovered autonomously.

As the mission progressed, localisation glitches became more frequent.

After completing the diagonal from the penultimate to the final waypoint, the AUV finished the mission. The overall RMSE was 0.45 m (excluding glitch segments) with a completion time of $T_{\text{completion}} = 28$ min. The cross-track was comparable to the simulated results from the second simulation cycle (Section F.4), indicating a smaller sim-to-real gap than initially expected.

Figure 6.5 shows the commanded and measured surge velocity and yaw rate over a 60 s window at the fifth waypoint at ($X_{\text{w5}} = -11$ m; $Y_{\text{w5}} = -1$ m). In the first 10 s, the surge command $\bar{v}_x$ (green dashed) decreased to zero for braking before the waypoint, but the estimated velocity v_x (blue) did not fully reach zero due to the vehicle's inertia and the northward current. The grey region highlights the hysteresis

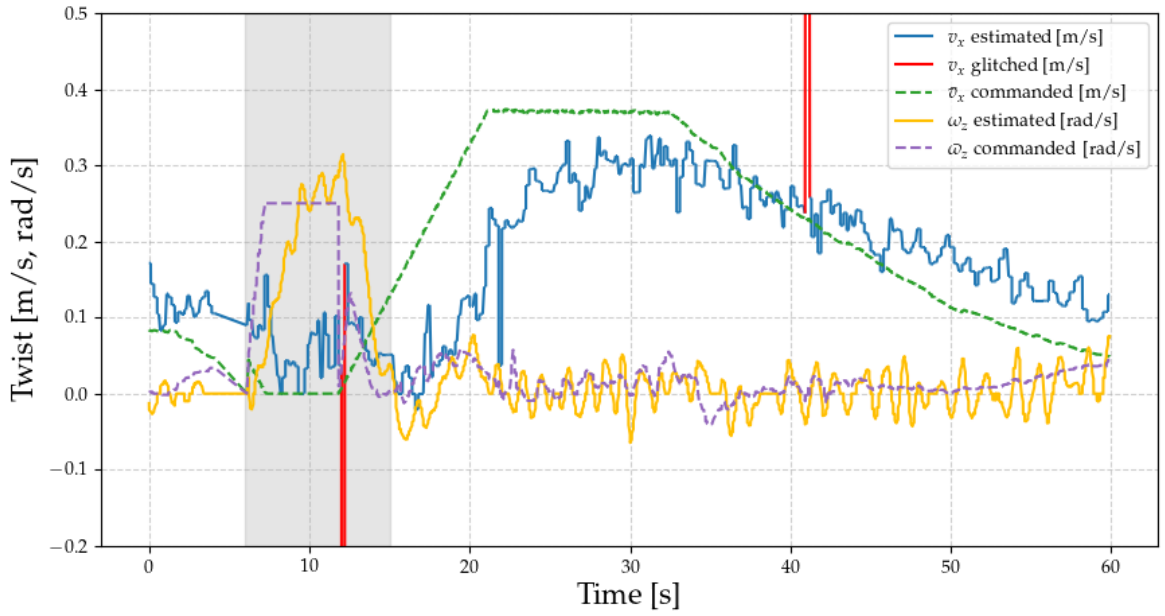


Figure 6.5 Commanded and estimated (partial) twist over a 60 s window of the 20 m × 20 m grid mission, recorded over the fifth waypoint. The commanded surge velocity $\bar{v}_x$ is indicated as a dashed green line, while the yaw rate $\bar{\omega}_z$ in a dashed purple line. The estimated surge velocity v_x is shown in a blue and the yaw rate ω_z in a yellow solid line. Invalid twist segments are marked with red solid lines.

activation: the yaw rate command $\bar{\omega}_z$ (purple dashed) increased for the sharp turn, while the estimated yaw rate ω_z (yellow) lagged behind the reference. After the turn was completed, the surge command ramped up again towards the cruising speed for the next leg. During this phase the yaw command returned to small values, yet the estimated yaw rate oscillated significantly around the reference. The hysteresis behaviour is presented in detail in Section F.7, which also shows that the commanded yaw rates were insufficient to fully correct the heading error after sharp turns. Reducing LAD further could address this. The resulting oscillations seen in simulation would be damped in the real environment by the higher drag and current effects, while the higher commanded yaw rates would compensate for the reduced tracking performance observed in practice.
The same configuration was additionally tested on the 40 m × 40 m grid. The system performed similarly until the localisation degraded abruptly and the mission was manually aborted (see Section F.7).

Conclusion

The AUV successfully completed the grid mission, navigating to all predefined waypoints and returning to the starting position. The results demonstrated the resilience of the modified Pure Pursuit controller with the hysteresis mechanism under real lake conditions with current. The invalid velocity readings corrupted the observer's state prediction. To mitigate this, outlier detection, such as the prediction-error thresholds proposed in [34] and Section 5.5, can be integrated into the state estimation.
As noted in Section F.4, adding integral action to the guidance controller could reduce the steady-state cross-track error, provided that the localisation estimate does not drift excessively.

6.4 Limitations and Future Work

The modified Pure Pursuit controller, being a purely geometric guidance law, lacks integral action on the cross-track error, resulting in a persistent steady-state offset under sustained current. Augmenting the guidance law with an integral cross-track term, or replacing it with a model-based controller capable of explicitly accounting for current disturbances, could further reduce this offset. Furthermore, tuning the guidance controller directly in the lake environment, rather than exclusively in simulation, could improve the tracking performance under real hydrodynamic conditions and provide additional insight into the remaining sim-to-real gap.

The low-level control loops were tuned only once and would benefit from further iteration, in particular adding integral action to the yaw-rate loop to reduce the tracking lag and unbounded steady-state error observed during ramp commands at sharp turns, and reducing the settling time of the surge controller. The current constant acceleration limiter (see Section 6.3.2) could be replaced by a more sophisticated reference governor. Such a scheme would shape the commanded velocity profile online to satisfy state and input constraints while preserving tracking performance. The unexpected vertical behaviour observed in Section 6.3.2 originates at the guidance layer rather than at the low-level controller. This currently prevents reliable multi-depth missions, so the altitude guidance controller requires further investigation. An autonomous mission below the water surface, relying on the SBL system instead of GNSS for localisation, has not yet been conducted and remains a necessary next step towards under-ice deployment. Several Behaviour Tree branches, most notably the ice-surface approach sequence, have only been validated in simulation and require field testing before deployment. On the simulation side, adding realistic state estimation noise and refining the hydrodynamic effects would further narrow the sim-to-real gap.

CHAPTER 7

Ice Measurement

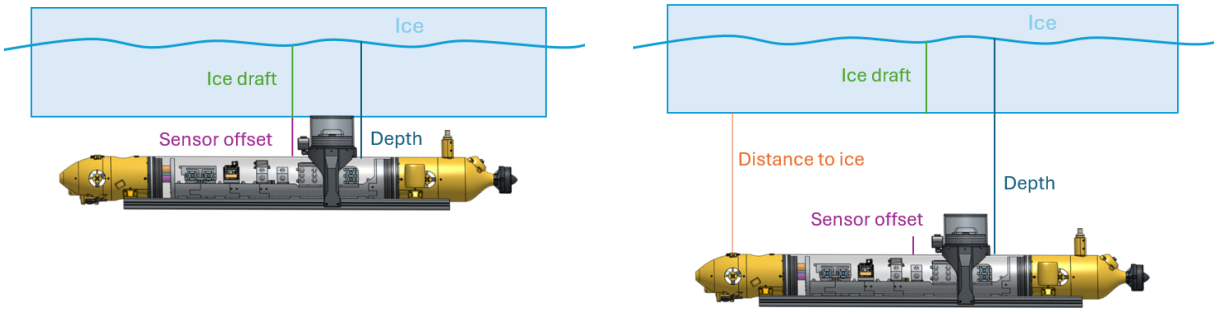
The primary scientific deliverable of the POLARIS mission is the measurement of ice thickness of frozen lakes. Ice thickness is the key quantity required for assessing ice safety and characterising lake ice conditions. Consequently, the ability to determine ice thickness accurately forms the central mission objective.
To achieve this goal, Mission Objective MO-01 requires the system to measure ice thickness with a measurement uncertainty not exceeding 1 cm (one standard deviation). This chapter presents the measurement principle, the validation methodology, and the test results used to verify compliance with MO-01. Although Measurement Requirement MR-01 specifies an accuracy of 1 cm over an ice-thickness range of 0 cm to 100 cm, with a relative accuracy of 1% beyond this range, dedicated verification during this project was limited to ice thicknesses between approximately 10 cm and 20 cm. This range was selected because it closely represents the ice conditions of interest identified in Section 1.1, while still allowing safe acquisition of reference measurements. The following sections present the experimental validation of the proposed measurement method within this range using the Keller Series 26X pressure sensor, followed by full-system testing to evaluate operational performance, identify practical failure modes, and discuss the effectiveness of the implemented mitigation strategies and opportunities for future improvement.

7.1 Measurement Principle and Method

The vehicle employs two complementary methods for ice-thickness estimation. The primary method is based on pressure measurements acquired while the vehicle is in contact with the ice underside. In this configuration, the pressure sensor directly measures the ice draft, which is converted to total ice thickness using Archimedes' principle:

$$h_{\text{ice}} = \frac{\rho_{\text{water}}}{\rho_{\text{water}} - \rho_{\text{ice}}} h_{\text{draft}} + \delta_{\text{sensor}} \tag{7.1}$$

where h_{draft} denotes the submerged ice draft and δ_{sensor} accounts for sensor offsets and calibration corrections. Since the measurement relies only on the pressure sensor as a sensing device, the resulting uncertainty is dominated by the pressure sensor accuracy. This configuration therefore provides the highest measurement accuracy and serves as the primary source of scientific data.
For continuous mapping, an upward-facing ultrasonic transducer measures the distance to the ice underside. Combined with the vehicle depth obtained from the pressure sensor, this enables ice-thickness estimation while travelling below the ice. As this method combines measurements from two sensors, uncertainties from both the pressure sensor and ultrasonic transducer contribute to the final estimate. Consequently, its accuracy is lower than that of the contact-based method, but it enables significantly faster and denser spatial coverage. One situation in which this method would be considered favourable over the direct contact measurement is when the ice sheet is particularly thin. In the case that the thickness of the ice sheet falls below 5 cm, direct contact is to be avoided as to not influence the hydrostatic pressure measurements by the presence of the vehicle physically pushing against the ice underside.
To ensure data quality, all measurements are subjected to admissibility checks using onboard sensor data. Measurements acquired during excessive vehicle motion, poor navigation quality, or inconsistent sensor readings are rejected. These checks primarily rely on IMU, pressure sensor, ultrasonic transducer, and DVL data.



(a) Primary measurement mode. The vehicle establishes contact with the ice sheet and determines ice thickness from its draft using the pressure sensor and the Archimedes relation.

(b) Secondary measurement mode. Ice thickness is inferred from the measured distance to the ice underside using the upward-facing ultrasonic transducer combined with pressure-based depth information.

Figure 7.1 The two ice-thickness measurement modes implemented in the vehicle. The contact-based method provides the highest accuracy and serves as the primary scientific measurement, while the distance-based method enables continuous measurements without requiring contact with the ice sheet.

7.2 Tests

The ice-thickness measurement system was evaluated during three field campaigns. The campaigns were conducted chronologically and served different purposes in the validation process. The initial St. Moritz campaign with the complete underwater vehicle demonstrated the feasibility of the measurement concept under realistic operating conditions. This was followed by a dedicated validation campaign using a standalone measurement jig equipped with a higher-accuracy pressure sensor, which provided the primary dataset for quantitative accuracy assessment. Finally, a second campaign with the complete vehicle was conducted in Zermatt to verify the measurement approach in a mission-representative configuration.

7.2.1 St. Moritz System Demonstration

Introduction

The first field campaign was conducted on Lake St. Moritz using the complete vehicle. The objective was to demonstrate that ice-thickness measurements could be obtained during realistic underwater operation and to evaluate both the Archimedes-based and ultrasonic measurement approaches.

Materials and Methods

The vehicle was equipped with a Blue Robotics pressure sensor and an upward-facing Ping sonar. Ice thickness was estimated using the method described in Section 7.1. Reference measurements were obtained from ice holes cut into the lake surface. A total of six comparison measurements were collected. For the uncertainty analysis, a natural ice-thickness variability of 1 cm and a manual measurement uncertainty of 0.5 cm were assumed based on the observed conditions and measurement procedure.

Results and Discussion

The measured ice thicknesses ranged from 48.2 cm to 51.9 cm, while the reference measurements ranged from 49.5 cm to 52.5 cm. The average measured thickness was 49.33 cm, compared to a reference average of 50.29 cm, corresponding to a mean residual of −0.96 cm. The residuals ranged from −2.3 cm to 2.4 cm, which is consistent with the expected uncertainty of the measurement setup.
Although the dataset was too small for rigorous validation of MO-01, the measurements demonstrated that the Archimedes-based approach produced physically plausible results and correctly captured the local ice thickness. The campaign also highlighted the need for a larger dataset and more accurate instrumentation to perform a meaningful accuracy assessment.
In contrast, the Ping sonar measurements frequently produced implausible results that could not be reconciled with the known ice geometry. This indicated that the ultrasonic distance measurements were not sufficiently reliable in their current implementation and reinforced the decision to use the pressure-based Archimedes method as the primary scientific measurement.

Conclusion

The St. Moritz POLARIS campaign provided the first successful system-level demonstration of ice-thickness measurements with the complete vehicle. While the limited dataset prevented formal verification of MO-01, the results confirmed the viability of the Archimedes-based measurement principle and motivated a dedicated validation campaign using a higher-accuracy Keller pressure sensor.

7.2.2 St. Moritz Measurement Method Validation

Introduction

Following the initial validation with the complete vehicle, a dedicated measurement campaign was conducted on Lake St. Moritz to quantitatively evaluate the accuracy of the Archimedes-based ice-thickness measurement method. The primary objective was to obtain a sufficiently large dataset for verification of MO-01. To isolate the measurement system from vehicle-related disturbances, a standalone measurement jig equipped with a Keller Series 26X pressure sensor was used.

Materials and Methods

Measurements were conducted on clear, snow-free lake ice with thicknesses ranging from approximately 12 cm to 24 cm. Ice thickness was estimated from hydrostatic pressure measurements using the method described in Section 7.1. Reference measurements were obtained using a calibrated ruler and the known geometry of the measurement jig. By measuring the portion of the jig protruding above the ice surface, the local ice thickness could be determined independently of the pressure sensor measurement while maintaining direct spatial co-location between the sensor and reference values.
A total of 35 measurements were collected at locations separated by distances ranging from 10 cm to 10 m. To ensure measurement quality, all measurements were subjected to consistency checks based on the geometry of the measurement jig. Six measurements were rejected due to procedural errors or inconsistencies, resulting in a final dataset of 29 valid measurements.
For the pressure-to-thickness conversion, freshwater and ice densities of $999.97\,\mathrm{kg\,m^{-3}}$ and $916.7\,\mathrm{kg\,m^{-3}}$, respectively, were assumed. A pressure offset correction of $-45\,\mathrm{Pa}$ was applied to compensate for sensor positioning, while a correction of $-0.25\,\mathrm{cm}$ was applied to the manual measurements to account for slight ruler penetration into the ice surface. Since the ice surface was free of snow, no freeboard correction was required.

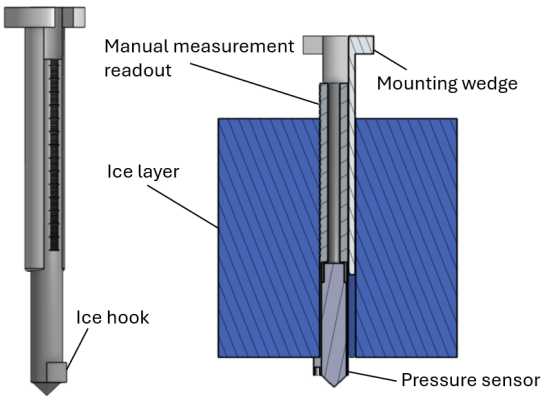


Figure 7.2 Measurement jig used during the St. Moritz validation campaign. The rigid structure ensured repeatable sensor placement and enabled direct comparison between pressure-based and manually derived ice-thickness measurements.

The complete measurement dataset used for the analysis is provided in Appendix G. A summary of the resulting statistical analysis is presented below.

Results and Discussion

The final dataset covered ice thicknesses between approximately 13 cm and 23 cm. Figure 7.3 compares the sensor-derived thickness estimates with the manually determined reference values. Good agreement is observed across the entire measurement range.

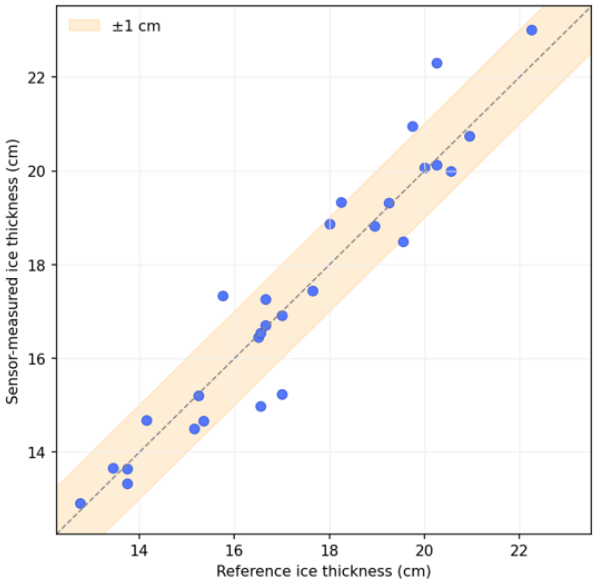


Figure 7.3 Comparison between pressure-sensor ice-thickness estimates and manually determined reference values.

Across the 29 valid measurements, the mean residual between sensor and reference measurements was 0.058 cm, indicating negligible systematic bias. The standard deviation of the residuals was 0.835 cm, which is below the 1 cm uncertainty limit specified by MO-01. The largest observed residual was approximately 1.77 cm. Statistical analysis showed that this value is consistent with the expected spread of the dataset and does not represent an anomalous measurement.
The results demonstrate that the Archimedes-based pressure reconstruction method provides stable and physically consistent ice-thickness estimates. Since both the pressure measurement and the reference measurement were obtained at the same location using the measurement jig, uncertainty contributions from natural spatial ice-thickness variations were minimised. Consequently, the residual distribution provides a close approximation of the true measurement uncertainty of the sensing method.
Part of the observed residual spread still originates from the reference measurements themselves. Manual ruler measurements introduce finite resolution, alignment uncertainty, and reading errors. Assuming a reference measurement uncertainty of approximately 0.5 cm, the intrinsic sensor uncertainty is estimated to be approximately 0.67 cm. This suggests that the sensing method itself performs even better than the combined residual statistics indicate.

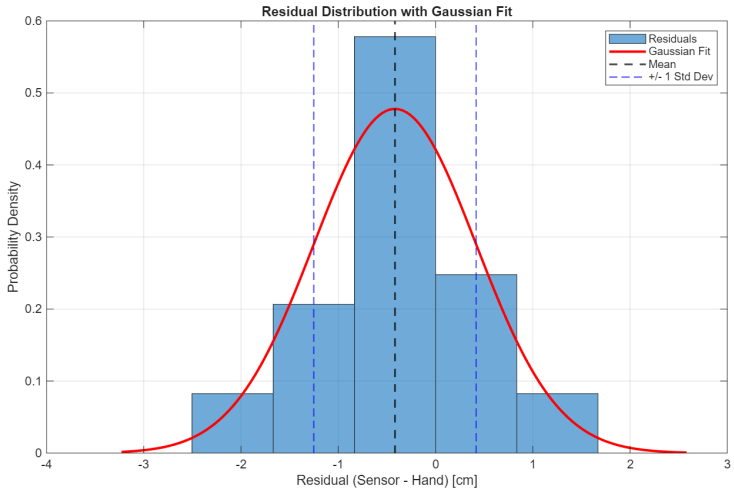


Figure 7.4 Distribution of measurement residuals relative to the mean residual. A Gaussian distribution fitted to the data closely matches the observed residual distribution.

The residual distribution shown in Figure 7.4 is approximately symmetric and follows the expected Gaussian shape. For visualisation purposes, the residuals were shifted such that the observed mean residual was centred in the middle of the central histogram bin, allowing the variance distribution to be

represented more clearly. This shift does not affect the underlying statistics. The close agreement between the measured histogram and the fitted Gaussian distribution indicates that the remaining measurement uncertainty is well approximated by normally distributed random error and that no dominant systematic effects remain in the dataset. This behaviour increases confidence both in the sensor performance and in the validity of the experimental methodology used for requirement verification.

Conclusion

The St. Moritz measurement-jig campaign provided the primary validation dataset for the POLARIS ice-thickness measurement system. Of the 35 acquired measurements, 29 were retained after quality filtering and used for analysis. The resulting mean residual of 0.058 cm and standard deviation of 0.835 cm demonstrate compliance with the measurement uncertainty requirement specified in MO-01. This campaign therefore constitutes the principal evidence for successful achievement of the project's primary scientific objective.

7.2.3 Zermatt Full-System Demonstration

Introduction

The final field campaign was conducted on Schwarzsee near Zermatt using the complete vehicle. Unlike the dedicated validation campaign presented previously, the objective was not to verify MO-01 but to evaluate the ice-thickness measurement system under realistic mission conditions. Particular focus was placed on identifying operational failure modes.

Materials and Methods

The vehicle was manually operated beneath an ice sheet with an approximate thickness of 70 cm. Ice thickness was measured using the Archimedes-based contact method described in Section 7.1. Measurements were collected on a predefined grid consisting of measurement sites distributed across the lake. Logistical time constraints prohibited the vehicle from completing the full grid, which resulted in only nine measurement locations being completed.

Figure 7.5 Measurement grid used during the Zermatt field campaign.

Due to technical issues with the Keller Series 26X pressure sensor used during the St. Moritz validation campaign, the vehicle was operated using a Blue Robotics Bar30 pressure sensor. The Keller sensor achieved an estimated intrinsic measurement uncertainty of approximately 0.67 cm during the previous campaign, while the Bar30 sensor is specified with an accuracy of approximately ±2 cm. Consequently, reduced measurement precision was expected during this campaign.
Only a limited number of reference measurements were available, which was anticipated during test planning due to the effort required to create validation holes through approximately 70 cm of ice. The

campaign therefore focused primarily on sensor behaviour, operational robustness, and identification of error sources.
In addition to the contact-based measurement method, a low-cost ultrasonic distance sensor was evaluated. The known offset between the ultrasonic sensor and the top of the vehicle allowed it to act as an independent indication of whether the vehicle was in contact with the ice sheet. The sensor was therefore used as part of the ice-detection and filtering logic. Furthermore, it provided continuous measurements of the distance to the ice underside, enabling continuous ice-thickness estimates between contact measurements. The sensor has a manufacturer specified accuracy of approximately ±0.6 cm at a measurement distance of 20 cm.

Results and Discussion

The campaign successfully demonstrated that the Archimedes-based measurement method can be integrated into vehicle operations and used to acquire measurements over extended areas. The most significant finding was the identification of horizontal vehicle drift relative to the ice sheet as the dominant source of measurement variability.
Several measurement locations exhibited multiple peaks in their thickness distributions rather than the single Gaussian distribution expected from sensor noise alone.

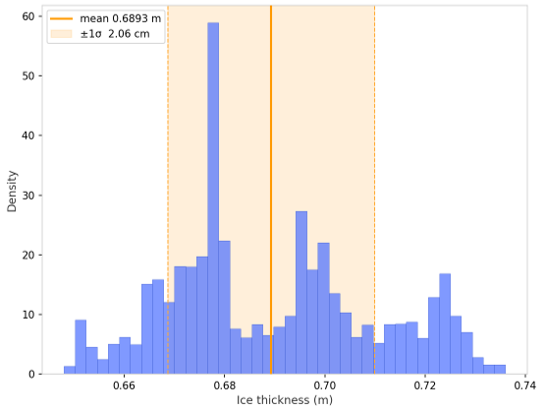


Figure 7.6 Ice-thickness distribution with multiple visible peaks, despite measurements being acquired at a single nominal location.

These peaks indicate that the vehicle remained in contact with the ice while drifting laterally beneath it. Since ice thickness varies spatially, small horizontal displacements result in different valid thickness measurements being recorded at what is nominally the same measurement location.

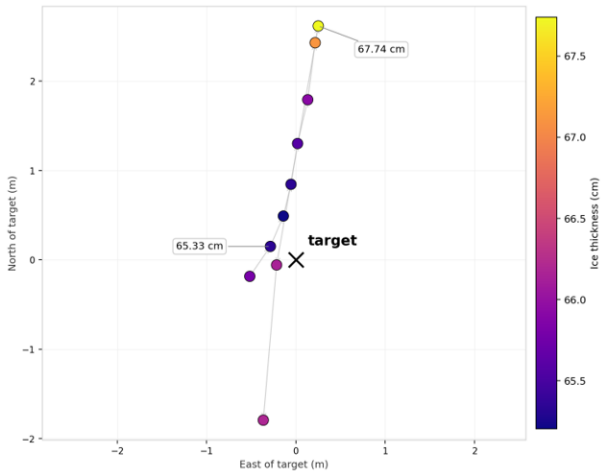


Figure 7.7 Illustration of positional-drift failure mode. While maintaining contact with the ice sheet, horizontal vehicle motion causes measurements to be acquired at neighbouring locations with different ice thicknesses.

The temporal evolution of the measurements supports this interpretation. At Grid Point 1, the measured thickness increased from approximately 65.2 cm to 67.9 cm over a period of approximately 205 s. The

correlation coefficient between thickness and time was 0.83, indicating a systematic trend rather than random sensor noise. Similar behaviour was observed at several measurement locations.

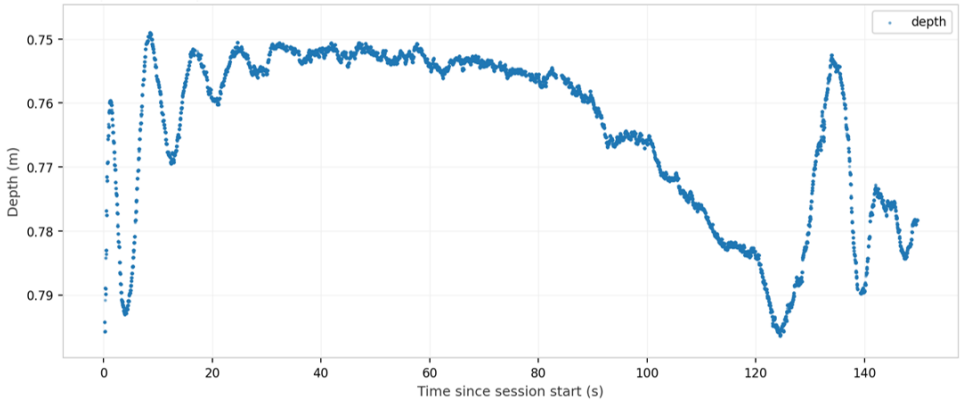


Figure 7.8 Depth as a function of time at a representative measurement point. Short-term oscillations originate from vehicle depth-control manoeuvres, whereas the long-term trend indicates horizontal drift beneath the ice sheet while maintaining ice contact.

A possible alternative explanation is that the vehicle was simply changing depth or intermittently losing contact with the ice surface. However, this is unlikely due to the vehicle's positive buoyancy, which continuously presses it against the ice sheet. Furthermore, active depth-control manoeuvres produce short-term oscillations similar to those visible at the beginning and end of Figure 7.8, rather than the gradual trend observed throughout the measurement. The latter is therefore more consistent with horizontal drift beneath the ice sheet.

Based on this interpretation, a filtering method was developed to better isolate individual measurement locations and reduce the influence of horizontal vehicle drift between neighbouring points with naturally varying ice thicknesses. Measurements without valid localisation, excessive roll or pitch, negative thickness estimates, or measurements more than 1 cm below the shallowest recent depth were rejected.

The filtering reduced the median within-point standard deviation from approximately 0.96 cm to 0.41 cm, corresponding to a reduction of 57 %. At the same time, the overall thickness range across the grid decreased only from 8.3 cm to 6.9 cm. This indicates that the filtering primarily removed variability introduced by vehicle motion while preserving the underlying spatial ice-thickness distribution. Although some measurements are discarded and thinner local ice regions are preferentially retained, this introduces a conservative bias that is favourable for safety-oriented applications, as underestimating ice thickness is generally preferable to overestimating it.

The ultrasonic measurements further reinforced the conclusions drawn from the contact measurements. Unlike the Ping sonar tested previously, which produced partly unphysical results and excessive variance, the new ultrasonic sensor generated physically plausible thickness estimates that followed the same large-scale trends as the Archimedes-based measurements.

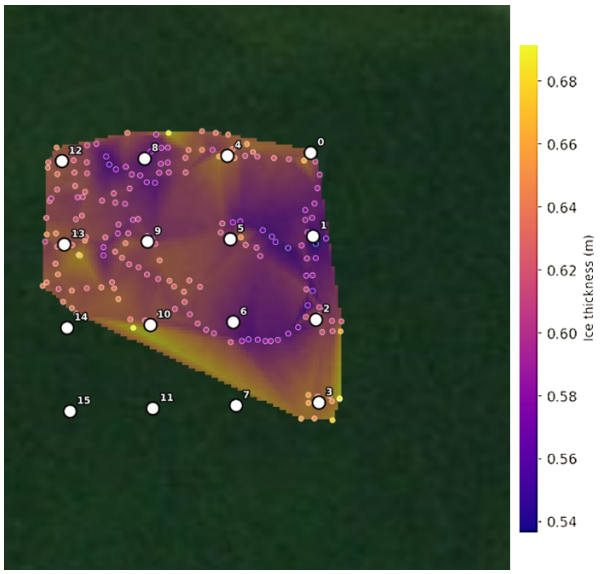


Figure 7.9 Continuous ice-thickness map generated from ultrasonic measurements. Measurements were aggregated into 1 m spatial bins to reduce noise while preserving thickness variations across the survey area.

The ultrasonic sensor enabled continuous ice-thickness measurements between the discrete contact-based measurement locations. Figure 7.9 shows the resulting ice-thickness map, generated by aggregating measurements into 1 m spatial bins. Although the limited number of reference measurements prevents a quantitative accuracy assessment, the map demonstrates that continuous ultrasonic measurements can capture meaningful spatial thickness variations and can therefore complement the more accurate Archimedes-based contact measurements when surveying larger areas.

Conclusion

The Zermatt campaign demonstrated the practical applicability of the ice-thickness measurement system under realistic mission conditions. Vehicle drift relative to the ice sheet was identified as the dominant operational error source, and an effective filtering methodology was developed to mitigate its effects. The filtering reduced within-point measurement variability by 57 % while preserving the overall spatial thickness distribution. The campaign further highlighted that manual vehicle operation can introduce positional drift during measurements, reducing repeatability and causing measurements to represent a local "point"-area average rather than a single point value. These effects are discussed further in Section 7.3. Finally, the campaign reinforced the robustness of the Archimedes-based contact measurement method and demonstrated the potential of continuous ultrasonic measurements for large-area ice-thickness mapping.

7.3 Limitations and Future Work

The ice-thickness measurement method relies on assumptions regarding ice and water density. While these assumptions are reasonable for the lake ice encountered during testing, deviations caused by snow loading, air inclusions, bubble layers, slush ice, or other anomalous ice structures can introduce systematic errors. Such effects are difficult to quantify without additional sensing modalities and dedicated scientific investigation, and addressing them was beyond the scope of the POLARIS project. Furthermore, the Zermatt campaign showed that vehicle motion relative to the ice sheet can reduce measurement repeatability by causing measurements to represent a local area average rather than a single point value. While filtering can mitigate this effect, it cannot fully eliminate it and introduces a bias towards measurements representing locally thinner ice regions.
An additional limitation concerns the verification campaign itself. While Measurement Requirement MR-01 specifies an absolute accuracy of 1 cm over an ice-thickness range of 0 cm to 100 cm, practical verification was limited to ice thicknesses between approximately 10 cm and 20 cm. This range was selected because it represents the most operationally relevant ice conditions discussed in Section 1.1 while allowing safe acquisition of reference measurements. Consequently, although the demonstrated

performance provides confidence in the measurement principle, compliance with MR-01 has only been experimentally verified over this subset of the specified operating range.
Future work should therefore include a comprehensive validation campaign spanning the full ice-thickness range specified by MR-01. Such testing would verify measurement performance under a broader range of environmental conditions and increase confidence that the demonstrated accuracy is maintained throughout the complete operational envelope. In parallel, measurement repeatability should be improved through active position hold and automated grid execution rather than depth hold alone, thereby reducing horizontal vehicle drift. Finally, the Keller Series 26X pressure sensor should be integrated into the final vehicle configuration, as its demonstrated uncertainty is significantly lower than that of the Blue Robotics Bar30 sensor used during the Zermatt campaign.

CHAPTER 8

Conclusion

The POLARIS project set out to replace the process of slow manual ice-drilling with a fully autonomous underwater platform capable of generating accurate, georeferenced ice-thickness maps in alpine environments. Over two semesters in collaboration with the Department of Earth and Planetary Sciences at ETH Zurich, a core team of ten students designed, manufactured, and field-validated a 1.7-metre AUV platform and the whole infrastructure needed for operation around it from the ground up. Most notably, full system validation was achieved in a five-day mission at Schwarzsee, Zermatt.

Out of the four mission objectives, the two most critical were met. First, the ice-measurement system reached its target accuracy of ±1 cm (MO-01). This was established in a dedicated validation campaign on the homogeneous ice at Lake St. Moritz, where 29 co-located comparisons against ruler ground truth yielded a mean residual of 0.058 cm and a standard deviation of 0.835 cm, within the ±1 cm requirement. This evidence spans the 13 cm to 23 cm ice range; verification across the full 0 cm to 100 cm range specified by MR-01 remains future work (Section 7.3). The subsequent Schwarzsee campaign demonstrated the measurement pipeline under realistic mission conditions on multi-layered ice, but, operating on a lower-accuracy sensor (±2 cm) with only limited reference holes, it served to characterise operational performance and failure modes rather than to verify MO-01. Second, the localisation stack achieved a horizontal accuracy within ±2 m under field conditions, fulfilling MO-03. This was done by fusing through-ice GNSS position fixes with continuous dead-reckoning, which kept long-term drift within bounds.
Full autonomous mission execution was demonstrated at Lake Zurich. There, the AUV covered a $20\,\text{m} \times 20\,\text{m}$ area and returned to its starting position, validating the complete autonomy stack from behaviour-tree execution down to velocity control and thruster actuation (MO-04). This demonstrates that the autonomy stack functions end-to-end, but only under these limited conditions. It has not yet been verified at the full survey scale or under the more demanding conditions of an ice-covered lake. Throughout all operations, the vehicle remained safely tethered and could be disarmed at any moment, satisfying the safety requirements that were central to operating in an enclosed under-ice environment (SR-SW-02 and SR-SW-04). The remaining objective, full coverage of the $10\,000\,\text{m}^2$ survey area (MO-02), was not achieved within the project timeline. The $40\,\text{m} \times 40\,\text{m}$ grid test at Lake Zurich was aborted after DVL outages on the final leg. This points to the main open challenge: improving the redundancy of the state-estimation pipeline so it can tolerate occasional sensor outages during extended missions. Scaling the path-following and localisation stack to the full survey dimensions remains the most concrete next step for the project.

The platform, its software stack, and the accumulated field data remain a functional, documented baseline for whoever takes the project forward. Just as important is what stays with the team: ten students who started from a concept and ended with a field-validated AUV, having learned to design, manufacture, and operate a complex autonomous system from the ground up. The key outcome of POLARIS is a validated system that demonstrates the feasibility of autonomous under-ice data collection. Building on the improvements identified in the preceding chapters, the system could reach the level of a deployable scientific instrument for ice-covered waters. Whether the next goal is completing the original survey mission or expanding into new environments the necessary system and methods are now in place.

APPENDIX A

General Appendix

A.1 Budget

Table A.1 Project POLARIS Cash Budget Summary

Financial Item	Amount (CHF)	Share (%)	Description
Total Cash Funding Received	45,158	100.0%	Direct cash funding received through grants and project funds.
Total Cash Expenditure	26,840	59.4%	Cash expenditure for hardware, manufacturing, and field logistics.
Project Continuation Fund	**18,318**	**40.6%**	Remaining cash funds reserved for potential future vehicle iterations.

Table A.2 Project POLARIS Cash Funding Sources

Funding Source	Amount (CHF)	Share (%)	Description
SCOR Foundation	22,805	50.5%	Direct financial support for the development and validation of the POLARIS platform.
IDSC	10,000	22.1%	Academic project funding and institutional support from ETH Zurich.
Guido Feger Foundation	5,000	11.1%	Foundation funding for the POLARIS project.
D-MAVT Sustainability Fund	5,000	11.1%	Departmental sustainability funding for the project.
SWARM Project	2,353	5.2%	Additional project funding transferred to POLARIS.
Total Cash Funding	**45,158**	**100.0%**	Total direct cash funding received by the project.

In addition to direct cash funding, POLARIS benefited from in-kind contributions with an estimated total value of CHF 34,439. These contributions comprised donated equipment, equipment loans, accommodation, transport support, and other services provided by project partners and sponsors. The reported values represent estimated market equivalents and do not constitute direct financial funding received by the project. The four largest estimated in-kind contributions, representing 77.8% of the total estimated in-kind value, are listed in table A.3.

Table A.3 Project POLARIS Main Estimated In-Kind Contributions

Sponsor	Value (CHF)	Share (%)	Description
Zermatt Bergbahnen	10,000	29.0%	Transport costs, mountain railway tickets, and field support.
WaterLinked	7,029	20.4%	DVL navigation system provided for development and field validation.
Chalet Ulysse	5,250	15.2%	Accommodation in Zermatt for five days for ten people.
Panasonic Connect	4,500	13.1%	Operational computer provided for field use.
Top Four Total	**26,779**	**77.8%**	Share of the total estimated in-kind contribution value represented by the four largest contributions.

A.2 Safety Concept

This operational safety concept (SiKo) governed all field deployments on frozen alpine lakes, established in cooperation with ARIS and our advisors. Risk thresholds and mitigation workflows are based on specialist guidelines, including the Canadian Centre for Occupational Health and Safety (CCOHS), with all source links embedded directly within the attached document. Due to compliance with these protocols, no safety incidents occurred across the entire testing campaign.

1. Mission Framework

Mission Identity

Mission Name	
Location	
Date of Operation	
Swisstopo link	

Responsibilities

Role	Name	Phone
Safety Officer (SO) *On site responsible for Safety (not also Mission Control)*		
Deputy Safety Officer (DSO) *Helps SO, replaces SO if needed (can be Mission Control)*		
Mission Control (MC) *Main Mission manager*		

Risk Assessment & Mitigation

Hazard	Mitigation Measure
Ice Breach	Mandatory PFDs, Buddy System, Ice Thickness Testing (min. 10 cm).
Hypothermia	Mandatory thermal layers, Buddy system, breaks
Open Water	Non-swimmer exclusion, Rescue Rope, Buddy System.
Equipment Loss	All submerged systems must be tethered.
Remote Location	Emergency Phone. Cellular connection on site. Evacuation Plan
Weather	48-hour trend check for melt-cycles. Monitor wind chill, visibility. Retreat if < -20 °C, < 50 m visibility, or storm.

Sources (19.03.2025)
https://www.ccohs.ca/oshanswers/phys_agents/cold/cold_health.html
https://www.cbc.ca/news/canada/ottawa/ice-rescue-save-tips-advice-video-cold-1.7074714
https://www.ccohs.ca/oshanswers/hsprograms/work_ice.htmlWeather Forecast

Safety Equipment

Personal Gear (per person)

☒	Items
	Layered warm clothing
	Weatherproof outer shell (wind & water)
	Insulated boots
	Gloves
	Hat (covering ears)
	Scarf
	Extra dry set in waterproof bag: Clothes & shoes
	Insulated bottle with warm liquid (no caffeine!)
	Sunglasses (optional)

Team

☒	Items
	Personal Flotation Devices
	Ice Claws
	Rescue Rope (20 m)
	Thermal Blanket
	First Aid Kit
	Snow Shovel
	Ice Drill
	Measuring stick
	Emergency Phones SO/DSO/MC (charged)
	Walkie-Talkies
	SiKo Hardcopy (SO/DSO)

Safety Briefing

The Safety Officer (SO) and Deputy Safety Officer (DSO) must conduct this briefing with all participants before operations begin.

1. **Safety Concept Walkthrough**: The SO/DSO leads a step-by-step review of the full Safety Concept. All participants must also watch the Ice Rescue video: https://youtu.be/N_1Roqh_jqA?si=7BE3tsWpjKHINeSr.
2. **Gear Testing**: Every participant must physically test their safety equipment (PFD, Rescue Rope, Ice Claws, Radio).
3. **Site Orientation**: Identify the Warm Shelter, Evacuation Route and Emergency Assembly Point on the Site Map.
4. **Buddy System**: Form pairs.
5. Final Confirmation: All participants must verbally confirm they understand the Rules, Abort Criteria, and Emergency Procedures before signing.

Personnel

Buddies: 1&2, 3&4, etc.

#	Name	Phone	Comments, Role	Emergency Contact *(Name + Phone)*	Signature *(Safety Briefing received)*
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					

Operational Safety

Abort Criteria

Trigger	Condition	Action
Personal Doubt	Any member expresses a safety concern or intuitive "bad feeling".	STOP for SO/DSO re-evaluation.
Ice Quality	Thickness <10 cm (transit) / <15 cm (work) or presence of Grey/Dark ice.	RETREAT via ingress path to shore.
Weather	Visibility < 50 m / below −20 °C / Storm OR severe rain with heavy winds	RETREAT to warm shelter. (Site map)
Comms/Gear	Walkie-talkie link lost or mandatory PPE (PFD/Ice Claws) is damaged.	SUSPEND technical work until comms/gear are 100% functional.

Rules

Operations On Ice (ice thickness < 27cm)

Ice Clearance	Entry onto ice is **strictly prohibited** until the Safety Officer **grants clearance** based on Ice Assessment. Designated **Safe Routes** must be clearly marked and followed.
Mandatory on Ice PPE	All personnel **on** the **ice** must wear a **PFD**. **Beyond 15 m** from the shore-based safety watch, **ice claws** must also be worn for immediate deployment.
Personnel on Ice	Access to the ice is restricted to the **minimum** required for the task. Personnel on ice must **keep their distance**.
Safety Watch	The SO or DSO must remain on shore as a dedicated **Safety Watch**, providing uninterrupted visual and verbal monitoring of the team and coordinating any necessary emergency response. The **rescue rope** must be staged and **ready** for immediate use at the safety watch station.
Radio Contact	The **on-ice team** must maintain constant **verbal contact** with the Safety Watch (SO/DSO) via Walkie-Talkie. A radio check must be performed and confirmed before any participant steps onto the ice.
Authorities	Local authorities must be informed in advance regarding the specific location, duration, and scope of the work.

Universal

Buddy System	No participant may work alone. Pairs must maintain contact at all times. In an emergency, the buddy pair must move together.
Hypothermia	Continuously monitor yourself and your buddy for signs of frostbite (numbness/pale skin) or hypothermia (shivering/confusion).
Non-Swimmer Restriction	Non-swimmers are prohibited from participating.
System Recovery	All equipment deployed in the water must be **tethered** to a fixed anchor point.
Trip Prevention	**Work areas** must be **kept clear** of loose gear to prevent tripping hazards into the water.
Avoid overexertion	Take regular **breaks** (small break every 1–2 hours, extended break every 4 hours). **Drink** warm liquids (avoid caffeine) and **eat** enough food.
Safety Briefing	Every participant must receive a formal briefing on this Safety Concept and specific instruction on emergency procedures prior to deployment.
Active Responsibility:	Every participant must think ahead and voice any safety doubts or "bad feelings" immediately. All concerns are taken seriously.

Ice Assessment

Thickness Thresholds (on foot)

Activity	Threshold **minimum**
Transit (<2 h)	> 10 cm
Stationary Work (>2 h)	> 15 cm
No special ice rules but extra caution on ice	> 27 cm
Measurement Frequency	every ≤5 m

Ice Quality Modifiers

- **Blue Ice:** High-density, strongest structural integrity.
- **White/Snow Ice:** Contains air bubbles; only 50% as strong as blue ice. Double thickness requirements.
- **Grey/Dark Ice:** Indicates melting or water saturation. Prohibited.
- **Refrozen Ice:** Avoid areas that have recently thawed and refrozen, as the bond between layers is weak.

Environmental Warning Signs

- **Acoustic Signs:** High-pitched "singing" or low-pitched "booming" (structural settling).
- **Visual Signs:** Rising water on top of ice, mist/steam (indicates open water nearby), or fresh cracks.
- **Snow Cover:** Treat snow-covered ice with extreme caution; snow acts as an insulator, often hiding thin ice.

Measurement Log

Time	Location	Assessment (Ice Thickness, Quality)	Who (SO, DSO)

2. Emergency Procedures

Emergency	112
Rega	1414
Jared Krause (ARIS President)	+41 79 364 53 86
Local Contact 1	
Local Contact 2	
Local Contact 3	

Emergency alert

1. **Who** is calling?
2. **What** happened?
3. **Where** is the location?
4. **When** did it happen?
5. **How many** victims/injured?

Scenarios

Fall through ice

1. Victim: **Stay calm!** Main priority: not drowning. Take slow, deep breaths.
   Others: Do not enter the water. Sound alarm "Alarm, Person eingebrochen!"
2. Use **Ice claws** + Throw **rescue rope.**
3. Instruct victim to **kick** and **slide horizontally onto ice**.
4. Pull victim onto ice, move to **safe zone**.
5. **Remove wet clothing**; wrap in blanket; gently warm.
6. If hypothermia: Call 112

Fall in open water

1. **Stay calm!** Main priority: not drowning. Take slow deep breaths. Others must not enter the water.
2. Throw **rescue rope.**
3. Help victim out of water, move to **safe zone**.
4. **Remove wet clothing**; wrap in blanket; gently warm.

Hypothermia

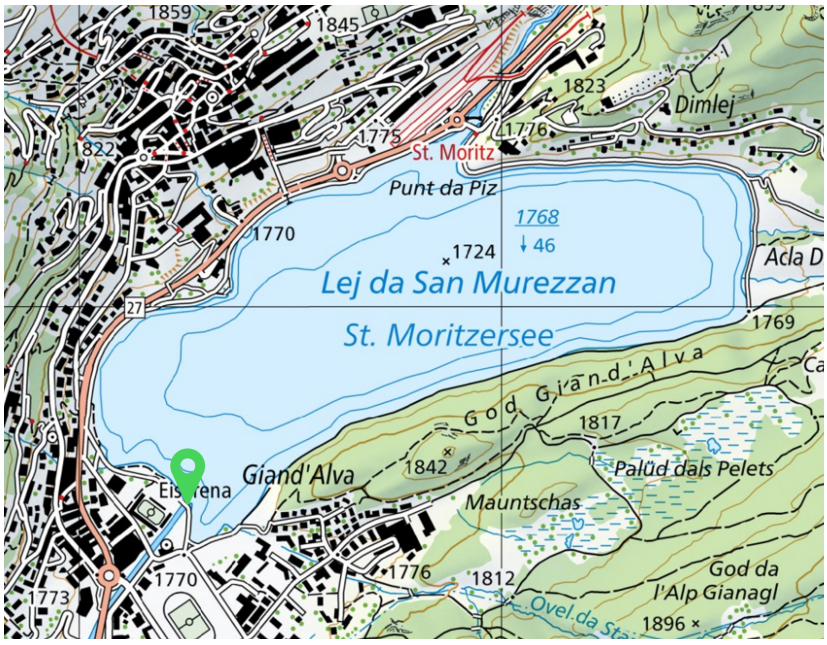
- **Do not:** massage skin, rewarm too quickly, use very hot objects directly.
- **Cold Injuries / Frostbite:** Numb, tingling, pale skin → immediately warm affected area with body contact or warm clothes. Avoid rubbing or applying direct heat.
- **Mild:** shivering, fatigue, poor coordination → move to shelter, remove wet clothing, give warm non-caffeinated drinks, gentle warming.
- **Moderate/Severe:** weak/no shivering, dilated pupils, slowed breathing → call 112, lay down, gentle handling, monitor breathing.

Site Map

What	Icon	Details
Top Station		
Warm shelter		
Emergency assembly point		
Evacuation route		

Detailed

Overview



APPENDIX B

Requirements Appendix

Requirement Definitions

The technical scope of the POLARIS project is structured as a top-down hierarchy divided into three distinct requirement levels:

1. **Mission Objectives** (MO): The high-level scientific and operational success criteria agreed upon with the project partner, D-EAPS.
2. **Mission-Level Requirements** (MR): High-level functional boundaries and safety constraints defining *what* the system must achieve in the field, independent of the physical platform's specific design.
3. **System-Level Requirements** (SR): Technical specifications defining *how* the chosen robotic system execution satisfies the parent mission-level parameters.

System-level requirements are assigned directly to their respective engineering domains:

- Ice Thickness Measurement (ICE)
- Localisation (LOC)
- Hardware Platform (HW)
- Software & Autonomy (SW)

Definition of System: For the scope of these requirements, the *system* comprises all physical and digital components necessary to complete the mission. This includes the hardware platform, the tether assembly, the top station, and all software frameworks executing for mission completion.

B.1 Mission Objectives (MO)

Compliance with the Mission Objectives is evaluated in the final conclusion (Chapter 8).

ID	Mission Objective	Reasoning
MO-01	Develop a system capable of ice-thickness measurements with an accuracy of ±1 cm.	This precision matches the Global Climate Observing System (GCOS) benchmark [17]. Maintaining a low uncertainty is essential because variations in early-winter ice formation occur on a centimetre scale, making high accuracy critical for valid structural and safety assessments.
MO-02	Cover an area of 10 000 m^2 to enable datapoint acquisition with a resolution of 10 m.	A 100 m × 100 m grid allows the system to completely map smaller alpine lakes or capture a representative section of larger basins like Lake St. Moritz (a primary focus for D-EAPS), while remaining operationally manageable within physical tether length limitations.
MO-03	Localise system in a global coordinate frame to map datapoints to absolute locations with an average accuracy of ±2 m.	Reliable global georeferencing is necessary to pinpoint structural weak points in real-world coordinates and to enable longitudinal studies where repeated survey grids are executed across the entire ice season. For the scientific frameworks utilised by D-EAPS, an average accuracy of ±2 m is sufficient to correlate these datasets.
MO-04	Autonomous mission completion, including navigation, data acquisition, and safe return to the recovery location.	Manual tracking of a dense measurement grid under ice is impractical. External forces cause the vehicle to drift during static measurements, and navigating the narrow water column between the lake bed and the ice ceiling requires active stabilisation; coupling position- and depth-hold into a fully autonomous system guarantees the spatial repeatability required for scientific modelling.

B.2 Mission-Level Requirements (MR)

Mission Level Requirements are verified across the introductory summaries of the main technical chapters Chapter 3 (Robotics Platform), Chapter 4 (Software Platform), Chapter 5 (Localisation), Chapter 6 (Autonomy) and Chapter 7 (Ice Measurement).

ID	Parent	Requirement	Reasoning
MR-01	MO-01	The system shall measure ice thickness between 0 cm and 100 cm with an absolute accuracy of ±1 cm, and with a relative accuracy of ±1 % for thicknesses exceeding 100 cm.	Matches GCOS benchmarks for climate monitoring. Centimetre accuracy is required for structural safety calculations of early-winter thin ice, while a larger relative error is sufficient for thicker sheets.
MR-02	MO-01	The system shall perform consecutive ice-thickness measurements throughout a single deployment.	Enables continuous spatial profiling along the path without stopping the measurement cycle or requiring manual re-initialisation.
MR-03	MO-02	The system shall sustain all mission-critical functions for the full mission duration.	Required to complete the full grid pattern across the complete 10 000 m^2 target area without power or thermal depletion.
MR-04	MO-03	System shall know position of measurement points with an absolute accuracy of ±2 m.	Required to overlay separate survey grids over the season and track local ice changes precisely and map them to actual locations.
MR-05	MO-03	The system shall maintain an onboard estimate of pose and motion with accuracy and update rate sufficient for autonomous path following and safe return to the launch point.	The autonomous path-following controllers need low-latency state vectors to maintain loop stability and prevent cross-track oscillations.
MR-06	MO-03	The system shall generate a georeferenced map of ice thickness.	Core scientific deliverable. Requires matching timestamps between the measurement and localisation data to build the final 2D map.
MR-07	MO-04	The system shall autonomously complete the mission, including navigation, data acquisition, and safe recovery.	Manual tracking under ice is impractical due to the dense measurement grid, and the need to constantly stabilise the sub against hydrodynamic drift.
MR-08	MO-04	The system shall be recoverable without endangering personnel or the environment.	Under-ice deployment has high structural trapping risks; requires mechanical fail-safes to prevent dangerous diving or recovery work.

B.3 System-Level Requirements (SR)

B.3.1 Ice Thickness Measurement (ICE)

Verified in the introductory summary of Chapter 7.

ID	Requirement	Reasoning
SR-ICE-01	System shall measure ice thickness between 0 cm and 100 cm with an absolute accuracy of ±1 cm, and with a relative accuracy of ±1 % for thicknesses exceeding 100 cm.	Direct flow-down from MR-01.
SR-ICE-02	System shall be able to generate a map of the ice layer with its ice-thicknesses.	Implements MR-06 by ensuring data-logging structures can format point matrices into spatial grids during post-processing.
SR-ICE-03	System shall measure depth at an accuracy of 5 mm.	Mathematical requirement for the contact-based Archimedes method. Because of the density ratio of fresh water and ice ($\frac{\rho_{\text{water}}}{\rho_{\text{water}}-\rho_{\text{ice}}} \approx 12$), depth errors scale twelvefold when calculating total thickness. A 5 mm sensor floor is necessary to meet the ±1 cm final target.

B.3.2 Localisation (LOC)

Verified in the introductory summary of Chapter 5.

ID	Requirement	Reasoning
SR-LOC-01	The localisation subsystem shall provide position estimates with a resolution and accuracy sufficient to support station-keeping within 20 cm horizontally and 2 cm vertically in calm water conditions.	Prevents horizontal sub drift from smearing acoustic sensor data or causing multi-modal distributions in a single point measurement phase.
SR-LOC-02	System shall measure its horizontal pose with an accuracy of ±2 m.	Direct implementation of MR-04 to map local data to correct absolute coordinates on the survey grid.
SR-LOC-03	System shall provide odometry data with a frequency of at least 20 Hz.	Minimum feedback rate required by the position controllers to handle current disturbances and avoid steering lag.
SR-LOC-04	The localisation subsystem shall periodically obtain global position fixes to bound accumulated dead-reckoning errors.	Handles open-loop integration drift of Dead Reckoning.
SR-LOC-05	System shall be able to measure the distance to the ice sheet above with an accuracy of 2 cm.	Required for closed-loop docking against the ice underside to prevent damage.

B.3.3 Hardware Platform (HW)

Verified in the introductory summary of Chapter 3.

ID	Requirement	Reasoning
SR-HW-01	The robotic platform shall move at a forward speed of 0.4 m s^{-1}.	Balances overall survey runtime against vehicle battery draw and hydrodynamic drag.
SR-HW-02	The robotic platform shall be waterproof up to a depth of 10 m.	Provides a safety margin to the operational depth which is near the ice sheet (≤4 m).
SR-HW-03	The robotic platform shall be slightly positively buoyant.	Acts as a passive safety mechanism to float the sub to the ice ceiling or water surface during a full power loss, while keeping vertical thruster overhead low during dive.
SR-HW-04	The robotic platform shall actively control itself in 6 −DOF under water.	Allows uncoupled translation and rotation control, which is necessary to counteract cross-currents while keeping the sensors vertical.
SR-HW-05	The robotic platform shall passively control roll to reduce active control effort.	Placing the centre of mass below the centre of buoyancy creates a static righting moment, reducing thruster power and stabilising the sub against roll disturbances.
SR-HW-06	The robotic platform shall not weigh more than 30 kg.	Logistical limit for manual field handling by a two-person team over snow and rough shoreline.
SR-HW-07	The system shall be able to operate in 0 °C water for 3 h.	Ensures physical components and O-rings survive cold water transit without thermal contraction or freezing failures.
SR-HW-08	The system shall provide power for all mission-critical tasks for 180 min.	Calculated from the worst-case power budget for concurrent thrust, compute, and logging, including low-temperature battery capacity loss and safety margin.
SR-HW-09	The system shall be recoverable without endangering personnel or environment.	Direct implementation of MR-08.

B.3.4 Software Platform & Autonomy (SW)

Verified in the introductory summaries of Chapter 4 (Software Platform) and Chapter 6 (Autonomy).

ID	Requirement	Reasoning
SR-SW-01	Software shall enable remote operationability and surveillance of vehicle.	Allows the topside operator to monitor live diagnostics, heartbeat signals, and system health checks before and during deployment.
SR-SW-02	Software shall ensure safe operation of vehicle in real world environments like open lakes.	Prevents vehicle loss by using virtual geofences and allowing immediate manual operator override to protect the sub from crashes.

ID	Requirement	Reasoning
SR-SW-03	Software shall enable live communication and data transfer with the vehicle at a rate of at least $100\,\mathrm{Mbit\,s^{-1}}$.	Required to handle high-bandwidth topside telemetry and raw sonar logs over the tether link in real time.
SR-SW-04	Software shall ensure failsafe mechanisms in case of communication loss.	Prevents uncontrolled manoeuvres during software crashes or hardware failure.
SR-SW-05	Software shall ensure time-synchronised data-logging.	Vital for post-mission data processing; without synced timestamps, real world malfunctions can not be mapped to code failures in recorded logs.
SR-SW-06	Software shall enable full manual operation of the vehicle.	Essential for initial field setup checks and serves as a fallback if autonomous navigation fails.
SR-SW-07	The software stack shall feature a modular architecture (e.g., via ROS 2 nodes) and must strictly maintain a clear separation between Localisation, Autonomy and Control.	Architectural requirement for collaborative team software development. Modularity decouples software system components to ensure that a failure in one node doesn't disrupt interacting nodes.
SR-SW-08	The entire software stack shall be containerised to ease integration from development workstations to onboard target hardware.	Docker containerisation guarantees an exact replication of dependencies from development computers to the onboard computer.
SR-SW-09	Autonomy stack shall be able to take measurements at specified locations, including navigation to and data collection at the measurement point.	Eliminates human piloting errors and enables repeatable, spatially accurate data sampling under ice.
SR-SW-10	System shall be able to traverse a specified, unobstructed area of maximum $10\,000\,\mathrm{m^2}$ in the N-E-plane.	MR-03 translated into a specific software requirement. This ensures the vehicle will be capable of reaching all the specified measurement points.
SR-SW-11	System shall be able to follow straight lines at linear currents of $0.3\,\mathrm{m\,s^{-1}}$ with a cross-track error of 1 m.	Models real environmental conditions of open lakes, where currents of up to $0.3\,\mathrm{m\,s^{-1}}$ can occur. The controller must maintain tight path accuracy despite these disturbances to avoid collisions.
SR-SW-12	System shall always be able to find back to the reference path in an unobstructed operating plane.	Prevents uncontrolled drifting during severe disturbances. The behaviour tree plugin triggers an automatic path replanning when the threshold ($\lvert e_{\text{track}}\rvert > d_{\text{safe}}$) is exceeded to actively bring the AUV back on track.
SR-SW-13	System shall be able to hold its depth within ±25 cm at a cruising speed of $0.4\,\mathrm{m\,s^{-1}}$.	Important for manoeuvring in shallow waters while avoiding collisions.
SR-SW-14	System shall be able to do a controlled rise to the ice sheet with a terminal velocity $<0.1\,\mathrm{m\,s^{-1}}$.	Slows the vehicle down right before ice contact to protect the AUV and the environment from impact damage.

ID	Requirement	Reasoning
SR-SW-15	System shall be able to actively hold its stationary position with a tolerance of 20 cm horizontally and 2 cm vertically in calm water conditions.	Guarantees stable hydrostatic conditions during the stationary ice-contact measurement phase to capture high-precision data profiles without dynamic pressure effects or sensor noise.

APPENDIX C

Robotics Platform Appendix

C.1 Material test

Table C.1 Mechanical Properties of Bambu Lab PETG HF [20]

Property	Testing Method	X-Y Direction	Z Direction
Young's Modulus	ISO 527 / GB/T 1040	1810 ± 190 MPa	1540 ± 130 MPa
Tensile Strength	ISO 527 / GB/T 1040	34 ± 4 MPa	23 ± 4 MPa
Breaking Elongation Rate	ISO 527 / GB/T 1040	8.6 ± 1.2 %	5.1 ± 0.8 %
Bending Modulus	ISO 178 / GB/T 9341	2050 ± 120 MPa	1810 ± 140 MPa
Bending Strength	ISO 178 / GB/T 9341	64 ± 3 MPa	48 ± 4 MPa
Impact Strength (Unnotched)	ISO 179 / GB/T 1043	31.5 ± 2.2 kJ/m²	10.6 ± 1.2 kJ/m²
Impact Strength (Notched)	ISO 179 / GB/T 1043	6.2 ± 1.8 kJ/m²	N/A

Test 1: Preliminary Waterproofing Assessment

- **Objective:** Initial evaluation of 3D-printed PETG HF water ingress resistance.
- **Key Learning:** Baseline validation showing that standard low-wall slicing allows fluid penetration via micro-voids between layers. Higher perimeter loops are required for structural sealing.

Test 2: Parametric Submersion and Absorption Analysis

- **Objective:** Quantify impact of slicing parameters (wall loops, overlap, flow ratio) on weight gain (moisture absorption), print time, and structural density.
- **Methodology:** 8 test objects printed (Bambu Lab X1C, generic PETG HF profile, 50% infill, away-from-overhang seams). Submersion in water for 24 hours. Weight measured with 0.01 g accuracy.

Tested Configurations

- **A:** 6 Wall loops, 30% Infill/Wall overlap (Baseline).
- **B:** 10 Wall loops, 30% Infill/Wall overlap.
- **C:** 6 Wall loops, 50% Infill/Wall overlap.
- **D:** 6 Wall loops, 30% Infill/Wall overlap, Flow ratio adjusted from 0.95 to 1.02.

Table C.2 Material Consumption, Print Efficiency, and Part Density

Config.	Filament [g]	Print Time	Initial Weight [g]	Density [g/cm^8]
A	34.81	1h 26min	17.72	0.941
B	39.87	1h 29min	20.32	1.078
C	34.96	1h 26min	17.79	0.954
D	37.35	1h 26min	19.14	1.016

Table C.3 Temporal Water Absorption and Weight Gain Metrics

	After 6 Hours		After 24 Hours	
Sample	Weight Gain [g]	Absorption [%]	Weight Gain [g]	Absorption [%]
A	0.045	0.254	0.090	0.508
B	0.045	0.221	0.075	0.369
C	0.055	0.309	0.085	0.478
D	0.305	1.593	0.390	2.038

Quantitative Results

Engineering Derivations

- **Wall Thickness (B vs. A):** Increasing wall loops to 10 reduced 24h absorption from 0.508% to 0.369% ($-\sim 27\%$) due to enhanced external boundary sealing, with minimal print time penalty (+3 min).
- **Flow Ratio (D vs. A):** Increasing flow ratio to 1.02 severely compromised print quality on complex geometries, leading to extreme water trapping (2.038% absorption).
- **Final Strategy:** A consolidated setting of **8 wall loops** was selected for production to provide an optimal watertight barrier without unnecessary filament weight, combined with **30% standard / 50% load-bearing infill**.

C.2 Temperature Analysis Electronics Tube

To estimate the steady-state temperature in the electronics tube, the total heat generation inside the tube must first be determined. The heat generation is taken as the sum of the electrical power consumed by all components inside the tube, which is assumed to be fully dissipated as heat, and the losses due to conversion and distribution inefficiencies in the DC-DC converter and battery pack.

C.2.1 Component Power Requirements

We first identify the electrical power consumed by components inside the tube that dissipate their full power as heat:

Table C.4 Electronics inside the tube considered for internal heat generation

Component	Power P_i [W]
NVIDIA Jetson AGX Orin	30.0
Pixhawk flight controller	2.5
Xsens Sirius IMU	3.6
GigaBlox SFP Ethernet switch	3.0
GPS module	0.165

Power values are determined from manufacturer datasheets as follows:

- **NVIDIA Jetson AGX Orin:** Assumed to run in its 30 W power mode
- **Pixhawk flight controller:** Power consumption assumed as 2.5 W, based on published measurements of Pixhawk-class controllers reporting total usage of approximately 2–2.5 W
- **GigaBlox SFP Ethernet switch:** Assumed to draw 3 W under nominal operating conditions, against a datasheet maximum of 4.5 W
- **Other components:** Calculated from nominal operating current and voltage specifications or direct power specifications

Subtotal (Direct Component Heat): Summing up the individual contributions in table C.4 provides us with the following subtotal:

$$\dot{Q}_{\text{components}} = 30.0 + 2.5 + 3.6 + 3.0 + 0.165 \approx \boxed{39.3\ \text{W}} \tag{C.1}$$

C.2.2 Heat Generation from Conversion & Distribution Losses

Now we consider losses from power conversion and distribution inside the tube. PDB resistive losses are neglected, since properly sized copper traces and connectors dissipate less than 1 W. The dominant contributors analysed in this section are the battery pack, ESC, and DC–DC converter losses.

ESC Conversion Losses

The ESCs route power to the thrusters (outside the tube) but retain a fraction as heat due to conversion inefficiency. To choose an operating point for the ESCs during steady cruise, the following assumptions are made:

- **Bus limits:** The ESC supply is $V_{\text{bus}} = 14.8$ V (4S) and the maximum thruster/ESC input current is $I_{\text{max}} = 30$ A.
- **Cruise loading:** At $v \approx 0.5$ m/s, the main pressure-housing is approximated as a smooth cylindrical body, for which the idealised viscous drag is on the order of 1 N. To account for additional form drag from protruding components and installation effects, we conservatively assume an effective cruise drag of $D_{\text{cruise}} \approx 10$ N.
  For the Blue Robotics T200 thrusters, this operating point is taken to correspond to approximately 20% of the rated maximum input current, such that

  $$I_{\text{cruise}} \approx 0.2\, I_{\text{max}}.$$

- **ESC loss model:** ESC efficiency is taken as constant $\eta_{\text{ESC}} = 95\%$. Although the vehicle is equipped with six thrusters, the two lateral (sway) thrusters are not actuated during steady straight-line cruise. The sustained cruise condition relevant to this steady-state analysis is therefore carried by the four remaining thrusters, so $N_{\text{active}} = 4$ is taken. The per-thruster loading (20% of maximum input current) is itself a conservative estimate of the cruise operating point.

With these assumptions:

$$\begin{aligned} I_{\text{cruise}} &= 0.2 \cdot 30 \text{ A} = 6 \text{ A} \\ V_{\text{bus}} &= 14.8 \text{ V} \\ P_{\text{ESC,through}} &= V_{\text{bus}} \cdot I_{\text{cruise}} = 14.8 \text{ V} \cdot 6 \text{ A} = 88.8 \text{ W} \\ \text{Loss fraction} &= 5\% \end{aligned}$$

$$\Rightarrow \dot{Q}_{\text{ESC,single}} = 0.05 \times 88.8 = \boxed{4.44 \text{ W}} \tag{C.2}$$

With four of the six ESCs active at this load, the total ESC heat contribution is:

$$\dot{Q}_{\text{ESC,total}} = 4 \times 4.44 \approx \boxed{17.8 \text{ W}} \tag{C.3}$$

DC-DC Converter Losses

The DC-DC converter steps down the battery voltage to 5 V and powers the following components inside the tube:

Table C.5 Components powered by the DC-DC converter

Component	Power P_i [W]
NVIDIA Jetson AGX Orin	30.0
Xsens Sirius IMU	3.6
GigaBlox SFP Ethernet switch	3.0
Waterlinked DVL	4.0
PingSonar	0.5
Front camera	0.495
GPS module	0.165
Distance sensors (2x, collision detection)	0.1
Blue Robotics pressure sensor	6.25×10^{-3} (≈ 0, neglected)
Total	**41.86**

Power values for components not listed in Table C.4 are calculated from nominal operating current and voltage specifications obtained from manufacturer datasheets.

The total output power delivered by the DC-DC converter is:

$$P_{\text{out,DC-DC}} = 41.86 \text{ W} \tag{C.4}$$

A DC-DC converter efficiency of 90% is assumed, representing a typical efficiency for a step-down buck converter.
The resulting heat dissipated by the DC-DC converter is:

$$\dot{Q}_{\text{DC-DC}} = P_{\text{out,DC-DC}} \left(\frac{1}{0.9} - 1 \right) = 41.86 \text{ W} \times 0.1111 \approx \boxed{4.7 \text{ W}} \tag{C.5}$$

Battery Pack Losses

We consider additional losses in the battery pack due to internal resistance. The following assumptions are made:

- **Battery pack voltage:** The nominal pack voltage is $V_{\text{batt}} = 14.8$ V (4 cells with 3.7 V each)

- **Battery pack internal resistance:** The effective internal resistance is $R_{\text{batt,int}} = 0.02\ \Omega$. To obtain this estimate, we assume a single cell internal resistance of 0.01 Ohm and multiply by 4 cells (4S). This obtained resistance is then halved, as our system consists of two packs in parallel.
- **Battery current:** Based on the total system power of approximately 244 W, the battery current is:

$$I_{\text{batt}} = \frac{P_{\text{total}}}{V_{\text{batt}}} = \frac{244\ \text{W}}{14.8\ \text{V}} = 16.5\ \text{A} \tag{C.6}$$

The resulting heat dissipated by the battery pack is:

$$\dot{Q}_{\text{batt}} = I_{\text{batt}}^2 \cdot R_{\text{batt,int}} = (16.5\ \text{A})^2 \times 0.02\ \Omega \approx \boxed{5.4\ \text{W}} \tag{C.7}$$

Subtotal (Conversion & Distribution Losses)

Summing up the losses from the ESC, DC-DC converter, and battery pack yields:

$$\dot{Q}_{\text{losses}} = \dot{Q}_{\text{ESC,total}} + \dot{Q}_{\text{DC-DC}} + \dot{Q}_{\text{batt}} = 17.8\ \text{W} + 4.7\ \text{W} + 5.4\ \text{W} = \boxed{27.9\ \text{W}} \tag{C.8}$$

Total heat generation

The total heat generated inside the tube is the sum of direct component dissipation and all conversion/distribution losses:

$$\dot{Q}_{\text{total}} = \dot{Q}_{\text{components}} + \dot{Q}_{\text{losses}} = 39.3\ \text{W} + 27.9\ \text{W} = \boxed{67.2\ \text{W}} \tag{C.9}$$

C.2.3 Thermal Resistance Analysis

To determine the steady-state temperature inside the electronics tube, we must analyse the thermal resistances through which heat is dissipated to the surrounding water. We first specify the relevant dimensions and thermal properties:

Electronics Tube Dimensions:

$$\begin{aligned} r_1 &= 0.0745\ \text{m} && \text{(inner radius)} \\ r_2 &= 0.080\ \text{m} && \text{(outer radius)} \\ L &= 0.70\ \text{m} && \text{(length)} \end{aligned}$$

Thermal Properties:

$$\begin{aligned} \alpha_{\text{air}} &= 5\ \text{W/(m}^2\text{K)} && \text{(natural convection)} \\ \lambda_{\text{acrylic}} &= 0.20\ \text{W/(mK)} && \text{(thermal conductivity)} \\ \alpha_{\text{water}} &= 4000\ \text{W/(m}^2\text{K)} && \text{(forced convection, } v = 0.5\ \text{m/s)} \end{aligned}$$

For conduction through the cylindrical wall, the thermal resistance is given by:

$$R_{\text{cond}} = \frac{\ln\left(\frac{r_2}{r_1}\right)}{2\pi\,\lambda_{\text{acrylic}}\,L} \tag{C.10}$$

For our given specifications, this results in the following thermal resistance for the conduction through the acrylic wall of our electronics tube:

$$R_{\text{cond}} = \frac{\ln\left(\frac{r_2}{r_1}\right)}{2\pi\,\lambda_{\text{acrylic}}\,L} = \frac{\ln\left(\frac{0.080\ \text{m}}{0.0745\ \text{m}}\right)}{2\pi \cdot 0.20\ \text{W/(m K)} \cdot 0.70\ \text{m}} \approx 0.081\ \text{K/W} \tag{C.11}$$

Additionally, we must consider the resistance that arises due to convection at the inner surface of the tube (from the internal air) and at the outer surface (to the surrounding water). These thermal resistances are given by:

$$R_{\text{conv,air}} = \frac{1}{\alpha_{\text{air}} 2\pi r_1 L} \tag{C.12}$$

$$R_{\text{conv,water}} = \frac{1}{\alpha_{\text{water}} 2\pi r_2 L} \tag{C.13}$$

Plugging in our given specifications results in the following convective thermal resistances:

$$R_{\text{conv,air}} = \frac{1}{5\ \text{W/(m}^2\,\text{K)} \cdot 2\pi \cdot 0.0745\ \text{m} \cdot 0.70\ \text{m}} \approx 0.610\ \text{K/W} \tag{C.14}$$

$$R_{\text{conv,water}} = \frac{1}{4000\ \text{W/(m}^2\,\text{K)} \cdot 2\pi \cdot 0.080\ \text{m} \cdot 0.70\ \text{m}} \approx 7.10 \times 10^{-4}\ \text{K/W} \tag{C.15}$$

Since these resistances are in series, their sum yields the total thermal resistance for the radial heat transfer through the electronics tube:

$$\begin{aligned} R_{\text{radial}} &= R_{\text{conv,air}} + R_{\text{cond}} + R_{\text{conv,water}} \\ &= 0.610\ \text{K/W} + 0.081\ \text{K/W} + 7.10 \times 10^{-4}\ \text{K/W} \\ &= 0.692\ \text{K/W} \end{aligned} \tag{C.16}$$

In addition to the radial heat transfer through the electronics tube, heat can also be transferred through the aluminium end caps of our tube. For that the relevant dimensions are:

Aluminium End Cap Dimensions:

$$\begin{aligned} d_{\text{cap,aluminium}} &= 0.006\ \text{m} && \text{(thickness)} \\ r_{\text{cap}} &= 0.080\ \text{m} && \text{(radius)} \\ A_{\text{cap}} &= 0.0201062\ \text{m}^2 && \text{(area)} \\ \lambda_{\text{alu}} &= 237\ \text{W/(m K)} && \text{(thermal conductivity)} \end{aligned}$$

Calculated Thermal Resistances: For the aluminium end caps, the relevant thermal resistances are given by:

$$R_{\text{conv,air}} = \frac{1}{\alpha_{\text{air}} A_{\text{cap}}},$$

$$R_{\text{cond,alu}} = \frac{d_{\text{cap}}}{\lambda_{\text{alu}} A_{\text{cap}}},$$

$$R_{\text{conv,water}} = \frac{1}{\alpha_{\text{water}} A_{\text{cap}}},$$

This leads to the following values:

$$\begin{aligned} R_{\text{air}} &= 9.9471\ \text{K/W} \\ R_{\text{alu}} &= 1.26 \times 10^{-3}\ \text{K/W} \\ R_{\text{water}} &= 0.0124\ \text{K/W} \end{aligned}$$

Combining these resistances in series, we can calculate the total thermal resistance for one of the end caps:

$$\begin{aligned} R_{\text{cap,alu}} &= R_{\text{air}} + R_{\text{alu}} + R_{\text{water}} \\ &= 9.9471\ \text{K/W} + 1.26 \times 10^{-3}\ \text{K/W} + 0.0124\ \text{K/W} \\ &= 9.9608\ \text{K/W} \end{aligned} \tag{C.17}$$

Since there are two caps, the thermal resistance is halved:

$$R_{\text{caps}} = \frac{R_{\text{cap,alu}}}{2} = \frac{9.9608}{2} = 4.9804\ \text{K/W} \tag{C.18}$$

C.2.4 Steady-State Temperature Calculation

After having calculated the individual thermal resistances, we can now calculate the change in temperature inside the tube that leads to a steady-state condition. We consider the resistances for the radial heat transfer through the electronics tube and the thermal resistance of the aluminium end caps to be in parallel. This leads to a total thermal resistance of:

$$\frac{1}{R_{\text{total}}} = \frac{1}{R_{\text{radial}}} + \frac{1}{R_{\text{caps}}} = \frac{1}{0.692} + \frac{1}{4.9804} = 1.646 \text{ W/K} \tag{C.19}$$

$$R_{\text{total}} = \frac{1}{1.646 \text{ W/K}} = 0.608 \text{ K/W} \tag{C.20}$$

At steady-state, the heat generated inside the tube must equal the heat dissipated:

$$\dot{Q}_{\text{total}} = \dot{Q}_{\text{out}} = \frac{\Delta T}{R_{\text{total}}} \tag{C.21}$$

Rearranging to solve for the temperature difference:

$$\Delta T = \dot{Q}_{\text{total}} \cdot R_{\text{total}} = 67.2 \text{ W} \cdot 0.608 \text{ K/W} \approx \boxed{40.9 \text{ K}} \tag{C.22}$$

Since our system operates beneath frozen lake environments, we assume a conservative ambient water temperature of $T_{\text{ambient}} = 4\,^{\circ}\text{C}$. This gives us an internal temperature of:

$$T_{\text{internal}} = T_{\text{ambient}} + \Delta T = 4 + 40.9 = \boxed{44.9\,^{\circ}\text{C}} \tag{C.23}$$

At this internal temperature, all components inside the electronics tube remain within their rated operating temperature range.

C.2.5 General Assumptions

1. The power consumption of the onboard systems amounts to $P_{\text{sys}} = 244\,\text{W}$.
2. The transit velocity is constant at $v = 0.5\,\text{m}\,\text{s}^{-1}$. Immediately after each stop, it is assumed that the velocity returns instantaneous to v.
3. The system requires a supply voltage $U = 4S$ configuration, assuming a nominal cell voltage of $S = 3.7\,\text{V}$ per cell.
4. The system operates in an environment with an average temperature of $T = 0\,^{\circ}\text{C}$
5. The available installation space for the battery is a cylindrical volume with a diameter of $d = 0.149\,\text{m}$ and a length of $l = 0.2\,\text{m}$. This corresponds to a total geometric volume of $V = 0.00349\,\text{m}^3$.
6. We follow the underwater navigation strategy illustrated in Figure C.1 and described in Section **??**.
7. At every observation point, the AUV comes to an immediate stop. A dwell time of $t_{\text{stop}} = 10\,\text{s}$ is assumed. In addition, up to four GNSS stop phases of $t_{\text{GNSStop}} = 180\,\text{s}$ are planned for system checks and GNSS signal acquisition. If the first two GNSS connection attempt fails, no further attempts (third and fourth) are carried out.
8. The assumption is made that the AUV can find no GNSS signals
9. In total 121 stops are required, where the measurements are taken.

Unknowns

The following unknowns must be determined from this assumptions:

1. Total path length, which the AUV drives along s
2. Complete endurance of the operation in the water t
3. Total energy consumption of the operation E_{sys}
4. Required battery energy E_{batt} by accounting for the overall efficiency of the system $\eta_{sys} = 0.9$, which includes losses in cables, converters, and the battery management system.
5. Adjusted energy consumption to $E_{batt,DoD}$ including a maximum DoD of 80%

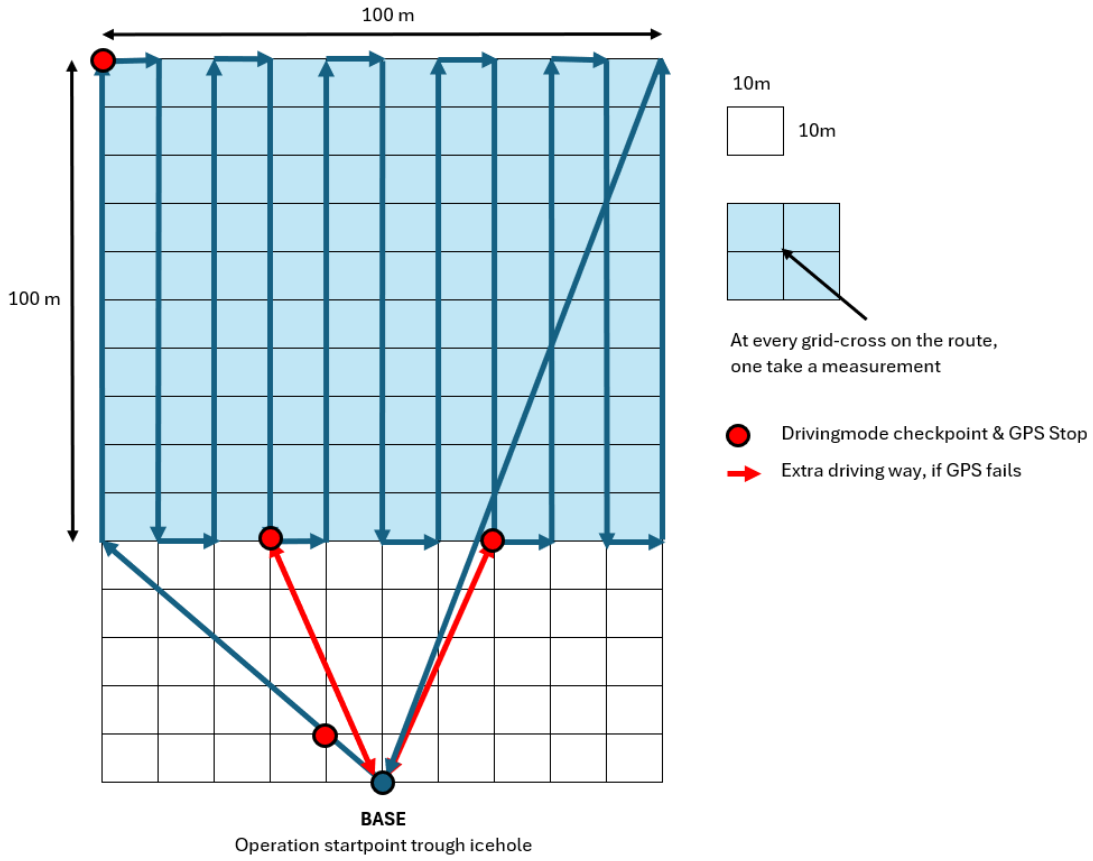


Figure C.1 Schematic overview of the planned AUV trajectory, starting at the base start point and covering the blue measurement grid.

6. Adjusted energy consumption to E_{batt,DoD,f_T} including a capacity factor of $f_T = 0.8$, due to harsh environmental temperature conditions.
7. Adjusted energy consumption to $E_{\text{batt,DoD},f_T,f_s}$ including a safety factor of $f_s = 0.95$, due to general component and operating uncertainties.
8. The volumetric energy density ρ in watt-hours per litre (Wh/L)
9. Charge capacity Q of the battery in ampere-hours (Ah).

Driving strategy and s, t derivation

The system follows the path exactly as shown in Figure
The path length s is calculated as follows:

$$s = 11 \cdot 100\,\text{m} + 10 \cdot 10\,\text{m} + \sqrt{50^2 + 50^2}\,\text{m} + 4 \cdot \sqrt{20^2 + 50^2}\,\text{m} + \sqrt{50^2 + 150^2}\,\text{m} \quad \approx 1{,}644.23\,\text{m} \tag{C.24}$$

Sidenote:
From equation (C.24) one concludes, that because of the drift, we have to take a total detour of 215.40 m, if we cannot find a GNSS signal, which costs us 0.12 h, regarding equation (C.25).

$$t_{detour} = \frac{s}{v} = \frac{215.4\,\text{m}}{0.5\,\text{m}\,\text{s}^{-1}} = 430.8\,\text{s} \approx 0.12\,\text{h} \tag{C.25}$$

Now t is calculated with s:

$$t_{driving} = \frac{s}{v} = \frac{1{,}644.23\,\text{m}}{0.5\,\text{m}\,\text{s}^{-1}} = 3{,}288.46\,\text{s} \approx 0.91\,\text{h} \tag{C.26}$$

$$t_{stops} = 121 \cdot\ t_{stop} + 2 \cdot\ t_{GNSStop} = 121 \cdot 10\,\text{s} + 2 \cdot 180\,\text{s} = 1{,}570\,\text{s} \approx 0.44\,\text{h} \tag{C.27}$$

$$t = t_{driving} + t_{stops} = 0.91\,\text{h} + 0.44\,\text{h} = 1.35\,\text{h} \tag{C.28}$$

Calculation of E_{sys}

From the above assumption and operating time t using (C.28), one obtains E_{sys} as follows:

$$E_{sys} = P_{sys} \cdot t = 244\,\text{W} \cdot 1.35\,\text{h} = 329\,\text{Wh} \tag{C.29}$$

Calculation of E_{batt}, $E_{batt,DoD}$, E_{batt,DoD,f_T}, E_{batt,DoD,f_T,f_s}

Using (C.29) one obtains the following values:

$$E_{batt} = \frac{E_{sys}}{\eta_{sys}} = \frac{329\,\text{Wh}}{0.9} = 366\,\text{Wh} \tag{C.30}$$

$$E_{batt,DoD} = \frac{E_{sys}}{\eta_{sys} \cdot DoD} = \frac{329\,\text{Wh}}{0.9 \cdot 0.8} = 457\,\text{Wh} \tag{C.31}$$

$$E_{batt,DoD,f_T} = \frac{E_{sys}}{\eta_{sys} \cdot DoD \cdot f_T} = \frac{329\,\text{Wh}}{0.9 \cdot 0.8 \cdot 0.8} = 571\,\text{Wh} \tag{C.32}$$

$$E_{batt,DoD,f_T,f_s} = \frac{E_{sys}}{\eta_{sys} \cdot DoD \cdot f_T \cdot f_s} = \frac{329\,\text{Wh}}{0.9 \cdot 0.8 \cdot 0.8 \cdot 0.95} = 601\,\text{Wh} \tag{C.33}$$

Calculation of the volumetric energy density ρ in watt-hours per litre (Wh/L) **and the charge capacity Q of the battery in ampere-hours** (Ah)

$$\rho = \frac{E_{batt,DoD,f_T,f_s}}{V} = \frac{601\,\text{Wh}}{0.00349\,\text{m}^3} = 172{,}206\,\frac{\text{Wh}}{\text{m}^3} \approx 172\,\frac{\text{Wh}}{\text{L}} \tag{C.34}$$

$$Q = \frac{E_{batt,DoD,f_T,f_s}}{U} = \frac{601\,\text{Wh}}{4 \cdot 3.7\,\text{V}} \approx 40.6\,\text{Ah} \tag{C.35}$$

Battery Selection

This led to the decision to use two SWAYTRONIC LiPo 4S batteries with a nominal voltage of 14.8 V and a capacity of 22 Ah each (rated at *25C/50C*). The batteries are equipped with *XT90-S* connectors and are connected in parallel, resulting in a total capacity of 44 Ah. Each battery has a mass of 1.765 kg and dimensions of $205 \times 90 \times 46$ mm, allowing them to fit within the electronic tube.

APPENDIX D

Software Platform Appendix

D.1 Package and Module Catalogue

The onboard software is organised as a single `colcon` workspace of ROS 2 (Humble) packages, grouped by functional domain that mirrors the workspace layout under `src/`. The full source code is hosted at `https://github.com/aris-space/project-polaris`. Tables D.1 to D.5 list every active package by group, giving its name, implementation language, and purpose. Sensor functionality for the sensing packages is treated in Section 3.3. Packages marked with a dagger (†) are externally maintained and integrated as git submodules; the remainder are first-party. Language is reported as "Python" for `ament_python` packages, "C++" for `ament_cmake` packages, and "IDL" for interface-definition (`.msg/.srv`) packages.

Table D.1 Communication and configuration packages.

Package	Language	Purpose
`mavlink_bridge`	Python	Bidirectional MAVLink–ROS 2 bridge between onboard computer and flight controller (see Section D.5).
`foxglove_bridge_pkg`	Python	Launch and configuration wrapper bridging ROS 2 topics to the Foxglove topside client.
`foxglove_bridge`†	C++	Foxglove SDK and bridge (submodule).
`custom_msgs`	IDL	Custom message definitions (`Distance`, `Profile`).
`config_pkg`	Python	Shared constants, system-wide launch files, and the rosbag recorder controller.

Table D.2 Sensor and hardware-abstraction packages.

Package	Language	Purpose
`keller_26x_pkg`	Python	Reads the Keller 26X pressure transducer and publishes gauge pressure and water temperature at 30 Hz.
`xsens_mti_ros2_driver`†	C++	Driver for the XSens Sirius AHRS; publishes IMU data at 100 Hz.
`imu_orientation_pkg`	Python	Converts IMU quaternions to roll/pitch/yaw topics for visualisation and analysis.
`dvl_a50_pkg`	Python	Launch and configuration wrapper for the Water-Linked DVL-A50; publishes velocity at 30 Hz.
`dvl_a50`†	Python	WaterLinked DVL-A50 driver (submodule).
`ublox_dgnss`†	C++	Driver for the u-blox Gen-9 GNSS receiver.
`ntrip_client`†	Python	Provides GNSS correction data for high-accuracy positioning.
`gnss_bringup_pkg`	Python	Project-local GNSS bringup launch and configuration wrapper.
`uwgpsg2_ros2_interface`†	Python	ROS 2 interface to the WaterLinked SBL acoustic positioning system.
`uwgpsg2_translator`	Python	Translates SBL output to `NavSatFix` and selects the active global position source (GNSS when its fix is accurate, otherwise SBL).
`ultrasonic_driver`	C++	Driver for the ultrasonic range sensors; the ice-facing sensor publishes at 10 Hz.
`temperature_sensor_pkg`	Python	Reads the internal temperature sensors.
`water_properties_pkg`	Python	On-demand speed-of-sound computation for fresh-water DVL and SBL calibration.
`bms_pkg`	Python	Reads the battery-management system and publishes voltage and current.
`jetson_temperature`	Python	Publishes Jetson junction-, CPU-, and GPU-temperature readings as diagnostics.
`usb_cam_pkg`	Python	Launch wrapper for the USB camera driver.
`auv_msgs`†	IDL	Shared AUV message definitions (submodule).

Table D.3 Navigation and state-estimation packages.

Package	Language	Purpose
`ekf_localization_pkg`	Python	`robot_localization`-based local EKF fusing IMU, DVL, and optional pressure depth.
`pressure_pose_pkg`	Python	Converts fluid pressure to a z-only `PoseWithCovarianceStamped` (ENU) for depth aiding.
`ice_touch_detection_pkg`	Python	Detects contact between the AUV tower and the ice ceiling from ultrasonic, IMU, and pressure data.
`measurement_points_pkg`	Python	Computes the survey grid and publishes the measurement-point sequence.
`adaptive_thruster_estimator`	C++	Online estimation of per-thruster force and torque contributions.

Table D.4 Ice-measurement and controller packages.

Package	Language	Purpose
`ping_sonar`	Python	Drives the Ping sonar and publishes raw distance and profile data.
`ice_estimates`	Python	Aggregates ice-thickness measurements into estimates and 2-D map products.
`mode_control_pkg`	Python	Translates topside joystick inputs into vehicle control commands.

Table D.5 Autonomy packages. Both are adapted from the orca4 project [4].

Package	Language	Purpose
`autonomy_bringup`	Python	Launch files, mission scripts, behaviour trees, and nav2 parameter configurations for the autonomy stack.
`orca_nav2`	C++	Custom nav2 plugins implementing a 3-D straight-line planner and a 3-D Pure Pursuit controller.

D.2 Deployment and Runtime Environment

The onboard software is deployed as a single Docker container on the Jetson AGX Orin. This section documents the container configuration, the host networking it relies on, and the time-synchronisation arrangement that keeps log timestamps coherent across the system. The in-container build and launch sequence executed by the entrypoint is described in Section D.3.

Container Configuration

A prebuilt ARM64 image (`ghcr.io/aris-space/project-polaris-docker:dev-arm64`) is pulled from the GitHub Container Registry and run through Docker Compose with the NVIDIA container runtime. Rather than baking the workspace into the image, the repository is bind-mounted live into the container

and built in place at startup (see Section D.3). This keeps the deployment and development images identical and avoids an image rebuild for every code change. Table D.6 lists the principal Compose directives and their purpose.

Table D.6 Principal Docker Compose directives for the onboard container and the reason each is required.

Directive	Value	Purpose
`privileged`	`true`	Grant direct access to host devices and kernel interfaces required by the sensor drivers.
`network_mode`	`host`	Share the Jetson network stack for DDS discovery, topside streaming, and NTRIP corrections (no port mapping).
`runtime`	`nvidia`	Expose the Jetson GPU and CUDA stack inside the container.
`restart`	`unless-stopped`	Restart the container automatically after a crash or power cycle.
`volume`	`.:/ros2_ws`	Bind-mount the workspace live so code is built in-container rather than baked into the image.
`volume`	`./recordings`	Persist rosbag (`.mcap`) recordings on the host.

Time Synchronisation

Coherent timestamps are essential for the recorded `.mcap` logs and for any data exchanged across machines, as the state estimator assumes that message stamps from different sources share a common time base. Therefore, the Jetson's system clock is synchronised by `chrony`. In the field, where the onboard computer lacks a direct internet connection, the topside station acts as the local time server. The Jetson synchronises to it and obtains network access via IPv4 forwarding over the tether LAN.
Because the Jetson carrier has no built-in battery-backed real-time clock, its clock would otherwise reset on every power loss, producing log files stamped near the Unix epoch and breaking time-sensitive operations. To avoid this, a battery-backed RTC module retains the time across power cycles; on boot the system clock is restored from the RTC via `hwclock` before `chrony` re-converges, so timestamps are valid from the first recorded message even before any network link to the topside is established.

D.3 System Bringup and Launch-File Map

The onboard software is brought up through a single master launch file, `start_system.launch.py` (in `config_pkg`), which aggregates the functional groups into one ROS 2 launch tree. Operationally the system is started from the host through a shell wrapper, through which the Docker container is brought up, the workspace is built, and the master launch file is invoked. This appendix documents that sequence from host script to running node graph, the launch hierarchy, and the launch arguments that parameterise it.

Host-Side Autostart

The container lifecycle is managed by two host scripts. The first, `full_autostart.sh`, performs an unattended start, during which any running stack is stopped, the deployment branch is checked out, and the container is brought up detached. The workspace is then built by the container entrypoint (described below), which emits a sentinel message once the build is complete. Once this sentinel is detected, a detached `tmux` session is started inside the container to run the master launch file. The script is wrapped in a `systemd` service that can be enabled or disabled to allow the full stack to launch automatically on power-on. `manual_start.sh` is identical except that no branch switch is performed and the operator's terminal is attached to the `tmux` session for interactive monitoring. Both ultimately result in `ros2 launch config_pkg start_system.launch.py` being executed inside the container.

Container Entrypoint and Build

When the container starts, the following steps are executed by `entrypoint.sh` before the launch file is invoked:

1. The base ROS 2 (Humble) environment is sourced and a shell hook is installed so that interactive shells source the workspace overlay automatically.
2. Git submodules are initialised and repaired via a helper script, ensuring vendored sensor drivers and message packages are up to date.
3. Dependencies are validated and installed with `rosdep install` when an internet connection is available, and skipped otherwise.
4. The workspace is built with `colcon build --symlink-install` and a sentinel marker is emitted to signal a successful build.
5. The workspace overlay (`install/setup.bash`) is sourced.
6. Control is passed to the command provided by Docker Compose, i.e. the master launch file.

Launch-File Hierarchy

The launch tree expanded from the master launch file is shown in Figure D.1. Four group launch files are included by `start_system.launch.py`, with the Foxglove bridge and the rosbag recorder controller additionally started as standalone nodes. The role of each top-level launch file is summarised in Table D.7. Sensor sub-launches are configured with per-driver `respawn` arguments so that a crashed driver is automatically restarted (see Table D.8). Rosbag recording is performed for every run, but is deliberately invoked separately from the master launch file so that each recording can be assigned a descriptive name at launch time.

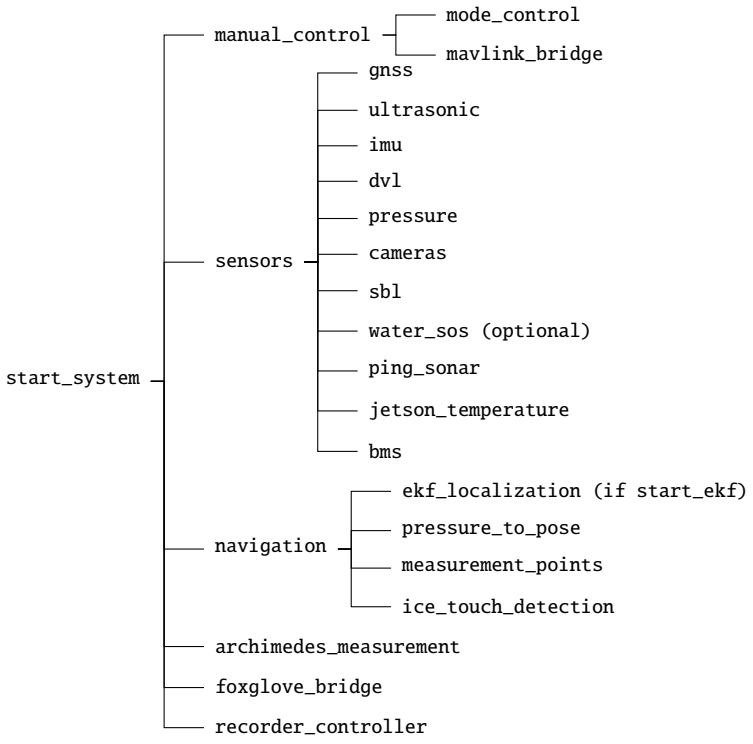


Figure D.1 Launch-file hierarchy expanded from the master launch file. Names are simplified for clarity. The EKF stack is included conditionally on the `start_ekf` argument.

Table D.7 Top-level launch files and the components they bring up. All paths are relative to their respective packages; `record_bag.launch.py` is invoked separately from the master launch file.

Launch file	Group	Components brought up
`start_system.launch.py`	master	Aggregates the four group launches, the Foxglove bridge, and the rosbag recorder controller.
`manual_control.launch.py`	controller	Mode-control nodes (`mode_control_pkg`) and the MAVLink bridge (`mavlink_bridge`).
`sensors.launch.py`	sensors	All sensor drivers (see the launch-file hierarchy above for the full list).
`navigation.launch.py`	navigation	Pressure-to-pose, measurement points, and ice-touch detection. The EKF localisation stack is included conditionally.
`archimedes_measurement.launch.py`	measurement	Ice-thickness estimation using the Archimedes principle (`ice_estimates`).
`record_bag.launch.py`	logging	Per-run `ros2 bag record` of all topics in `mcap` format, with camera image streams excluded by default.

Key Launch Arguments

The launch arguments that most affect bringup behaviour are listed in Table D.8. Notably, the localisation EKF is not started with the rest of the system by default: `start_ekf` is `false` so that the IMU and pressure sensor can settle on land before the filter is started separately once the vehicle is in the water.

Table D.8 Principal launch arguments of the master launch file and their default values. Per-driver `respawn` flags share the `respawn_delay` default.

Argument	Default	Effect
`start_ekf`	`false`	Defer the EKF localisation stack so the IMU and pressure sensor can settle on land before the filter is started in water.
`use_ntrip`	`true`	Enable RTK corrections for the GNSS receiver.
`<driver>_respawn`	`true`	Automatically restart the corresponding sensor driver if it exits or crashes.
`respawn_delay`	2 s	Delay before a crashed process is restarted.

Persistent Device Naming

The sensor drivers brought up by `sensors.launch.py` (Table D.7) open their devices by path, so a consistent mapping between each physical sensor and its `/dev` entry is a prerequisite for a reproducible bringup. This is handled on the companion computer by `udev` rules. USB and serial devices enumerate in a non-deterministic order across reboots and hot-plug events, so a kernel-assigned path may point to a different device on each boot. Each rule matches a device on stable attributes and binds it to a fixed name, so that launch files and driver configurations reference a persistent device rather than an enumeration-dependent path.

Two strategies are used. Devices attached directly over USB are matched by their hardware serial number and assigned a persistent symlink (Table D.9). For sensors connected via a four-port USB-to-UART hub, individual udev rules could not be applied since all ports expose the same chip attributes of the

hub. Each sensor is therefore wired to a fixed port and addressed through that port's stable name (`/dev/uart_port_1` to `/dev/uart_port_4`).

Table D.9 Persistent device names assigned by `99-sensors.rules`. Directly-attached USB devices receive a dedicated symlink; the serial sensors are addressed through the named ports of the USB-to-UART hub.

Sensor / device	Persistent name	Connection
Forward camera	`/dev/cam_front`	USB hub
Electronics-tube camera	`/dev/cam_tube`	USB hub
XSens MTi IMU	`/dev/xsens_imu`	USB hub
Keller 26X pressure sensor	`/dev/keller`	USB
Ultrasonic obstacle sensor	via `/dev/uart_port_1`	UART hub
Ultrasonic obstacle sensor	via `/dev/uart_port_3`	UART hub
Ping360 sonar	via `/dev/uart_port_2`	UART hub
BMS	via `/dev/uart_port_4`	UART hub

Startup Health Check

Once the stack has been brought up, an operator-run script verifies that the vehicle is fully operational before a mission is committed. It proceeds in two phases:
The first phase confirms device presence, checking that each expected `/dev` symlink assigned by the `udev` rules exists and resolves to an actual device, catching an unplugged or mis-enumerated sensor before launch-time errors propagate. The second phase assesses ROS 2 topic health by sampling key topics and comparing their publish rates against per-topic frequency thresholds. A consolidated pass/warn/fail summary is printed at the end, giving the operator a single go/no-go indication for the mission.

D.4 Custom Thruster Mixing Matrix

To ensure that the desired vehicle motions are correctly mapped to individual thruster commands, a thruster mixing matrix is defined for the specific vehicle geometry. All vector quantities are expressed in the NED frame.
ArduSub uses the mixing matrix $\mathbf{B}$ to map the six control axes (roll, pitch, yaw, throttle, forward, lateral) into individual thruster outputs. The vehicle has six thrusters, giving $\mathbf{B} \in \mathbb{R}^{6\times6}$. The control input vector $\mathbf{u}$ is given by:

$$\mathbf{u} = \begin{bmatrix} u_{\text{roll}} \\ u_{\text{pitch}} \\ u_{\text{yaw}} \\ u_{\text{throttle}} \\ u_{\text{forward}} \\ u_{\text{lateral}} \end{bmatrix}. \tag{D.1}$$

The thruster output vector $\mathbf{t}$ is then computed as:

$$\mathbf{t} = \mathbf{B}\mathbf{u}, \quad \mathbf{t} \in \mathbb{R}^6, \quad \mathbf{B} \in \mathbb{R}^{6\times6}. \tag{D.2}$$

With six thrusters and six control axes, the mixing matrix $\mathbf{B} \in \mathbb{R}^{6\times6}$ is square. Three thrusters act in the horizontal (xy) plane and three along the vertical (z) axis. The vertical thrusters provide heave, while their front-to-back spacing generates pitch torque and the left-right offset between the two rear vertical thrusters generates roll torque. Among the horizontal thrusters, a single forward-aligned thruster

provides surge, and the two lateral thrusters provide sway as well as yaw torque through their differential output. The six thruster contribution vectors are therefore linearly independent, giving rank($\mathbf{B}$) = 6 and full six-DOF actuation.
In the ArduSub firmware, $\mathbf{B}$ is defined by specifying, for each thruster i, six dimensionless contribution factors

$$\mathbf{b}_i = \begin{bmatrix} b_{i,\text{roll}} & b_{i,\text{pitch}} & b_{i,\text{yaw}} & b_{i,\text{throttle}} & b_{i,\text{forward}} & b_{i,\text{lateral}} \end{bmatrix}.$$

Stacking these row vectors yields the mixing matrix:

$$\mathbf{B} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \\ \mathbf{b}_4 \\ \mathbf{b}_5 \\ \mathbf{b}_6 \end{bmatrix}. \tag{D.3}$$

First, the throttle, forward, and lateral components of $\mathbf{B}$ are defined from the thruster pointing directions. Let $\mathbf{d}_i \in \mathbb{R}^3$ denote the unit thrust direction of thruster i, expressed in the vehicle body frame, with components $\mathbf{d}_i = [d_{x,i},\ d_{y,i},\ d_{z,i}]^\top$. The *unnormalised* translational mixing factors for thruster i are equal to these components:

$$\tilde{b}_{i,\text{forward}} = d_{x,i}, \qquad \tilde{b}_{i,\text{lateral}} = d_{y,i}, \qquad \tilde{b}_{i,\text{throttle}} = d_{z,i}. \tag{D.4}$$

Because the axis commands u_k and the mixing factors in $\mathbf{B}$ are dimensionless and the resulting motor outputs are subject to saturation, each translational column is normalised independently so that a unit command in any translational axis can utilise the available actuation in that axis up to saturation. The translational columns of $\mathbf{B}$ are normalised by their maximum absolute entry:

$$b_{i,k} = \frac{\tilde{b}_{i,k}}{\max_j |\tilde{b}_{j,k}|}, \qquad k \in \{\text{forward, lateral, throttle}\}. \tag{D.5}$$

In this vehicle, all thrusters are of the same model and aligned with the body axes, hence $d_{x,i}, d_{y,i}, d_{z,i} \in \{-1, 0, 1\}$.

To define the angular contributions of each thruster in $\mathbf{B}$, the moment generated by a thrust force applied at an offset from the vehicle centre of mass is computed. Let $\mathbf{p}_i \in \mathbb{R}^3$ denote the position of thruster i in the body frame and let $\mathbf{p}_{\text{CoM}} \in \mathbb{R}^3$ denote the centre-of-mass position in the same frame. The lever arm of thruster i about the centre of mass is defined as

$$\mathbf{r}_i = \mathbf{p}_i - \mathbf{p}_{\text{CoM}}. \tag{D.6}$$

The *unnormalised* torque-direction contribution $\boldsymbol{\tau}_i$ of each thruster i is given by the cross product of the lever arm and the thrust direction:

$$\tilde{\boldsymbol{\tau}}_i = \mathbf{r}_i \times \mathbf{d}_i = \begin{bmatrix} \tilde{\tau}_{x,i} \\ \tilde{\tau}_{y,i} \\ \tilde{\tau}_{z,i} \end{bmatrix}. \tag{D.7}$$

For the same reason as for the translational columns, the angular columns of $\mathbf{B}$ are normalised by their maximum absolute entry:

$$b_{i,k} = \frac{\tilde{b}_{i,k}}{\max_j |\tilde{b}_{j,k}|}, \qquad k \in \{\text{roll, pitch, yaw}\}, \tag{D.8}$$

where $\tilde{b}_{i,\text{roll}} = \tilde{\tau}_{x,i}$, $\tilde{b}_{i,\text{pitch}} = \tilde{\tau}_{y,i}$, and $\tilde{b}_{i,\text{yaw}} = \tilde{\tau}_{z,i}$, so that the largest magnitude in each angular column is one.

Evaluating these expressions with the thruster positions and orientations taken from the vehicle model yields the final mixing matrix $\mathbf{B}$, with columns ordered as roll, pitch, yaw, throttle, forward, lateral and rows corresponding to thrusters 1–6:

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0.775 & 0 & 0 & -1 \\ 1 & -0.833 & 0 & 0.6 & 0 & 0 \\ -1 & -0.833 & 0 & 0.6 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix}. \tag{D.9}$$

Here row 1 is the forward (surge) thruster, rows 2 and 6 are the two lateral (sway) thrusters, and rows 3 to 5 are the three vertical (heave) thrusters.
The matrix was implemented in the ArduSub firmware as a custom motor frame (`SUB_FRAME_CUSTOM`). Each thruster is registered via `add_motor_raw_6dof`, which accepts the six normalised contribution factors in the order (roll, pitch, yaw, throttle, forward, lateral).

D.5 MAVLink Interface

All communication between the onboard computer and the flight controller uses the MAVLink protocol. Telemetry is extracted from the flight controller while commands and odometry are pushed back to it, with both directions bridged into the ROS 2 graph by the `mavlink_bridge` package. This section describes how a single physical serial link to the flight controller is shared by several MAVLink clients without contention, and documents the implementation of the custom MAVLink bridge ROS 2 node.

The Serial-Port Contention Problem

The flight controller is connected to the onboard computer over a single UART. The `mavlink_bridge` comprises multiple independent ROS 2 nodes that each need access to this link. Because the Linux kernel limits serial port access to a single process at a time, multiple nodes attempting to open the same device concurrently will fail, with all but the first receiving an error.

mavlink-router as the Single Port Owner

This contention is resolved by `mavlink-router`, a lightweight MAVLink multiplexer that runs as a `systemd` service and is the *sole* owner of the serial port. It opens the serial port once and exposes the link to local clients through TCP and UDP network endpoints instead of the raw device. `mavlink-router` forwards every MAVLink frame received on the serial link out to all connected endpoints, and conversely routes any frame received on an endpoint back down to the serial link. Because exactly one process holds the physical device, the operating-system-level port conflict cannot occur.
This indirection decouples message extraction from message injection. Telemetry is read out of one endpoint while commands are written to another at the same time; `mavlink-router` serialises the two streams onto the single UART internally, so the application code never has to arbitrate access to the device. The `mavlink_bridge` ROS 2 nodes connect to these endpoints: a publisher node reads telemetry from a UDP endpoint and republishes it as ROS 2 topics, while a receiver node subscribes to ROS 2 command topics and forwards them to the flight controller over TCP. Adding or removing a client is a matter of connecting to an endpoint and does not disturb the others, which also allows external tools (for example a ground station for parameter inspection) to attach to a spare UDP endpoint during testing without interrupting the autonomy stack.

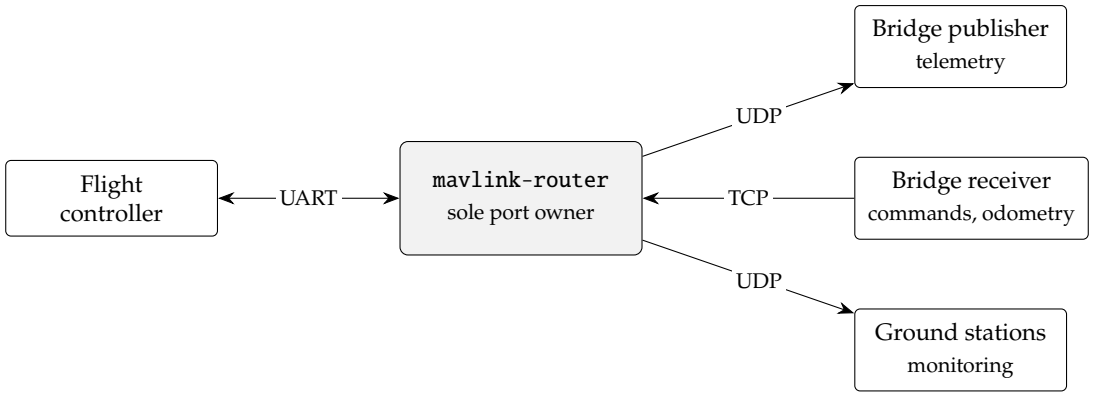


Figure D.2 MAVLink routing topology. The `mavlink-router` service is the sole owner of the serial (UART) link to the flight controller and multiplexes it onto local TCP and UDP endpoints. The bridge publisher reads telemetry, the bridge receiver injects commands and odometry, and ground stations attach for monitoring, each through an independent endpoint.

Custom MAVLink Bridge

The `mavlink_bridge` package connects to the `mavlink-router` endpoints through `pymavlink`, which maintains each connection's own message parser so the clients remain isolated despite sharing the underlying serial link. Two nodes form the bridge:
A *publisher* node reads telemetry from the flight controller via the UDP endpoint and republishes it as ROS 2 topics. On startup it requests the required MAVLink streams (attitude, scaled pressure, depth, battery and PID tuning) at their target rates, then translates each incoming message into the corresponding ROS 2 message.
A *receiver* node subscribes to ROS 2 command topics and forwards them to the flight controller via the TCP endpoint as MAVLink messages: `SET_MODE` for flight-mode changes, `COMMAND_LONG` for arming, disarming and reboot, and `MANUAL_CONTROL` for operator axis input. The same node forwards the onboard computer's EKF output (`/odometry/filtered/local`) as a MAVLink `ODOMETRY` message so that ArduSub can use it as an external navigation reference, transforming the pose from the ROS 2 ENU convention to the ArduSub NED convention before transmission. Additionally, the receiver sends a GCS heartbeat at 1 Hz, which ArduSub monitors through its GCS failsafe and disarms the vehicle autonomously if they stop arriving, independently of the higher-level software.

D.6 ArduSub Flight Modes

During operations, four ArduSub flight modes are used, listed below. In every mode the inner stabilisation loops and the thruster-allocation mixer execute on the flight controller; the modes differ in which degrees of freedom (DOF) are commanded by the onboard computer or the operator, and which DOF the flight controller regulates autonomously. The mode names used here match the ArduSub flight-mode names exactly.
Operator commands are forwarded to the flight controller as a `MANUAL_CONTROL` message whose channels map to surge (x), sway (y), heave (z), yaw (r), roll (s), and pitch (t).

Table D.10 ArduSub flight modes used by POLARIS.

Mode	Commanded by onboard computer / operator	Stabilised or held autonomously by flight controller
`MANUAL`	Surge, sway, heave, roll, pitch and yaw (full 6-DOF open-loop pilot input)	None; pilot input is mapped directly through the thruster-allocation mixer without attitude or depth regulation
`STABILISE`	Surge, sway, heave and yaw rate setpoint	Roll and pitch (the flight controller auto-levels attitude about the horizontal axes)
`ALT_HOLD` (depth hold)	Surge, sway, yaw rate setpoint, and depth setpoint via the heave channel	Roll and pitch, and the heave axis: the flight controller closes the depth loop and holds the commanded depth setpoint
`GUIDED`	A body-frame velocity command from the autonomy controller — surge velocity, heave velocity and yaw rate; sway and attitude are not commanded, and there is no direct pilot input	Roll and pitch (attitude auto-levelled); the cascaded PID loops regulate the thrusters to track the commanded surge velocity, heave velocity and yaw rate

The `MANUAL` mode provides an unregulated fallback for bench testing and recovery. In `ALT_HOLD`, the commanded depth setpoint is nudged via the heave channel. In `GUIDED` mode, the path-following controller (Chapter 6) issues body-frame velocity commands at 20 Hz. The onboard computer state estimate is forwarded to the flight controller as an external navigation reference, ensuring both the guidance controller and the velocity-tracking loops operate against a consistent estimate.

PID Gain Tuning

The PID gains governing heave control in `ALT_HOLD` and roll/pitch stabilisation across all modes were tuned iteratively during pool and lake testing. The `mavlink_bridge` receiver (Section D.5) fetches the relevant ArduSub parameters from the flight controller on startup and exposes them as ROS 2 parameters, which Foxglove allows to be edited live from the topside interface. Any change triggers an immediate write-back to the flight controller via MAVLink and takes effect without a reboot between runs. The vehicle response, comprising depth from the pressure sensor and attitude from the IMU, was monitored as live plots in Foxglove, allowing gains to be adjusted in response to observed oscillations and steady-state error.

D.7 Topside Visualisation with Foxglove

The topside operator station runs Foxglove Studio as the primary interface for monitoring and interacting with the vehicle. Foxglove connects to the `foxglove_bridge` node running on the onboard computer (Section D.3) over the tether, exposing the live ROS 2 graph to the operator. The same tool is used both for real-time supervision and for replaying recorded `.mcap` logs offline, so the visualisation workflow is identical during a mission and during later analysis.

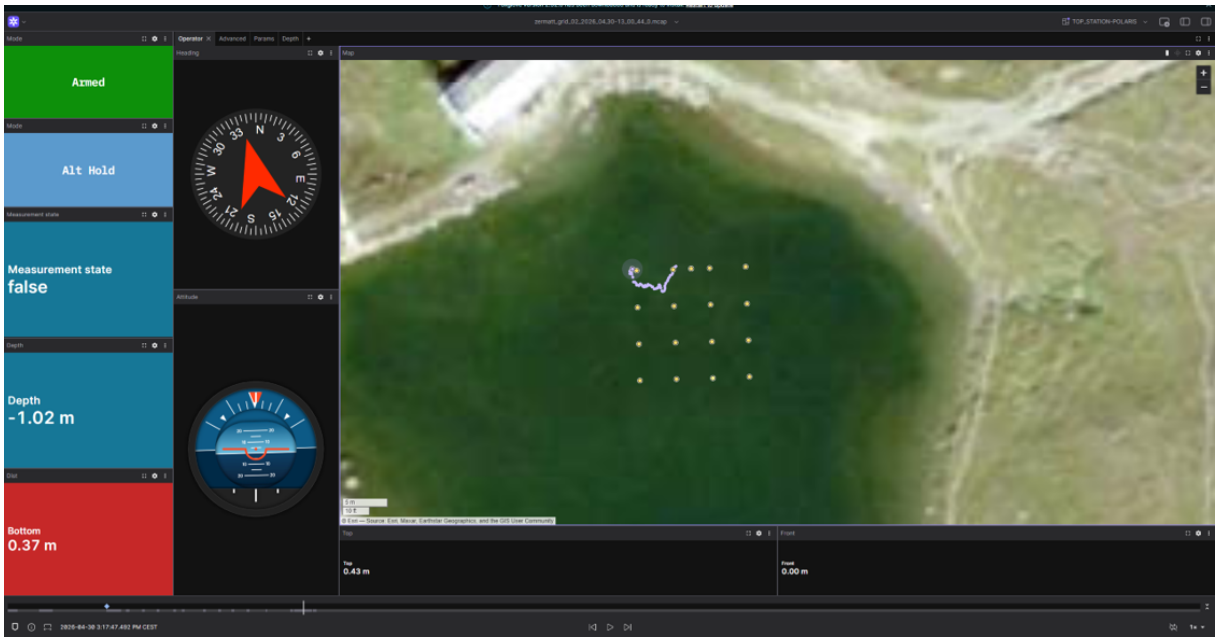


Figure D.3 Topside Foxglove layout during operation, presenting the principal vehicle telemetry as instantaneous indicators. Shown are the arming state and flight mode, the IMU heading and attitude, the measured depth and bottom distance, and the vehicle position together with the measurement grid on a map.

Choice of Foxglove as main User Interface

Foxglove was selected for its native ROS 2 support, which brings the operations most needed during a mission together in a single configurable interface. The `foxglove_bridge` node serves a WebSocket endpoint to which the topside client connects over the tether LAN. Beyond streaming topics, the bridge is configured to expose the capabilities listed in Table D.11, allowing the operator to act on the running system without opening a terminal on the onboard computer. The plotting panels additionally render any numeric topic as a time series, making trends in quantities such as battery voltage, roll, pitch, and depth immediately visible.

Table D.11 Capabilities enabled on the `foxglove_bridge` node and the interaction each provides to the topside operator.

Capability	Interaction enabled
`parameters`	Read and set ROS 2 node parameters live from the client.
`services`	Invoke ROS 2 services from the client.
`clientPublish`	Publish messages from the client, e.g. joystick teleoperation.

Custom Panels

In addition to the built-in panels, a set of project-specific Foxglove extensions is maintained in `top_station/Foxglove-extensions`. These include custom flight indicators, a joystick panel for teleoperation (publishing through the `clientPublish` capability), and a recording-control panel, tailoring the interface to the POLARIS workflow.

APPENDIX E

Localisation Appendix

E.1 Online Estimator Choice: EKF and Alternatives

This section gives the fuller justification of the estimator choice summarised in Section 5.1. The online estimator must fuse a heterogeneous sensor set (Section 3.3) into a real-time state estimate at the rate the controller and autonomy stack consume, run on the onboard computer alongside the sensor drivers and the control stack, and stay predictable and field-tunable across missions in which absolute aiding (GNSS, acoustic SBL) is intermittent [35, 36]. Four estimator families were considered: the extended Kalman filter (EKF) [37, 38], the unscented Kalman filter (UKF) [39, 40], the particle filter [41, 42, 43], and fixed-lag smoothing or factor-graph optimisation such as iSAM2/GTSAM [44, 45, 46]. Pure dead reckoning was excluded at the outset because it cannot incorporate absolute fixes.
Table E.1 compares the families on the criteria that matter for this application.

Table E.1 Estimator families compared on the criteria relevant to the AUV's online localisation. The local filter runs at 30 Hz, so a measurement update spans a step of about 33 ms.

Aspect	EKF (chosen)	UKF	Particle filter	Factor-graph smoothing
Posterior model	Unimodal Gaussian	Unimodal Gaussian	Arbitrary, multimodal	Gaussian (MAP)
Nonlinearity over a 33 ms step	Small; analytic Jacobian adequate	Sigma points; no gain here	Handled, but unneeded	Handled
Onboard compute	Low, bounded	Higher, no benefit	Poor for a 15-state vector	Highest; bounded only fixed-lag
Refines past states	No, current state only	No	No	Yes
ROS 2 maturity	High	Available	Low	Medium

The EKF wins on three discriminators. First, the posterior is unimodal: the primary inputs (inertial, Doppler velocity, pressure) are near-Gaussian after the per-sample covariance handling done in the driver wrappers, and the AUV does not run terrain- or scan-matching during the dive, so there is no multimodal hypothesis for a particle filter to track. Second, the dynamics are nearly linear over the 33 ms step, because the slow underwater motion and bounded angular rates keep the change in orientation within a step small. The EKF's Jacobian linearisation is therefore accurate, and the UKF's sigma points buy no measurable accuracy for the extra computation. Third, the controller and autonomy stack consume the most recent state at the next control tick. A smoothing estimator optimises a window of past states, which is the wrong tool for closed-loop control: the most recent state is exactly the one a smoother has not yet refined. The EKF is also the cheapest of the four on the onboard computer and has the most mature ROS 2 tooling and the best-understood failure modes (innovation gating, normalised-innovation tests, residual checks) [47], which were used throughout tuning.

E.2 Filter Implementation

E.2.1 Choice of the `robot_localization` Package

Given the EKF choice, the implementation question was whether to use the community-standard `robot_localization` package [11] or write a custom filter. The package was chosen, for four main reasons.

First, risk reduction: a correct EKF requires the state-transition and per-sensor measurement Jacobians, covariance propagation, quaternion and frame handling, and numerical conditioning all to be right before the genuinely hard work (noise tuning, sensor characterisation, timestamp handling) can begin. `robot_localization` is a validated implementation maintained for over a decade and widely used in field robotics, so the project effort went into the project-specific work instead of reimplementing matrix bookkeeping. Second, the package operates natively in the coordinate frames the rest of the stack expects, following the standard ROS 2 convention (REP-105 [48]): it maintains the `map`, `odom`, and `base_link` frames and broadcasts the `map` → `odom` and `odom` → `base_link` transforms directly (Section 5.1). The autonomy stack navigates in those same frames and consumes the `map` → `odom` transform, so the estimator integrates with it without any intermediate frame conversion. Third, the sensor-fusion surface is modular: each input is declared by its topic, a per-field fusion mask, a queue size, and a small set of flags, so integrating an additional sensor is a matter of declaring it rather than modifying the filter implementation. Fourth, all inputs are standard ROS 2 message types and the filter runs identically on live data and on recorded data with simulated time, which is essential for offline iteration.

The accepted trade-offs are minor for this application. The package fixes the 15-element state vector $[x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}]$ with an Euler-angle orientation representation. This is acceptable for the AUV's operating envelope. Pitch stays well inside $\pm 90°$, so there is no gimbal-lock risk. The angular uncertainty between updates also stays small, thanks to the calibrated initial yaw (Section 5.2), the high-rate IMU updates, and the bounded survey angular rates, so the standard EKF's linearisation at the current state estimate remains accurate. It is a standard EKF rather than an error-state or invariant variant. Should a future mission push the envelope (near-vertical pitch, long absolute-aiding outages, or much larger initial heading uncertainty), an error-state (ESKF) or invariant (IEKF) filter would be the natural upgrade path. The dependence on an external package is mitigated by pinning its version, by reproducibility from recorded data, and by keeping the project-specific operational logic in our own wrapper nodes around the unmodified filter.

E.2.2 EKF Formulation

The navigation estimate is maintained by the package's generalised EKF [11], which keeps a single state estimate $\hat{x}$ and its covariance P and alternates a prediction step with a correction step. The prediction propagates the state through the nonlinear rigid-body kinematic process model f and propagates the covariance through its Jacobian $F = \partial f / \partial x$. When a measurement z_k arrives, the correction step forms the Kalman gain K_k and pulls the state towards the measurement:

$$\hat{x}_k^- = f(\hat{x}_{k-1}), \tag{E.1}$$

$$P_k^- = F\, P_{k-1}\, F^\top + Q, \tag{E.2}$$

$$K_k = P_k^- H^\top \left(H P_k^- H^\top + R\right)^{-1}, \tag{E.3}$$

$$\hat{x}_k = \hat{x}_k^- + K_k\left(z_k - h(\hat{x}_k^-)\right), \tag{E.4}$$

$$P_k = (I - K_k H)\, P_k^-. \tag{E.5}$$

Here Q is the process-noise covariance: the covariance of a zero-mean white-noise term that captures how far the true motion is expected to deviate from the deterministic prediction over one step, absorbing unmodelled effects such as drag variation and model mismatch as random noise. R is the per-sensor measurement-noise covariance. The measurement model h that maps the state to a sensor's observation is linear: each sensor observes a subset of the state directly, so its Jacobian H is a constant selection matrix, and the only nonlinearity in the filter is the process model f. This structure provides the property used throughout the stack, partial measurement updates. Each sensor corrects only the state variables it observes, so the pressure sensor constrains depth, the DVL constrains the body-frame velocities, and an absolute fix constrains the horizontal position, without imposing constraints on states it does not measure. The covariances Q and R are tuned from logged data using innovation-based diagnostics, with the normalised innovation squared as a secondary consistency check.

E.2.3 Independence of the Two Filter Instances

The local and global instances can be wired in two ways. In a *stacked* arrangement the global instance takes the local instance's filtered pose as an input, on top of the absolute fix. In the *parallel* arrangement used here (Section 5.1), both instances dead-reckon independently from the same raw sensors, and only the global instance additionally fuses the absolute fix. The local estimate is never fed into the global filter. The parallel arrangement was chosen because the stacked one fails in two ways.
First, it makes the global filter overconfident. The local pose is an already-filtered estimate whose information comes entirely from the IMU and DVL, and whose errors are strongly correlated across consecutive samples because they share the local filter's integrated bias and drift. The global filter treats each incoming local pose as an independent measurement carrying the local filter's reported (posterior) covariance, so fusing this stream as if its samples were independent counts the same inertial and Doppler information repeatedly. The global covariance then shrinks below the true uncertainty, and an overconfident covariance produces a small Kalman gain for the genuinely new absolute fix, so the filter under-weights the correction and rides its own drift.
Second, it creates a feedback loop in the `map` → `odom` transform. The local pose is expressed in the `odom` frame, so the global filter must transform it into `map` using the `map` → `odom` transform that the global filter itself publishes from its own state. Once the global state drifts away from the absolute fix, that transform drifts with it, and subsequent local poses are rotated through the corrupted transform, which compounds the error on each cycle. In replay this produced a short stable period followed by divergence to the kilometre scale. The parallel arrangement removes both failure modes, because the global filter neither re-fuses the local estimate nor depends on its own transform to interpret its inputs. It is the configuration used for the results in Section 5.3.

E.2.4 Timestamp Handling and Sensor Latency

Each measurement carries the timestamp at which the sensor sampled it, and the filter fuses it relative to that timestamp: for a measurement newer than the filter's current time it predicts the state forward to the measurement's time and then applies the correction. This is the right reference for fusion, because a Kalman update must be applied at the time the measurement was valid rather than when it happened to arrive over the bus. The sensor latencies differ markedly: the IMU is effectively immediate, whereas the DVL arrives about 120 ms behind on the wire (a transport delay in the driver, not a clock offset), with a worst-case tail of one to a few seconds during dynamic motion.
This latency is not fully corrected in the current configuration. Because the IMU arrives first, it advances the filter's clock past the DVL's timestamp before the DVL is delivered. The late DVL sample is then older than the filter's most recent measurement, and the filter applies its correction to the current (slightly later) state without stepping back to the sample's true time. Delayed-measurement smoothing, which would revert the filter to the sample's time and re-fuse the intervening measurements in order, is not enabled. The same behaviour occurs in offline replay, because the recording preserves the arrival lag and the player reproduces it. The resulting timing skew is bounded by the latency multiplied by the rate of change of the velocity, which is small for the AUV's slow, smooth motion. A dedicated lag analysis could not resolve any skew beyond about 150 ms. Closing this gap exactly, by enabling delayed-measurement smoothing or by reducing the DVL transport latency at the driver, is left as a future improvement. A separate clock-integrity guard rejects any pose whose timestamp jumps forward by more than 60 s, an anomaly seen in replay when the simulated clock was briefly unavailable, which would otherwise produce a multi-day prediction step.

E.3 Dead-Reckoning Range Before a Global Fix

This section estimates how far the vehicle can dead-reckon on the local estimator before its horizontal error is expected to exceed the ±2 m budget of MR-04 and SR-LOC-02, justifying the range quoted in Chapter 5.
The dead-reckoning error is modelled as linear in the travelled path length L,

$$\varepsilon(L) = b + m\,L + \eta, \qquad \eta \sim \mathcal{N}(0, \sigma^2), \tag{E.6}$$

with drift slope m, initial offset b, and residual scatter η about the fit. The parameters are taken from the open-equivalent drift fit on the `grid_02` dataset (derived in Section E.5): a slope $m = 0.42$ m per 100 m (= 0.0042) and a residual scatter $\sigma = 0.235$ m.

To bound the error rather than its mean, a one-sided upper prediction limit adds $z\sigma$ to the trend,

$$\varepsilon_{\mathrm{ub}}(L) = b + m\,L + z\,\sigma, \tag{E.7}$$

with $z = 1.645$ for a one-sided 95 % limit. Setting $\varepsilon_{\mathrm{ub}}(L) = 2\,\mathrm{m}$ and solving for the range gives

$$L_{\max} = \frac{2\,\mathrm{m} - b - z\,\sigma}{m}. \tag{E.8}$$

For a fresh anchor ($b = 0$) and the fitted slope,

$$L_{\max} = \frac{2\,\mathrm{m} - 1.645 \times 0.235\,\mathrm{m}}{0.0042} \approx 384\,\mathrm{m}. \tag{E.9}$$

Table E.2 lists $L_{\max}$ for progressively more conservative assumptions.

Table E.2 Dead-reckoning range $L_{\max}$ at which the upper error bound reaches 2 m, for different assumptions on the drift slope, initial offset b, and confidence factor z.

Scenario	Slope [m/100 m]	b [m]	z	$L_{\max}$ [m]
Best estimate (one-sided 95 %)	0.42	0	1.645	384
Upper-bound slope	0.46	0	1.96	335
Upper-bound slope with offset	0.46	0.33	1.96	263

Operational Budget. A dead-reckoning limit of 300 m is adopted for mission planning. This sits below the best estimate for three reasons. The fit rests on a single dataset of approximately 120 m, so longer ranges are extrapolation. Currents and calibration vary between missions. Finally, the model assumes uninterrupted DVL bottom lock, whose loss raises the drift rate by orders of magnitude and collapses the budget.

Interpretation. $L_{\max}$ is the range at which the one-sided 95 % upper prediction bound reaches 2 m. Under the model the error has roughly a 5 % chance of exceeding 2 m at that range, not a 95 % chance.

E.4 Drift-Rate Estimation and Its Uncertainty

The drift slope m, initial offset b, and residual scatter σ in the error model (E.6) are obtained by an ordinary-least-squares regression of the horizontal position error on the cumulative travelled path. The fitted slope m is the drift rate (error accumulated per unit path), the intercept b is the error extrapolated to zero path, and σ is the residual standard deviation, the root-mean-square scatter of the data about the fitted line, which is the scatter term used in the budget of Section E.3.
The uncertainty of the slope is its standard error,

$$\mathrm{SE}(m) = \frac{\sigma}{\sqrt{\sum_i (x_i - \bar{x})^2}} = \frac{\sigma}{\sqrt{N\ \mathrm{Var}(x)}}, \tag{E.10}$$

where the x_i are the cumulative-path values, $\bar{x}$ their mean, and N the number of error/path pairs. The standard error therefore shrinks as more data are gathered over a wider path span. The 95 % confidence interval of the drift rate is then $m \pm 1.96\,\mathrm{SE}(m)$.

E.5 Open-Loop Drift Rate from the Truncation Sweep

The drift slope m used above is estimated from the `grid_02` dataset by regressing the horizontal position error on the cumulative travelled path. The position error is measured against the RTK GNSS reference, gated to 0.5 m horizontal accuracy so that only high-quality fixes define the ground truth. Because `grid_02` is a closed-loop trajectory (the vehicle returns to within 3.15 m of its start), the geometry partly

cancels the accumulated error over the final stretch of the run. The full-run fit therefore gives only 0.28 m per 100 m and under-reports the sensor-driven drift (Figure E.1).
To recover the open-trajectory (geometry-independent) drift, the regression is repeated on truncated versions of the run, discarding the last fraction of cumulative path over which the closure occurs. Table E.3 reports the fitted slope against the retained fraction of the path.

Table E.3 Truncation sweep on `grid_02`: drift slope from the error-vs-path regression as the last fraction of cumulative path is removed. The 90 % row is the open-loop estimate.

Retained fraction	Path span [m]	N	Slope [m/100 m]	SE [m/100 m]
1.00	134.4	1153	0.28	0.02
0.95	125.9	1104	0.40	0.02
0.90	**120.8**	**1089**	**0.42**	**0.02**
0.80	105.9	981	0.31	0.02

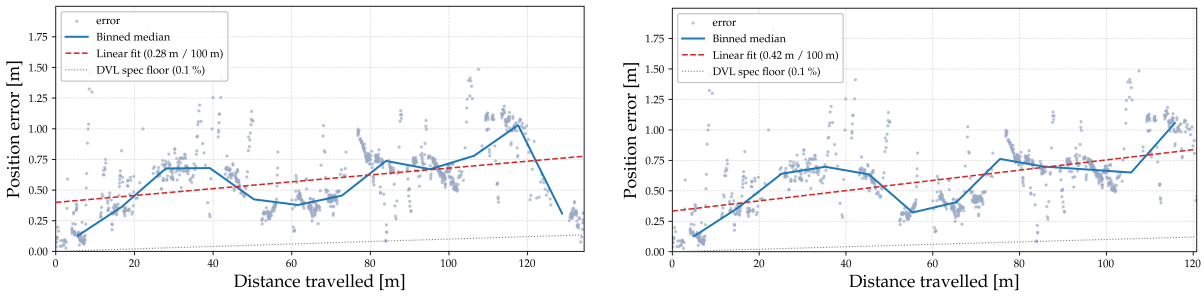


Figure E.1 Dead-reckoning error against distance on `grid_02`: full loop (left) and with the closing leg removed (right). On the full loop the error collapses over the final 10 m to 15 m as the vehicle returns near its start, pulling the linear fit down to 0.28 m per 100 m; removing that leg gives the open-trajectory rate of 0.42 m per 100 m.

Retaining the first 90 % of the path removes the closure tail while keeping almost all of the data (1089 of 1153 error/path pairs, spanning 120.8 m of 134.4 m) at essentially the full-run fit precision, yielding the open-loop estimate $m = 0.42$ m per 100 m.
Applying the slope standard error (E.10) with $\sigma = 0.235$ m and $N = 1089$ over the 120.8 m span gives $\mathrm{SE}(m) \approx 0.02$ m per 100 m, so the 95 % confidence interval of the open-loop drift is [0.38, 0.46] m per 100 m. Below 85 % retention the fitted slope reflects the geometry of whichever sub-segment remains rather than the global drift, and $\mathrm{SE}(m)$ grows as the path span shrinks, so 90 % is used.
The cancellation is geometric, not a property of the filter, and it acts only on *constant* biases. For a time-invariant systematic error the position error is a fixed linear function of the vehicle's net displacement $\Delta\mathbf{p}$ from its start, independent of the path length travelled in between: a constant heading bias α rotates the integrated velocity and contributes $(\mathbf{R}(\alpha) - \mathbf{I})\,\Delta\mathbf{p}$, while a constant DVL scale error s scales it and contributes $s\,\Delta\mathbf{p}$. On a closed loop $\Delta\mathbf{p} \to \mathbf{0}$, so both vanish at closure however far the vehicle actually drove. The two dominant dead-reckoning error sources on `grid_02`, the DVL scale factor and the residual heading bias, are exactly of this constant form. This is why the full-loop fit collapses to 0.28 m per 100 m and understates the sensor drift. Only the time-varying and random contributions (gyro bias instability, velocity random walk, current variation) survive a closed loop. Truncating the closing leg restores the constant-bias term, so the 90 % fit recovers the rate that an equally long open, out-and-back trajectory would actually accumulate.
Being the dominant term, the DVL scale factor is also the simplest to remove. A straight single-bearing surge of at least 100 m under RTK-fixed GNSS yields both the DVL forward distance (the integral of its reported velocity) and the true displacement (between the first and last fix). Their ratio is the scale-factor error, which is then compensated by a constant gain in the DVL driver. This one calibration would remove most of the open-loop drift.

E.6 Magnetometer Heading Characterisation

This section gives the evidence for rejecting the magnetometer, summarised in Section 5.2. On the 2026-04-23 surface runs RTK GNSS provides an independent heading: on straight forward segments (chord ≥ 4 m, horizontal standard deviation ≤ 1 m) the bearing between the segment endpoints is the true heading of travel, which is compared against the tilt-compensated magnetic compass heading (Table E.4).

Table E.4 Magnetometer compass heading versus the GNSS two-fix bearing on straight forward segments of the 2026-04-23 surface runs. The offset is the magnetic heading minus the true heading, wrapped to $(-180, 180]$. Headings are compass degrees (clockwise from north).

Run	True heading [°]	Mag heading [°]	Offset [°]
gnss_01	228	313	+85
	329	192	−137
	70	242	+172
gnss_03	219	331	+112
	326	191	−136
	57	243	−174
	141	297	+156
gnss_04	240	351	+111
	332	193	−139
	63	241	+178
	141	299	+158
surge_03	218	341	+123
surge_05	318	171	−147
surge_07	241	359	+119

The magnetic heading is wrong by 85° to 178°, and the error is not a constant offset: within a single run it swings by more than 100° as the heading changes (run `gnss_01`: +85, −137, +172 degrees), yet it is repeatable across runs at a given heading (true 327° reads near 191° in every run). A repeatable, heading-dependent error is the signature of combined hard- and soft-iron distortion, severe here because the steep magnetic inclination [49] leaves a weak horizontal field that the vehicle's ferromagnetic structure dominates. The short chords (4 m to 13 m) add only about 10° of bearing noise, far below the discrepancy. The heading-dependence together with the cross-run repeatability identifies this as hard- and soft-iron distortion, the stationary form of which a full-rotation calibration ($H_{\text{meas}} = A\,H_{\text{Earth}} + b$) is in principle designed to remove [50]. Repeated calibration attempts on the assembled vehicle nonetheless failed to produce a usable heading. Two factors limit what a calibration can achieve here: the steep magnetic inclination leaves an intrinsically weak horizontal (heading-bearing) field, a signal-to-noise floor that removing the distortion cannot raise. A clean full-rotation calibration is also hard to obtain on the vehicle, where the field at the sensor is dominated by the nearby motors, ESCs, and steel. Heading is therefore taken from the gyro and tied to GNSS bearing (Section 5.2).

E.7 Sensor-Noise Characterisation

The measurement noise of the IMU and DVL is characterised from a 64-minute stationary recording (Lake Zurich, 2026-04-19; DVL at 8.9 Hz, IMU at 86.5 Hz). To isolate sensor noise from platform motion, the DVL was removed from the vehicle and fixed rigidly in the water, where it held continuous bottom lock, while the AUV carrying the IMU rested on land. Both sensors were therefore static and free of wave-driven motion, so the recorded signal is sensor noise and bias. The pressure noise is characterised from a separate 165-second recording of the sensor floating in water (2026-04-24), in which slow water motion is necessarily present and is separated out below. For the IMU and DVL the overlapping Allan deviation [12] of each channel is computed with `allantools`. The white-noise coefficient N and the bias-instability floor B are read from the curves, and the per-axis measurement variance is taken as $R = N^2 f$ at the sample

rate f. A channel is white-noise-dominated where the deviation falls with averaging time (slope $-\frac{1}{2}$) and bias-dominated where it flattens onto a floor. The white-noise region sets the measurement variance R. The deployed measurement noise loaded into both filters is summarised in Table E.5, as the variance R and the equivalent standard deviation $\sigma = \sqrt{R}$. The per-channel values below follow from the Allan analysis of this section.

Table E.5 Approximate deployed measurement noise per fused channel: variance R and the equivalent standard deviation $\sigma = \sqrt{R}$. Velocity, angular-rate, and depth noises are measured here. The orientation values are the IMU driver's stated accuracy, the orientation being an internal estimate rather than a raw signal. The accelerometer is not fused, and a DVL sample without bottom lock is rejected by inflating its variance to 1×10^6.

Measurement	R	σ
DVL velocity v_x	$4.4 \times 10^{-6}\,\mathrm{m^2\,s^{-2}}$	$2.1\,\mathrm{mm\,s^{-1}}$
DVL velocity v_y	$7.8 \times 10^{-6}\,\mathrm{m^2\,s^{-2}}$	$2.8\,\mathrm{mm\,s^{-1}}$
DVL velocity v_z	$1 \times 10^{-6}\,\mathrm{m^2\,s^{-2}}$ †	$1.0\,\mathrm{mm\,s^{-1}}$
IMU angular rate $\omega_{x,y,z}$	$2.1 \times 10^{-6}\,\mathrm{rad^2\,s^{-2}}$	$1.4\,\mathrm{mrad\,s^{-1}}$
IMU orientation roll, pitch	—	3.5 mrad to 4.4 mrad
IMU orientation yaw	—	17 mrad ($\approx 1°$)
Pressure z	$4 \times 10^{-6}\,\mathrm{m^2}$	2 mm

† Held at the sensor specification floor; the measured value is lower.

Table E.6 DVL velocity-noise parameters from the overlapping Allan deviation of `stationary_11`: white-noise coefficient N, bias-instability floor B, and the deployed measurement variance $R = N^2 f$. The vertical channel is the quietest in both N and B; its Allan-derived variance falls below the sensor specification floor and is held there.

Channel	N [$\mathrm{m/\sqrt{s}}$]	B [$\mathrm{m\,s^{-1}}$]	R [$\mathrm{m^2\,s^{-2}}$]
DVL v_x	7.03×10^{-4}	4.71×10^{-5}	4.42×10^{-6}
DVL v_y	1.06×10^{-3}	4.31×10^{-5}	7.84×10^{-6}
DVL v_z	1.33×10^{-4}	1.44×10^{-5}	1.0×10^{-6} †

† The Allan-derived variance for v_z ($1.57 \times 10^{-7}\ \mathrm{m^2\,s^{-2}}$) falls below the $1 \times 10^{-6}\ \mathrm{m^2\,s^{-2}}$ sensor specification floor, which is retained as R_{v_z}.

The DVL velocity noise is white at the few-millimetre-per-second level on all three axes over the sub-second-to-second averaging times the filter operates at (Figure E.2, Table E.6), so the white-noise R model is appropriate. The vertical channel is the quietest, with the smallest N and B. Its measured variance is so low that it is held at the sensor specification floor. The strong weight this places on v_z is what makes its innovations sensitive to any unmodelled vertical motion during tuning (Section 5.5).

The gyroscopes are white at short averaging times before settling onto a bias-instability floor at averaging times of order 100 s, with measurement variances near $2.1 \times 10^{-6}\,\mathrm{rad^2\,s^{-2}}$. The accelerometers (Figure E.3, right) show the same white-noise-then-floor shape, but at a much higher level. They are not fused (Section 5.1): the DVL measures linear velocity directly, and integrating the noisier acceleration would only compound its bias into the position estimate.

The navigation pressure sensor (Figure E.4) has a depth standard deviation of 0.880 mm after removing a slow $-0.46\,\mathrm{mm\,min^{-1}}$ settling trend. First differencing isolates the high-frequency sensor content at 0.167 mm. The remaining 96 % of the variance is low-frequency water motion (platform tilt and sub-hertz waves). The sensor is therefore far more precise than its 2 cm absolute-accuracy specification, so the depth channel is not sensor-limited. The deployed measurement variance, $4 \times 10^{-6}\,\mathrm{m^2}$ ($\sigma = 2$ mm), is a deliberately conservative value set well above this floor. Its exact size is not critical, since the sensor far exceeds the depth tolerance and the pressure update constrains only depth, not the horizontal solution that the ±2 m requirement targets.

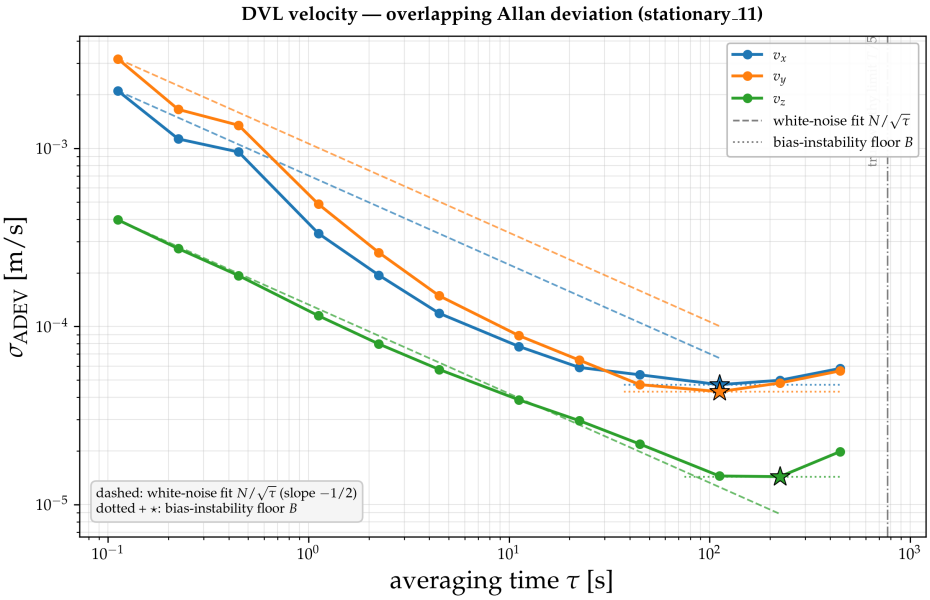


Figure E.2 Overlapping Allan deviation of the three DVL velocity channels (`stationary_11`, 64.2 min). Each curve falls with averaging time in the white-noise region (slope $-\frac{1}{2}$) before reaching its bias-instability floor at an averaging time of about 100 s to 200 s; the vertical channel (v_z, lowest curve) is the quietest at every averaging time.

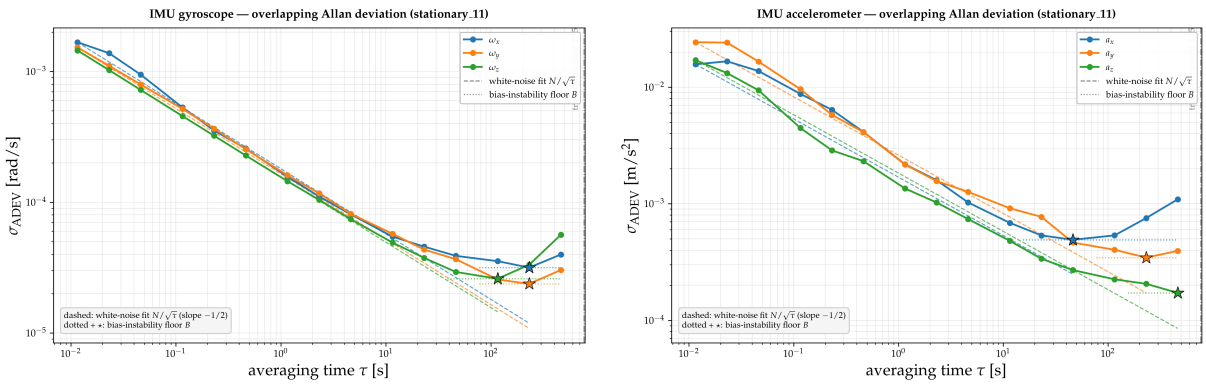


Figure E.3 Overlapping Allan deviation of the IMU gyroscope (left) and accelerometer (right) channels. Both show a white-noise region at short averaging times (slope $-\frac{1}{2}$) before flattening onto a bias-instability floor; the accelerometer noise is at a much higher level.

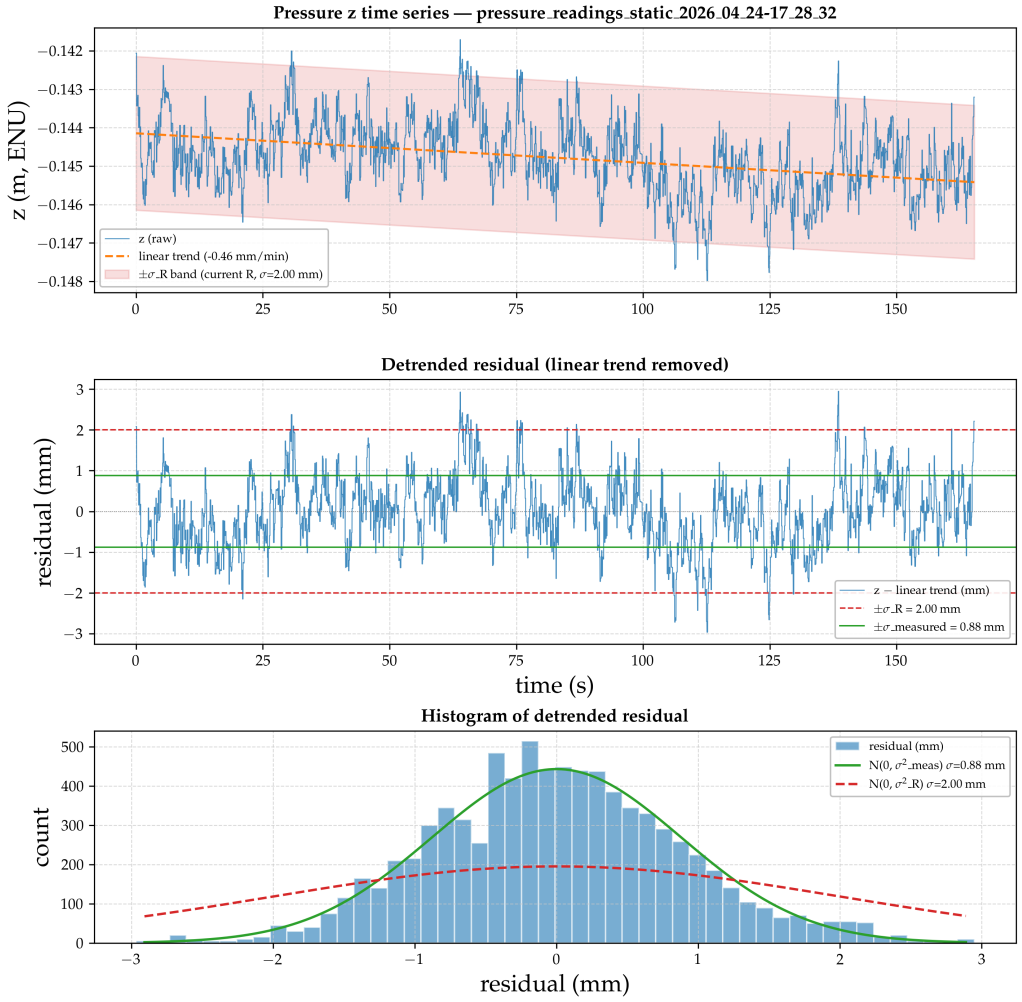


Figure E.4 Pressure-derived depth from a 165-second floating recording (2026-04-24): time series, residual after removing a slow settling trend, and residual histogram. The detrended standard deviation is 0.880 mm, of which only 0.167 mm (first-difference, high-frequency) is sensor noise.

The rate-random-walk coefficient K is the third Allan-variance regime, the rising branch at long averaging times that follows the white-noise (N) and bias-instability (B) regions, and it quantifies how fast the sensor's own bias random-walks. It is the coefficient that would parameterise an explicit sensor-bias state, modelled as a random walk driven by K, in a bias-augmented estimator (Section 5.5). The present fixed 15-element filter carries no such bias state, so K is not used to set the process noise Q, which models vehicle dynamics rather than sensor drift. For the DVL $K^2 \approx 2 \times 10^{-11}\,\mathrm{m^2\,s^{-3}}$, roughly ten orders of magnitude below the velocity process-noise term, so it would affect only long-term dead-reckoning drift, not the short-term estimate.

E.8 Process-Noise (Q) Tuning

The measurement noise R (Section E.7) is a property of the sensors. The process noise Q is a property of the motion. It is the covariance of the zero-mean term by which the true motion is expected to depart from the constant-velocity prediction over one step (Section E.2.2), absorbing drag, thruster transients, and the constant-velocity assumption itself. It cannot be read off a stationary recording the way R can, and is instead tuned from manoeuvring data so that the filter's stated uncertainty matches the errors it actually makes.

The measure of that match is the normalised innovation squared [47], NIS $= \nu^2/S$, with innovation $\nu = z - h(\hat{x}^-)$ and predicted innovation covariance $S = R + HP^-H^\top$. For a correctly tuned scalar update the NIS is χ^2-distributed with one degree of freedom: its median is 0.455, and its 5th and 95th percentiles, 0.0039 and 3.841, set an acceptance band. A median above the band means the filter is over-confident, its S (and through P^- its Q) too small. Below the band, the reverse. Each velocity channel is examined on the manoeuvres that excite it, among them surge for v_x, a yaw turn for v_y through the DVL lever arm, and heave or station-keeping for v_z.

One limitation shapes the procedure. `robot_localization` publishes only the posterior state, not the prior $\hat{x}^-$ that the innovation requires, so the offline diagnostic uses the previous published estimate as an approximate prior. How well that substitution holds depends on how quickly the state moves between samples. The velocity channels are slowly varying and well described by the constant-velocity model, so the previous estimate is close to the true one-step prior. The NIS then shifts by less than about a factor of two, and the verdicts stand. The IMU orientation and angular rate change much faster from one sample to the next and are tightly fused, so the previous posterior is a poor stand-in for the prior, the substitution drives the innovation towards zero, and those channels cannot be tuned this way. Only the velocity channels are treated here.

The recording-time configuration left the velocity NIS medians far above the band, at 6.95 (v_x), 1.68 (v_y), and 16 (v_z): the filter trusted its own prediction far more than the DVL noise warranted. Scaling each diagonal by its measured median, $Q_{\text{new}} \approx Q_0\,\text{NIS}_{\text{median}}$, gives the values of Table E.7, an increase of about seven on v_x and sixteen on v_z. Replaying the tuned filter over the manoeuvre set brings every velocity median inside the band (Figure E.5), for instance v_x from 6.95 to 0.97 on the surge run and v_z from 16 to between 0.2 in steady water and 3.7 during heave.

Table E.7 Velocity process-noise diagonals before and after tuning. The pre-tuning NIS median is measured on the manoeuvre that excites each axis; the tuned value is the moment match $Q_0\,\text{NIS}_{\text{median}}$, rounded. A larger Q raises the predicted innovation covariance and brings the NIS into the acceptance band [0.0039, 3.841].

Velocity state	Q_0 [m^2 s^{-3}]	pre-tuning NIS median	tuned Q [m^2 s^{-3}]
v_x	5×10^{-4}	6.95	3.5×10^{-3}
v_y	1×10^{-3}	1.68	1.7×10^{-3}
v_z	1×10^{-4}	16.0	1.6×10^{-3}

The same over-confidence is visible in the time domain (Figure E.6). Under the recording-time Q the $\pm 2\sqrt{S}$ consistency envelope is too narrow, and the forward-velocity innovation leaves it 63 % of the time, far above the 5 % a consistent filter would show. The tuned Q widens the envelope to match the innovation, and the exceedance falls to 12 %. That it stops at 12 % rather than 5 % reflects the moment match being a single-step approximation, $Q_{\text{new}} \approx Q_0\,\text{NIS}_{\text{median}}$, rather than an exact solve. Driving the exceedance and the NIS median fully onto target would take a further tuning iteration on the same data.

A consistent NIS magnitude confirms only that the innovations have the right variance, not that they are uncorrelated in time. The latter is checked with the innovation autocorrelation $\hat{\rho}(k)$ [51], which measures how strongly the standardised innovation correlates with the one k samples later. For an optimally tuned filter successive innovations are independent, so $\hat{\rho}(k)$ for every lag $k \geq 1$ should fall within the $\pm 1.96/\sqrt{N}$ band, the 95 % interval for zero correlation. Lags outside it signal genuine time correlation the filter is not capturing. Figure E.7 compares the two velocity channels with the most and the least to gain. On the forward channel the recording-time innovations are correlated out to roughly twenty-five lags, and the larger Q pulls the autocorrelation down into the band within a few lags: the surge channel is close to white after tuning, confirming that its pre-tuning colouring was the over-smoothing the increase corrects, not the water current first suspected. The vertical channel behaves differently. Its autocorrelation

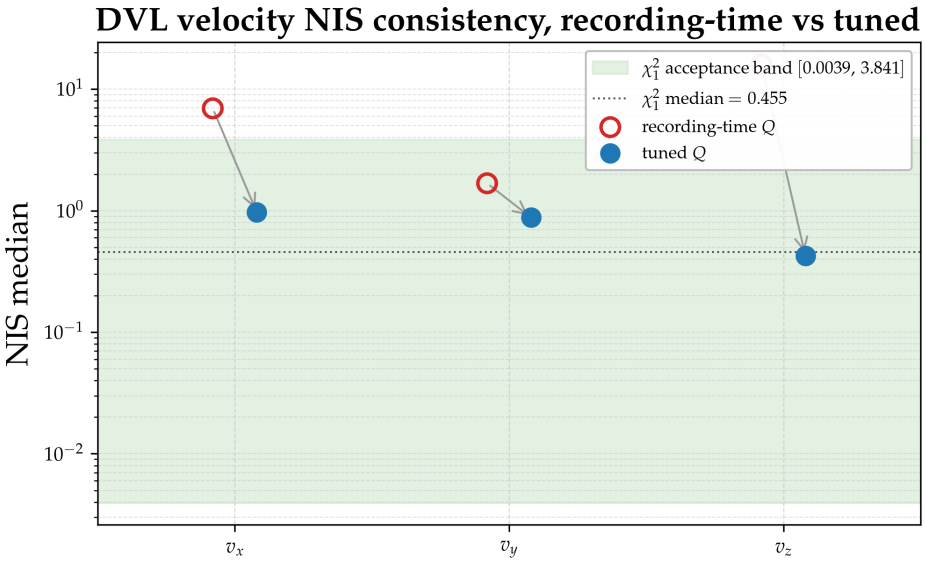


Figure E.5 DVL velocity NIS median before and after tuning, against the χ^2_1 acceptance band [0.0039, 3.841] (shaded) with its median at 0.455. Under the recording-time Q the forward and vertical channels sit above the band (over-confident); the moment-matched Q brings all three inside it. Values are per-bag medians on the manoeuvre exciting each axis.

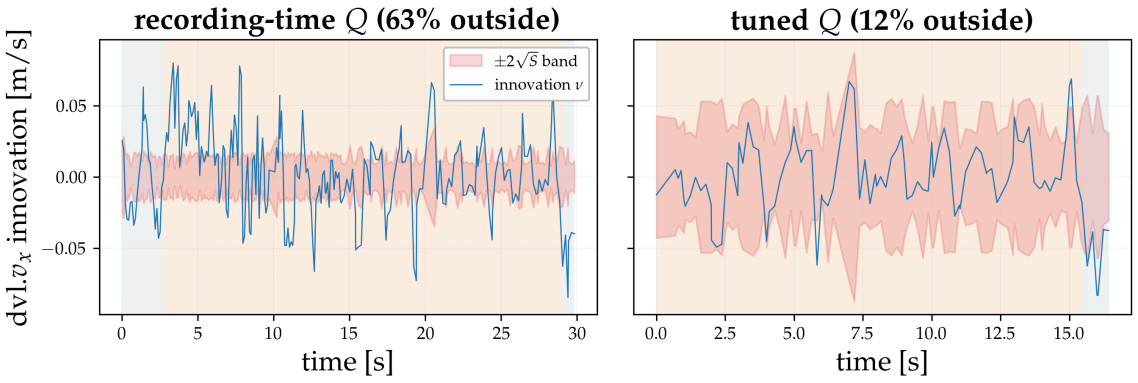


Figure E.6 Forward-velocity (dvl.v_x) innovation and its $\pm 2\sqrt{S}$ consistency envelope on `surge_02`, recording-time versus tuned Q (the time-domain view of the NIS reduction). The recording-time envelope is too narrow and the innovation leaves it 63 % of the time; the tuned Q widens it so that the innovation mostly fits, 12 % outside. Background shading marks the manoeuvre regime.

is essentially unchanged by the sixteen-fold increase and stays outside the band to about fifteen lags under both configurations. Since the Allan analysis found the v_z sensor noise itself to be white at the measurement timescale (Section E.7), this residual correlation is not measurement noise but structure the constant-velocity model omits, most plausibly the vertical motion of the heave manoeuvre itself. Q scales variance, not time correlation, so no value of it removes the tail. This is a model limitation rather than a tuning failure, and is taken up in Section 5.5.

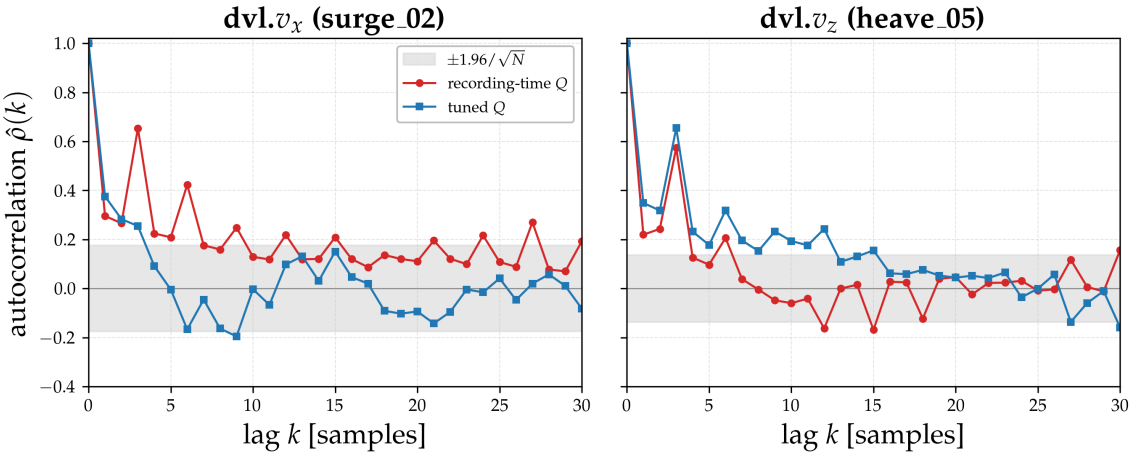


Figure E.7 Innovation autocorrelation $\hat{\rho}(k)$ against lag k (in samples), recording-time versus tuned Q, on the same bag within each panel; the shaded band is the $\pm 1.96/\sqrt{N}$ white-noise interval. The forward channel (left, `surge_02`) decorrelates within a few lags after tuning, whereas the vertical channel (right, `heave_05`) keeps its slow tail under both configurations.

Tuning Q restored the filter's self-consistency but did not measurably change the dead-reckoning drift: on `grid_02` the rate moved from 0.28 to 0.30 m per 100 m, within the confidence interval (Section 5.3.2). The two are decoupled because the drift is set by integrated systematic bias, the DVL scale factor and the gyro's yaw drift, which dominate dead-reckoning error in DVL-aided navigation and are reduced by calibration and bias-state estimation rather than by process noise [52, 35] (Section E.5). Q does not touch them, governing only the short-term velocity-tracking lag, a minor contributor at survey speed. The gain from tuning is therefore not a smaller drift but a state covariance P that matches the filter's actual error. The recording-time filter reported a P several times too tight on the velocity states, so any consumer reasoning about uncertainty would have been misled by it. With the tuned Q the reported P is self-consistent (NIS near one), so the innovation gating, the global filter's weighting of each absolute fix, and any future autonomy function that plans or decides on the state covariance act on an honest uncertainty rather than an over-confident one.

One caveat bounds this result. A sustained water current is not white noise, so Q can be sized to match the variance it injects but not its time structure, and the value found here is calibrated to the current present during these runs. Under a markedly different current the velocity channels would fall out of consistency and Q would have to be retuned, so this is a per-mission calibration rather than a property of the vehicle. The durable fix is not a larger Q but an explicit water-current state in the model, the standard model-aiding remedy in the AUV-navigation literature [35] (Section 5.5). It would carry the deterministic part of the current and leave Q to represent only genuine random disturbance. Only in such an augmented filter would process-noise tuning regain any leverage over drift, and then through the bias and current states, whose process noise sets how quickly the filter follows a drifting bias or a changing current, not through the directly-measured velocity states, where Q cannot reduce the integrated-bias and random-walk terms that set the drift.

Estimating such a state online is limited by observability. The DVL measures the vehicle's velocity over the lake bed, not through the water, and the DVL has no water-track mode, so the current is not directly observable from it. Without a through-water velocity reference it could only be inferred weakly through a thrust-to-velocity hydrodynamic model. A direct measurement is, however, available offline: with the thrusters idle the vehicle drifts with the water, so its bottom-track velocity is then the local current, which calibrates the current for a given site and time but does not track its spatial and temporal variation.

E.9 GNSS Through Ice and Slush

h_{acc} is the GNSS receiver's self-reported horizontal accuracy, not an external reference. When submerged with no usable solution the receiver reports a very large h_{acc}, so a usable fix is taken as $h_{\text{acc}} < 5\,\text{m}$, and depth is the pressure-sensor depth. Three datasets are used: a controlled pre-deployment test at Trübsee, and the St. Moritz (clear ice) and Zermatt (slush) field campaigns. The accuracy is the receiver's own estimate and there is only one field campaign per condition, so the figures below are order-of-magnitude rather than tightly generalisable.
The controlled test separates the ice from the slush. With the antenna held against the ice underside, the sub-ice layer was varied between natural ice with slush beneath it and the same ice with the slush removed (Table E.8): removing the slush recovered centimetre-level accuracy, whereas with slush present the accuracy collapsed to 9 m to 26 m and missed the ±2 m requirement, the per-satellite signal strength (carrier-to-noise-density ratio, C/N_0) dropping by about 25 dB to 30 dB. Since the radome housing itself costs only 1 dB to 2 dB and statistical post-processing does not recover usable accuracy under slush, it is the slush layer, not the ice or the housing, that defeats the receiver.

Table E.8 Controlled Trübsee submersion test: receiver horizontal accuracy and mean signal strength by sub-ice condition.

Sub-ice condition	h_{acc} [m]	mean C/N_0 [dB-Hz]
reference (antenna in air / at surface)	0.014	41–44
under solid ice, slush removed	0.055–0.15	24–30
under natural ice, slush present	9.2 / 16 / 26	9–15

The field campaigns agree with the controlled result. Held against the underside of the clear ice at St. Moritz the receiver resolved a usable fix to a best of 0.58 m (Figure 5.4). Under the slush at Zermatt the usable fix degraded to a median of about 3.5 m, and at the surface both sites reached RTK (≈0.01 m). Reacquisition was quick once the receiver, kept powered, returned to the ice: the first usable fix ($h_{\text{acc}} < 2\,\text{m}$) arrived about 19 s after contact at Zermatt, and the harder sub-metre fix through clear ice about 72 s after reaching the ice at St. Moritz, one clean event at each site.

E.10 Acoustic SBL Accuracy and Ice Comparison

This section gives the evidence for the SBL characterisation summarised in Section 5.4.2. Short-baseline acoustic systems are a standard means of absolute positioning for underwater vehicles, where GNSS is unavailable [53]. The SBL absolute accuracy cannot be ground-truthed under ice, where no GNSS reference exists, so its quality is characterised through metrics that do not depend on the absolute georeferencing: the driver-reported covariance, the position jitter, and a near-hole comparison against the local-EKF dead-reckoning.
Ice (Zermatt, 2026-04-29/30) is compared against submerged open-water runs (Lake Zurich, 2026-04-23). The open-water SBL was mostly logged at the surface, so the comparison uses only its submerged subset (depth-hold, vertical, and yaw runs) and matches the under-ice depth in a common 0.8 m to 2.2 m band. The driver-reported covariance (std_m, the 5.0 m no-fix sentinel excluded) is then lower under ice at every matched range (Table E.9), though it is only a self-report. The offset-independent position jitter, the detrended short-window scatter of the SBL track, is comparable, with a per-bag median of 0.06 m in open water against 0.09 m under ice. Both are at the centimetre level, so ice does not substantially change the SBL noise.
The remaining effect is confined to the ice holes themselves. On `rectangle_01` (Figure 5.5) the SBL residual against the dead-reckoned track, rigidly aligned to remove any constant frame error, is 0.74 m median over the grid but forms a localised cluster at the diagonal corner, 14.7 m ($\approx \sqrt{2} \times 10\,\text{m}$) from the deploy hole where the second hole was cut: 4.7 m median and 6.3 m peak against a 0.76 m baseline elsewhere (permutation test $p = 0.0002$). The jump is thus statistically localised at the hole, and the reported covariance does not rise with it.

Table E.9 Driver-reported covariance std_m (median) under ice (Zermatt) versus submerged open water (Lake Zurich), depth-matched to the 0.8 m to 2.2 m band, by range to the topside buoy.

Range to topside [m]	Open water std_m [m]	Ice std_m [m]
0–5	2.2	1.0
5–10	1.4	1.0
10–15	1.4	1.1

E.11 DVL Bottom-Lock Under Ice

This section gives the evidence behind the DVL characterisation of Section 5.4.3. The screening covers three under-ice campaigns (St. Moritz, clear ice, deep; Zermatt, slush, shallow) against the open-water Lake Zurich reference (Section 5.3), with every metric binned by altitude since the acoustic return falls with range to the bed. As a sanity check, the return does fall with altitude in every campaign (correlation −0.83 to −0.92), confirming the field is read correctly.

The time-weighted lock fraction ranged 93.6 % to 98.6 % across the campaigns and the four-beam fraction stayed at or above 90.9 %, under ice and open water alike. The 3 dB to 5 dB under-ice deficit in the acoustic return (Table E.10) comes from the lake bed, not the ice. The bins where it appears are populated under ice only by Zermatt, since St. Moritz ran too deep to reach them, so the table really sets the Zermatt bed against the Lake Zurich bed, two different beds, rather than ice against open water. St. Moritz settles the question: where its deeper data overlap the open-water range, its return nearly coincides with open water, whereas ice, present in both under-ice campaigns, would have weakened it too. The deficit therefore follows the lake bed, and even the weakest under-ice returns (−42 to −50 dB) stay far inside the lock margin.

Table E.10 DVL acoustic return and beam dropout under ice versus open water, at matched range to the bed and near-level flight (vehicle tilt ≤ 5°), by altitude bin. RSSI is the per-beam acoustic return strength reported by the DVL (higher, i.e. less negative, is stronger); only bins populated by both groups are shown. The under-ice values pool the Zermatt campaigns, as St. Moritz operated too deep to overlap these bins. The high open-water dropout in the shallowest bin reflects the DVL's minimum operating range, where beams drop regardless of environment, and is not an ice advantage.

Altitude [m]	under-ice RSSI [dB]	open-water RSSI [dB]	Δ [dB]	under-ice dropout	open-water dropout
0.0–0.5	−35.3	−32.3	−3.0	0.9 %	7.7 %
0.5–1.0	−42.1	−37.2	−4.9	2.1 %	0.2 %
1.0–1.5	−47.8	−44.3	−3.5	1.5 %	0.9 %
1.5–2.0	−50.2	−45.6	−4.6	3.1 %	0.3 %

The two beam-loss modes named in Section 5.4.3 can be quantified from the binned data. Tilt raises dropout from about 0.1 % below 5° to about 13 % at 10° to 20° on a manoeuvring run, against only 0.9 % to 3.1 % for level under-ice cruise (Table E.10). The thruster-aeration loss reaching 60 % to 67 % while holding at the ice is pinned to propulsion by two facts: across two station-keeping runs the windowed dropout correlated with thruster effort (the mean servo-PWM deviation from idle) at +0.86 and +0.88 and fell to zero at idle, and a down-facing DVL cannot in any case be occluded by the vehicle's upper surface touching the ice. The competing ice mechanism, a bottom-to-ice reverberation cavity, is ruled out: its signature would worsen towards the shallowest range, yet the shallowest bin is the cleanest, and the ice underside reflects only about 11 % of the incident energy against about 99.9 % for a water-air surface, making it the weaker secondary reflector.

The comparison is a screening one, across different lakes, depths, and a fixed configured sound speed (1500 m s^{-1}), so the 3 dB to 5 dB return difference is established as not-ice rather than attributed to a single cause. A direct measurement-noise (R) comparison would need a stationary under-ice recording, which was not collected, so the under-ice R is taken equal to the open-water value (Section E.7) pending that recording (Section 5.5). At St. Moritz the DVL held 0 % dropout while station-keeping under the ice, but the thrust-aeration mechanism could not be reproduced there for lack of thruster telemetry.

APPENDIX F

Path Planning and Control Appendix

F.1 Behaviour Tree

The Behaviour Tree determines which phase of the mission is active and ensuring that the correct plug-ins are executed. For further details on Behaviour Trees [54] provides a thorough introduction. The `nav2` documentation describes how to integrate the Behaviour Tree into the `nav2` framework [3], and the implementation strategy was taken from the `orca4` repository [4]. Several modifications were necessary for the present application. The `orca4` version recalculates the reference path at $f_{\text{replanning}} = 2.0\,\text{Hz}$ without considering how far the AUV has drifted from the reference path. In the presence of simulated current, this led to poor path-tracking behaviour in the POLARIS simulation environment. Because the reference path was re-computed at $f_{\text{replanning}} = 2.0\,\text{Hz}$ from the estimated position, the vehicle was never driven back towards the original reference path but instead followed a shifting one. In the present application, the AUV must travel along specific grid lines — the connecting lines between waypoints — so a stronger attraction to the original reference path was required. This was achieved by a custom plug-in that checks whether the lateral distance between the AUV and the active reference path exceeds a safety threshold $|e_{\text{track}}| > d_{\text{safe}}$. A new reference path is computed only when this check succeeds. Figure 6.2.b, shows an example of this re-planning on the fourth leg. Since an under-ice mission involves ascending to the ice surface, waiting for a GNSS fix, and other recovery behaviours that the Lake Zurich testing did not require, two versions of the Behaviour Tree were developed.

Without Ice Measurement

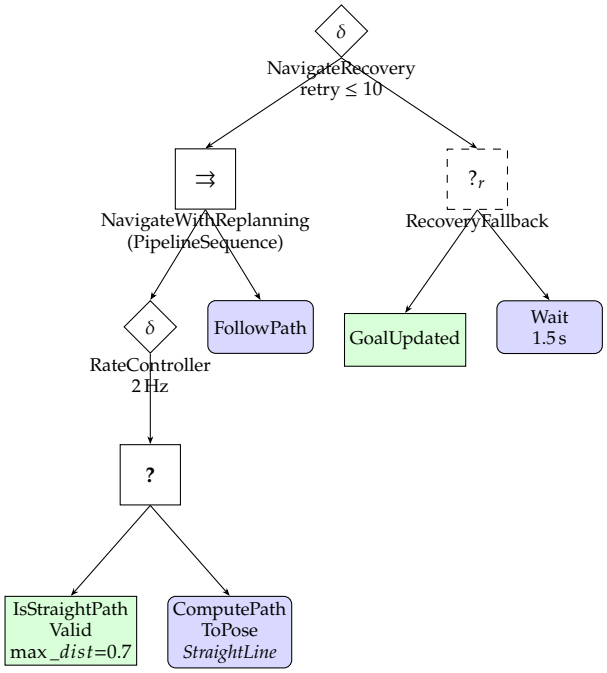


Figure F.1 Behaviour Tree for navigation with replanning, read top-down and left-to-right. Boxes are control nodes: Sequence (→) ticks children until one fails, Fallback (?) until one succeeds; a dashed border ($?_r$) marks a `nav2` reactive Fallback. Rhombi are decorators (δ): recovery/retry and rate controller. PipelineSequence ($\rightrightarrows$) is a `nav2` extension that keeps earlier children ticking while later ones run. Leaves are execution nodes: light-blue rounded boxes are actions (send commands), light-green boxes are conditions (check state).

Figure F.1 visualises the Behaviour Tree used for simple lake missions. At the top level a `RecoveryNode` ticks the main navigation branch: if it fails, a recovery branch is activated and the main branch is retried, up to ten times before the mission is aborted. The tree handles one goal pose at a time — when a mission consists of multiple waypoints, the `nav2` waypoint follower sends each goal to the tree in sequence and advances only after receiving success from the `RecoveryNode`.
The main branch is a `PipelineSequence` that ticks planning and path-following on every cycle. A `RateController` limits replanning to $f_{\text{replanning}} = 2.0\,\text{Hz}$. At each replanning tick a fallback node first checks whether whether the cross-track error remains within d_{safe}; only if the check fails a new straight-line path is computed. The second child of the pipeline sequence, `FollowPath`, continuously sends the active path to the `LongStraightLine` controller and returns success once the goal is reached.
If the main branch fails, a reactive recovery fallback either retries immediately when a new goal has been received or waits two seconds before re-attempting navigation.

With Ice Measurement

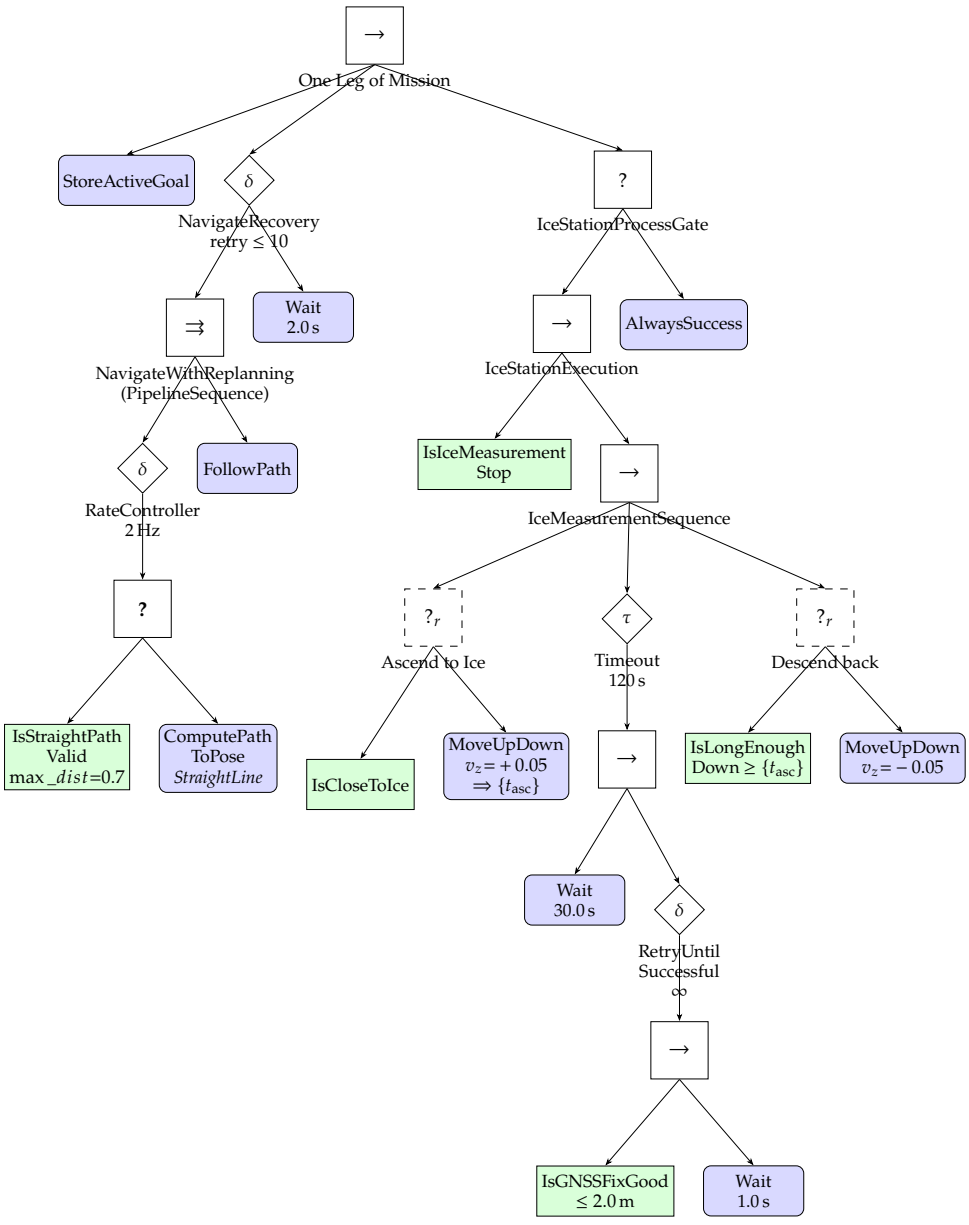


Figure F.2 Behaviour Tree for one waypoint leg for an under-ice mission, read top-down and left-to-right. Boxes are control nodes: Sequence (→) ticks children until one fails, Fallback (?) until one succeeds; a dashed border ($?_r$) marks a `nav2` reactive Fallback. Rhombi are decorators (δ): recovery/retry, rate controller, and timeout (τ). PipelineSequence ($\rightrightarrows$) is a `nav2` extension that keeps earlier children ticking while later ones run. Leaves are execution nodes: light-blue rounded boxes are actions (send commands), light-green boxes are conditions (check state). Blackboard variables are written in braces, e.g. $\{t_{\text{asc}}\}$.

Figure F.2 extends the lake version described above. The core navigation logic — straight-line planning, controller selection, and recovery — remains unchanged. The key additions handle goal latching and an ice-measurement procedure executed upon arrival at designated waypoints. This version of the Behaviour Tree has not yet been tested on hardware and its full functionality therefore cannot be guaranteed. Each mission leg is governed by a top-level sequence with three children that must succeed in order.
The first, `StoreActiveGoal`, captures the incoming goal and writes it to the blackboard along with a flag indicating whether the waypoint is an ice-measurement station or not.
The second child is a `RecoveryNode` identical in structure to the lake version (Figure F.1).
The third child handles post-arrival actions. If the waypoint is flagged as an ice-measurement station, the AUV executes a measurement sequence: it ascends at $\bar{v}_z = 0.1\,\text{m}\,\text{s}^{-1}$ until it has made contact with the ice ceiling, recording the ascent duration t_{asc}. It then waits briefly and attempts to acquire a GNSS fix with a horizontal accuracy of at most $h_{\text{min}} = 2.0\,\text{m}$, subject to a hard timeout of $t_{\text{timeout}} = 1200\,\text{s}$. If no fix is

obtained, the mission continues regardless. Finally, the AUV descends at the same speed for the recorded ascent duration, returning to approximately its original depth. If the waypoint is not an ice-measurement station, this stage is skipped.

F.2 Planner

Algorithm 1 Reference Path $\mathcal{P}$ calculation

Require: start pose $\mathbf{p}_s = (x_s, y_s, z_s)$ and goal pose $\mathbf{p}_g = (x_g, y_g, z_g)$ in map frame, discretisation step d
Ensure: reference path $\mathcal{P}$

$\mathcal{P} \leftarrow \emptyset$
$N \leftarrow \left\lfloor \sqrt{(x_g - x_s)^2 + (y_g - y_s)^2 + (z_g - z_s)^2} \,/\, d \right\rfloor$
$\Delta x \leftarrow (x_g - x_s)/N$
$\Delta y \leftarrow (y_g - y_s)/N$
$\Delta z \leftarrow (z_g - z_s)/N$
for $i = 0$ **to** $N - 1$ **do**
    append $(x_s + i \cdot \Delta x,\ y_s + i \cdot \Delta y,\ z_s + i \cdot \Delta z)$ to $\mathcal{P}$
end for
append $\mathbf{p}_g$ to $\mathcal{P}$
return $\mathcal{P}$

The `planner` plug-in is a straight-line planner that is called by the Behaviour Tree (Section F.1) whenever a new reference path needs to be calculated. It is taken directly from the `orca4` repository [4], with the modification that the reference path can be configured to always start from the origin of the `map` frame. This allows step responses to be simulated by placing the AUV at a lateral offset relative to the reference path.
The reference path $\mathcal{P}$ is expressed in the `map` frame since it is static and drift-free, unlike the drifting `odom` frame. Storing the path in `map` therefore ensures that the geometric line between start and goal stays anchored to real-world coordinates, regardless of how the localisation filter corrects the vehicle pose over time.
Algorithm 1 shows how a reference path is calculated in the `map` frame from the current position to the next waypoint. The Euclidean distance between the start and the goal is divided by the discretisation step $d = 0.05\,\text{m}$ to obtain the number of intermediate poses N. The path is then discretised into the N equally spaced poses along the line connecting start and goal, with each axis interpolated independently.

Future Work The implemented planner generates straight-line paths without considering the vehicle dynamics, which can lead to reference paths that are not kinematically feasible — particularly during sharp turns. A model-based planner that accounts for the the vehicle's state could generate more feasible transition paths between waypoints. This would reduce corner overshoots, lower the cross-track error during turns, and improve overall energy efficiency.

F.3 Modified Pure Pursuit Controller

The modified Pure Pursuit controller (MPPC) is called by the Behaviour Tree at $f_{\text{controller}} = 20\,\text{Hz}$ (see Section F.1).

Architecture

Overview

Figure F.3 Block diagram of the modified Pure Pursuit controller. It takes two inputs: the reference path (generated by the planner) and the estimated state $\hat{x}$ from the EKF. Its output is the commanded velocity $\mathbf{v}_{\text{cmd}}$ with the three components $\bar{v}_x$, $\bar{v}_z$ and $\bar{\omega}_z$.

Figure F.3 shows the architecture of the MPPC. It is a modified version of the controllers presented in [4] and [9]. The guidance controller takes the state estimate from the EKF and the reference path — represented as a set of ordered discrete poses $\mathcal{P}$ — and maps these to a commanded velocity vector for the flight controller. The commanded velocity is defined as $\mathbf{v}_{\text{cmd}} = \begin{bmatrix} \bar{v}_x & \bar{v}_z & \bar{\omega}_z \end{bmatrix}^\top$.

Algorithm

The modified Pure Pursuit controller can be separated into three main parts: the Pure Pursuit geometry, the control law, and the output limiting.

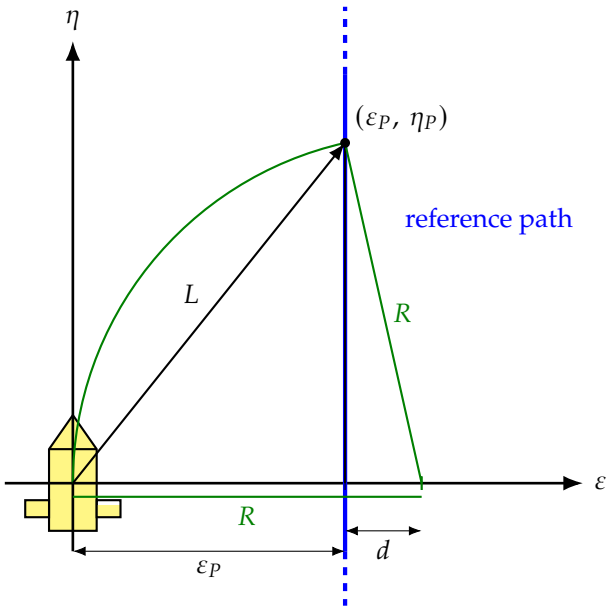


Figure F.4 Geometry of the Pure Pursuit algorithm. The figure has been taken with major changes from [9].
From the current position of the AUV, a look-ahead point on the reference path (blue) is selected at the distance L = LAD. From the current orientation of the drone a circle (green) with radius R can be drawn. Its midpoint always along the ε-direction. Here ε and η are the body-frame FLU coordinates. The distance d is measured along the ε-axis from the midpoint of the arc to the intersection between the reference path and the ε-axis. The look-ahead point has coordinates (ε_P, η_P).

Geometry Figure F.4 visualises the Pure Pursuit geometry. From the current position of the AUV and a lookahead distance LAD and L, a desired position on the reference path is determined.
Following [9], the geometric relations between the lookahead distance L, the body-frame offsets to the lookahead point: η_P along the forward axis and ε_P along the lateral, right axis. d is is the perpendicular

distance from the arc centre to the path

$$L^2 = \varepsilon_P^2 + \eta_P^2 \tag{F.1}$$

$$R = d + \varepsilon_P \tag{F.2}$$

$$R^2 = d^2 + \eta_P^2. \tag{F.3}$$

Substituting $d = R - \varepsilon_P$ and simplifying yields

$$R = \frac{L^2}{2|\varepsilon_P|} \qquad \Leftrightarrow \qquad \kappa = \frac{2}{L^2} \cdot \varepsilon_P \tag{F.4}$$

where κ is the required curvature of the arc (green) and the signed ε_P gives the curvature its correct orientation. The relation $\kappa = 2\varepsilon_P / L^2$ shows that the curvature is proportional to the lateral offset. The geometry therefore acts as a proportional controller on the cross-track error: the further the AUV deviates from the path, the tighter the corrective arc. The commanded yaw rate $\bar{\omega}_z$ then follows from the desired surge speed $\bar{v}_x$:

$$\bar{\omega}_z = \bar{v}_x \cdot \kappa. \tag{F.5}$$

In future work, an alternative formulation using the estimated surge velocity v_x instead of the commanded $\bar{v}_x$ should be investigated, as this would better account for the lag between commanded and actual speed as observed in Section 6.3.2. However, this approach would also propagate the noisy surge estimates v_x into the yaw-rate commands $\bar{\omega}_z$.
The resulting geometry computation is summarised in Algorithm 2.

Algorithm 2 Pure Pursuit Geometry

Require: robot pose $\mathbf{p}_r$ (`odom` frame), estimated twist v_x, v_z, and ω_z, reference path $\mathcal{P}$ (`map` frame)

1: transform $\mathbf{p}_r$ into `map` frame $\rightarrow \mathbf{p}_r^{\text{map}}$ ▷ via Transform Tree
2: $L \leftarrow$ `getLookaheadDist`(v) ▷ static or velocity-scaled lookahead distance
3: $\mathbf{g}^{\text{base link}} \leftarrow$ `findLpoint`$(\mathbf{p}_r^{\text{map}}, L, \mathcal{P})$ ▷ Find lookahead point on $\mathcal{P}$ and transform into base_link
4: $d_z \leftarrow |g_z|$
5: $\kappa \leftarrow 2\,\varepsilon_P / L^2$ ▷ Pure Pursuit curvature
6: $\theta_c \leftarrow \text{atan2}(\eta_P,\ \varepsilon_P)$ ▷ Heading towards lookahead point
7: $s_{\text{rem}} \leftarrow \|\mathcal{P}_{\text{end}} - \mathbf{p}_r^{\text{map}}\|$ ▷ Remaining distance to waypoint

Control Law Algorithm 3 describes how the geometric quantities are mapped to velocity commands. The controller treats depth and the horizontal plane independently. Along the vertical axis, if the depth error d_z exceeds the goal tolerance, the maximum vertical speed $\bar{v}_z^{\max}$ is commanded in the appropriate direction and then progressively reduced as the target depth is approached. For future work, a linear controller such as a PID on the depth error could be investigated as an alternative.
In the horizontal plane, the controller distinguishes three operating modes based on the vehicle's state. If the heading error e_φ is large, the controller stops translating and rotates in place towards the path's direction. The transition between rotation and tracking is governed by a hysteresis: rotation engages when $|e_\varphi|$ exceeds an upper threshold $e_\varphi^{\text{engage}}$ and disengages only when $|e_\varphi|$ drops below a lower threshold $e_\varphi^{\text{release}} < e_\varphi^{\text{engage}}$. This hysteresis prevents the controller from repeatedly entering and leaving the rotation mode when the heading error fluctuates near a single threshold.
In the nominal tracking mode, the surge velocity is set to the desired cruising speed $\bar{v}_x = v_{x,\text{cruise}}$ and then progressively reduced by three independent mechanisms.
A curvature scaling slows the vehicle proportionally as the turning radius $R = 1/|\kappa|$ decreases below a configurable minimum radius $R_{\min}$, preventing the AUV from attempting tight turns at full speed. A waypoint approach scaling ramps down the commanded surge speed $\bar{v}_x$ linearly when the remaining distance d_{rem} falls below a braking distance d_{break}, ensuring a gradual transition into the waypoint region. Finally, a kinematic deceleration profile limits the surge velocity to

$$\bar{v}_x \le \min\Big(K_{\text{decel}}\, d_{\text{rem}},\ \sqrt{2\, a_x^{\max}\, d_{\text{rem}}} \Big), \tag{F.6}$$

where d_{rem} is the Euclidean distance to the active waypoint, K_{decel} is a proportional gain, and $a_x^{\max}$ is the maximum surge de-/acceleration. The proportional term provides a smooth linear ramp-down far

from the waypoint, while the square-root term enforces a hard physical constraint: it is the maximum speed from which the AUV can decelerate to rest within the remaining distance d_{rem}, assuming constant deceleration $a_x^{\max}$. The minimum of the two ensures that at large distances the gentler proportional ramp dominates, while close to the waypoint the kinematic bound takes over and guarantees that the vehicle can stop in time.

Algorithm 3 Modified Pure Pursuit Control Law

Require: $\mathbf{g}$, κ, θ_c, e_φ, d_{xy}, d_{rem}, s_{rem} from Algorithm 2
Ensure: velocity command $(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z)$

$(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z) \leftarrow (0,\ 0,\ 0)$

▷ Depth control - independent vertical axis

if $g_z > d_{z,\text{goal}}$ **then**
  $\bar{v}_z \leftarrow \text{sign}(g_z) \cdot \bar{v}_z^{\max}$
  $\bar{v}_z \leftarrow \text{sign}(\bar{v}_z) \cdot \min\left(|\bar{v}_z|,\ K_{\text{decel}}\,|g_z|,\ \sqrt{2\,a_z^{\max}\,|g_z|}\right)$
end if

▷ XY control - Modified Pure Pursuit

if ShouldRotateToPath(e_φ) **then** ▷ Hysteresis
  $(\bar{v}_x,\ \bar{\omega}_z) \leftarrow$ RotateToHeading(e_φ)
else
  $\bar{v}_x \leftarrow \bar{v}_x^{\max}$
  $\bar{v}_x \leftarrow \bar{v}_x \cdot \min\left(1,\ |\kappa|^{-1}/R_{\min}\right)$ ▷ curvature scaling
  $\bar{v}_x \leftarrow \min\left(\bar{v}_x,\ K_{\text{decel}}\,d_{\text{rem}},\ \sqrt{2\,a_x^{\max}\,d_{\text{rem}}}\right)$ ▷ deceleration
  $\bar{\omega}_z \leftarrow \bar{v}_x \cdot \kappa$
end if
return $(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z)$

Safety and Rate Limiting After the control law has determined the desired velocity command, two post-processing steps are applied before the command is sent to the flight controller, as summarised in Algorithm 4.

The first step enforces acceleration limits on all three command channels. The change in each velocity component between consecutive ticks is clamped to $\pm a^{\max}\Delta t$, where $a^{\max}$ is the per-axis maximum acceleration and $\Delta t = 1/f_{\text{controller}}$ is the control period. This prevents the flight controller from receiving step changes in the commanded velocity, which would excite the vehicle dynamics and degrade tracking performance.

The second step is a velocity-divergence emergency stop. At each tick the controller computes the angle between the AUV's actual velocity vector and the path direction. If this angle exceeds a configurable threshold $\theta_{\max}$ and the vehicle is moving faster than a minimum speed $v_{\min}$, the controller immediately disarms the thrusters and switches the flight controller to manual mode. This safeguard catches situations in which the AUV is being carried away from the path — for example by a strong current that the controller cannot overcome — and prevents the vehicle from travelling further in an uncontrolled direction.

Algorithm 4 Safety and Rate Limiting

Require: velocity command $(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z)$ from Algorithm 3, previous command $(\bar{v}_x^{\text{prev}},\ \bar{v}_z^{\text{prev}},\ \bar{\omega}_z^{\text{prev}})$, estimated velocity $\mathbf{v}$, yaw error e_φ
Ensure: safe velocity command $(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z)$

▷ Acceleration limiting
$\bar{v}_x \leftarrow \text{clamp}\big(\bar{v}_x,\ \bar{v}_x^{\text{prev}} - a_x^{\max}\Delta t,\ \bar{v}_x^{\text{prev}} + a_x^{\max}\Delta t\big)$
$\bar{v}_z \leftarrow \text{clamp}\big(\bar{v}_z,\ \bar{v}_z^{\text{prev}} - a_z^{\max}\Delta t,\ \bar{v}_z^{\text{prev}} + a_z^{\max}\Delta t\big)$
$\bar{\omega}_z \leftarrow \text{clamp}\big(\bar{\omega}_z,\ \bar{\omega}_z^{\text{prev}} - \dot{\omega}_z^{\max}\Delta t,\ \bar{\omega}_z^{\text{prev}} + \dot{\omega}_z^{\max}\Delta t\big)$

▷ Velocity-divergence emergency stop
$v_{xy} \leftarrow \sqrt{v_x^2 + v_y^2}$
if $v_{xy} > v_{\min}$ **then**
  $v_{\parallel} \leftarrow v_x \cos(e_\varphi) - v_y \sin(e_\varphi)$
  $v_{\perp} \leftarrow v_x \sin(e_\varphi) + v_y \cos(e_\varphi)$
  $\theta_{\text{div}} \leftarrow \text{atan2}(|v_{\perp}|,\ v_{\parallel})$
  if $\theta_{\text{div}} > \theta_{\max}$ **then**
    disarm thrusters, switch to MANUAL, abort mission
  end if
end if

return $(\bar{v}_x,\ \bar{v}_z,\ \bar{\omega}_z)$

Tuning

The main tuning parameter is the lookahead distance LAD, which, as explained in Section F.3, acts as a proportional gain on the lateral offset. The tuning was first carried out in simulation, using the constraints described in Section 6.3.1 and listed in Table 6.1. Once the best lookahead distance was identified, the configuration was validated in different simulated scenarios with water currents.

Lookahead-distance

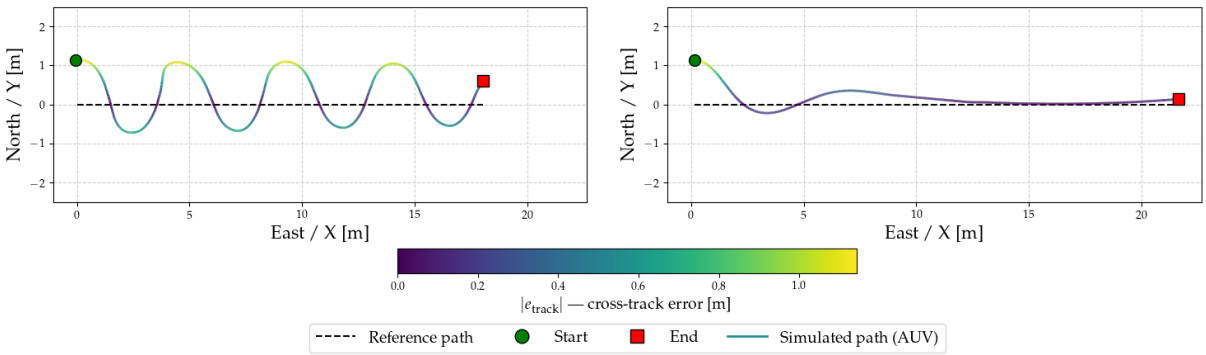


Figure F.5 Effect of the lookahead distance LAD on the step response onto a straight reference path, with the AUV starting from a lateral offset of $d_{\text{offset}} = 1.2\,\text{m}$. The simulated path is coloured by the cross-track error $|e_{\text{track}}|$, defined as the distance from the AUV to the closest point on the reference path; the start and end points are marked. Subfigure a) (left) uses a small lookahead distance LAD = 2.0 m, Subfigure b) (right) a larger one of LAD = 2.5 m.

The approach for tuning the lookahead distance was to create a reference path starting from the origin of the `map` frame rather than from the initial placement of the AUV, and to position the vehicle with a lateral offset of $d_{\text{offset}} = 1.2\,\text{m}$ from the path. This configuration produced a step response, allowing the convergence behaviour to be observed as the AUV approaches the path from a significant lateral distance. Figure F.5 shows the effect of the lookahead distance on the step response. In Subfigure a) the AUV had LAD = 2.0 m and oscillated with an amplitude of $\hat{a} = 0.75\,\text{m}$ without converging to the path. In Subfigure b) the AUV had LAD = 2.5 m and settled smoothly onto the reference path without significant overshoot

with.
This behaviour was expected: as shown in Equation (F.4), the curvature scales with $|\kappa| \propto \frac{1}{\text{LAD}^2}$, so a shorter lookahead distance increases the effective proportional gain and amplifies the corrective action on the lateral offset, leading to oscillation as Figure F.5.a clearly shows. A longer lookahead reduces the gain and produces a less oscillatory response (see Figure F.5.b). The Pure Pursuit controller also lacks derivative action, which could further damp the oscillations.

Validation with Water Current

To test whether the tuned parameters are also applicable in the presence of water currents, a current simulator was added to the simulation environment. The current was modelled as a first-order Gauss–Markov process in the $X-Y$-plane, as described in [10] and in Section 6.2.

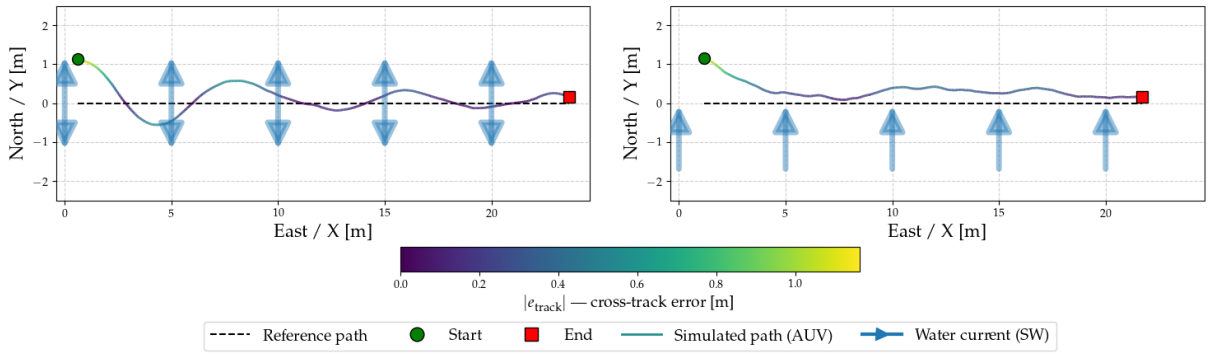


Figure F.6 Effect of the simulated water current on the step response with a lookahead distance LAD = 2.5 m and an initial lateral offset $d_{\text{offset}} = 1.2\,\text{m}$. The simulated path is coloured by the cross-track error $|e_{\text{track}}|$, defined as the distance from the AUV to the closest point on the reference path; the start and end points are marked. Subfigure a) shows the response with a simulated water current of $v_{x,\text{current}} = v_{y,\text{current}} = 0.5\,\text{m}\,\text{s}^{-1} \cdot \sin\left(\frac{2\pi}{15\,\text{s}}\,t\right)$ and b) with $v_{x,\text{current}} = 0\,\text{m}\,\text{s}^{-1}$ and $v_{y,\text{current}} = 0.5\,\text{m}\,\text{s}^{-1}$.

Figure F.6 shows the same step-response setup with the added current. The AUV was still able to track the reference path despite the lateral offset and the disturbance, although a small residual cross-track error remained. The controller also lacks integral action, which could improve steady-state tracking in the presence of currents but would risk integrator wind-up in the presence of sensor drift. These tuned parameters were therefore considered suitable for further testing on a square trajectory. The square mission results are discussed in Section 6.3.1.

Choice of Guidance Controller

The Pure Pursuit controller [9] was selected as the baseline guidance controller for its simplicity, geometric transparency, and minimal tuning effort. These properties make it a natural baseline for a first integration of the autonomy stack, where the priority was a working end-to-end pipeline rather than optimal path-tracking performance. An alternative geometric approach is the line-of-sight controller [55], which offers comparable complexity but differs in how it defines the cross-track error and the convergence law. More advanced options include for example model-predictive path-following [56], backstepping [57], or sliding-mode controllers [58]. They would improve disturbance rejection and constraint handling at the cost of requiring a hydrodynamic model and significantly more tuning. Starting from the Pure Pursuit allowed the full vehicle pipeline to be validated in simulation and in the field as fast and reliable as possible.

F.4 Second Simulation Cycle

The low-level controller tuning carried out at Lake Zurich revealed tighter acceleration constraints for the surge and yaw rate axes, which are a factor of 5 and 1.75 smaller than the initial simulation values; the updated values are listed in Table 6.2. To account for these reduced constraints, a second tuning iteration of the lookahead distance was performed, followed by validation on the same square mission

in Section 6.3.1.

Tuning — Straight

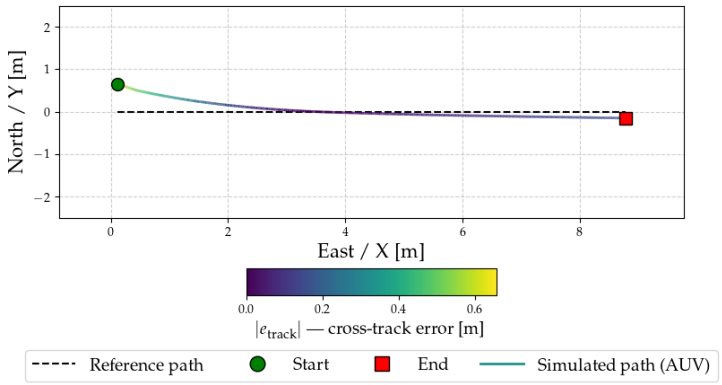


Figure F.7 Step response, with the AUV starting from a lateral offset of $d_{offset} = 1.2\,m$. The simulated path is coloured by the cross-track error $|e_{track}|$, defined as the distance from the AUV to the closest point on the reference path. The constraints for the velocities have been taken from Table 6.1 and the maximum acceleration from the column "**Final Values**" of Table 6.2.

Figure F.7 shows the step response with an initial lateral offset of $d_{offset} = 0.8\,m$. The lookahead distance yielding the best tracking performance was LAD = 1.7 m. Since the lookahead distance was shorter, a smaller safety threshold was also required, $d_{safe} = 0.7\,m$. The step response shows comparable tracking quality to the previous iteration with the original constraints.

Validation — Square

To validate the tuned parameters, the same square mission as in Section 6.3.1 was executed, with edge lengths of 11.5 m.

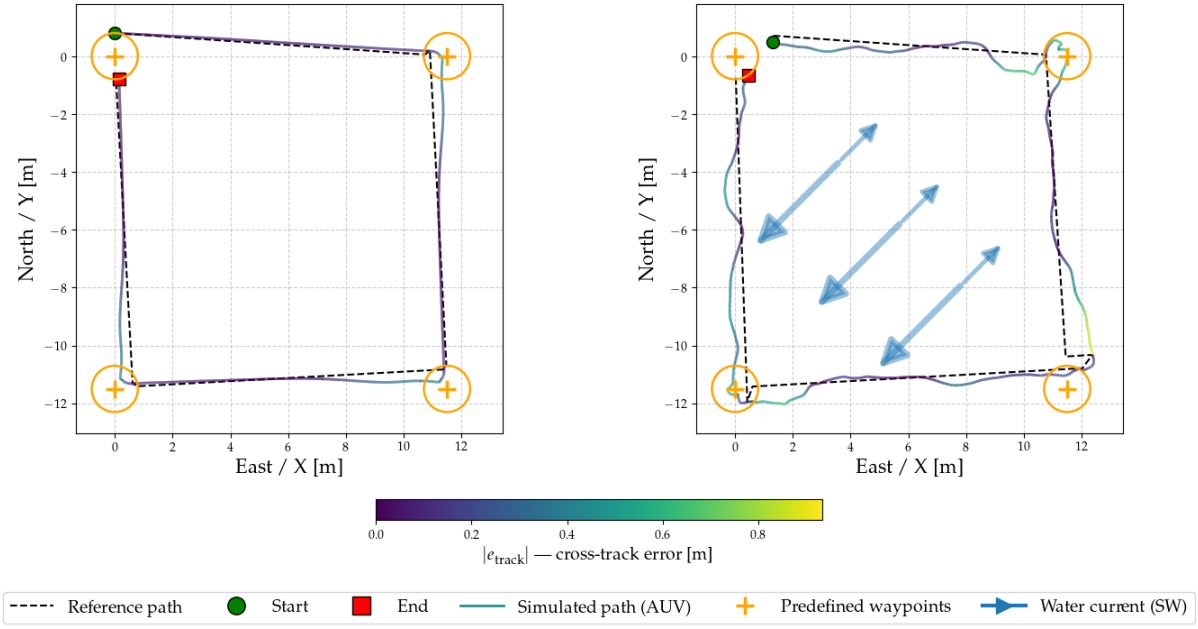


Figure F.8 Simulated AUV path for the square mission with edge lengths of 11.5 m, coloured by the cross-track error $|e_{track}|$, defined as the distance from the AUV to the closest point on the reference path. The predefined waypoints are marked. A lookahead distance of LAD = 1.7 m is used. Subfigure a) (left) shows the result without water current, while Subfigure b) (right) shows the result with a simulated diagonal water current.

Figure F.8 shows the AUV's simulated path along the four waypoints, analogous to the results presented in Section 6.3.1 as Figure 6.2.
Figure F.8.a, shows similar tracking behaviour but with reduced overshoot at the corners, which was a direct consequence of the stronger and earlier braking deceleration.
Figure F.8.b, shows the result with the simulated water current with the same parameters as in Section 6.3.1. The root-mean-square cross-track error for Subfigure a) was RMSE = 0.21 m with a completion time of $T_{\text{completion}} = 4\,\text{min}$. With simulated water current (Subfigure b), the error increased to RMSE = 0.40 m and the completion time was reduced to $T_{\text{completion}} = 3\,\text{min}\,48\,\text{s}$. Overall, the modified Pure Pursuit controller improves upon the previous configuration: the proximity and deceleration scaling cause the AUV to brake earlier before each waypoint, reducing corner overshoot, while the path-tracking accuracy is maintained.

F.5 Low-Level Controller Tuning — Further Reading

The low-level controller runs on the flight controller, the Pixhawk 6C [8], on which the Polaris-modified ArduSub firmware is deployed [2, 59]. The cascaded PID gains must therefore be modified on the flight controller itself, not on the onboard computer. This was done using QGroundControl [6], an open-source ground station application that communicates with the Pixhawk and allows the flight-controller parameters to be read and written in real time.
To achieve accurate tracking of the velocity commands issued by the MPPC, the surge and yaw-rate PID loops were tuned separately using ramp inputs and then validated together in a combined test.

Surge Commands

Introduction

The surge PID should accurately track surge velocity commands $\bar{v}_x$ during both transient and steady-state phases. Of these, maintaining the commanded cruising speed without droop over long straight segments was the more critical requirement: during a typical mission the AUV spends the majority of its time at constant velocity between waypoints, so even a small steady-state error accumulates into a significant time loss and, given the MPPC's relation $\bar{\omega}_z = \bar{v}_x \cdot \kappa$ propagates into incorrect yaw-rate commands. Additionally, the maximum ramp rate that the vehicle can reliably follow was identified from the test data and used to set the surge acceleration constraint $a_x^{\max}$ in the MPPC.

Materials and Methods

Velocity ramp commands were sent to the surge velocity PID loop while the vehicle's response was recorded. The PID gains were tuned iteratively by analysing the commanded and estimated velocity signals live in Foxglove [7], adjusting the gains between runs until satisfactory tracking was achieved.

Results and Discussion

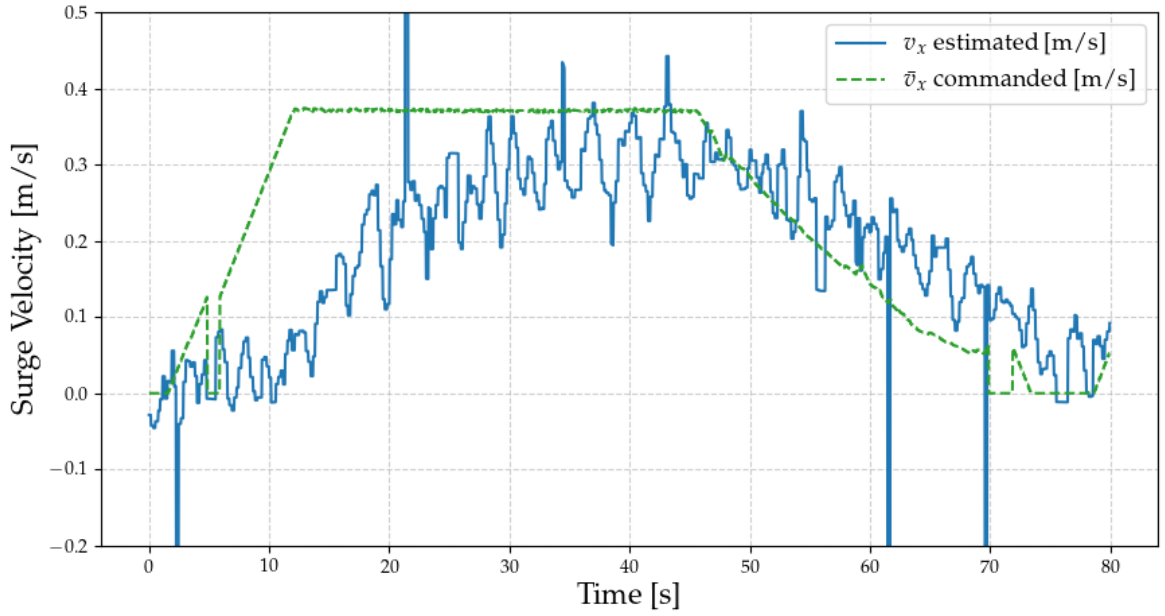


Figure F.9 Surge velocity response for a ramp with rise time $t_{\text{rise}} = 10.0\,\text{s}$. The commanded surge velocity $\bar{v}_x$ (green) ramps up to the predefined cruise speed $v_{x,\text{cruise}} = 0.4\,\text{m}\,\text{s}^{-1}$, while the estimated surge velocity v_x (blue), obtained from the EKF as the estimated body-frame surge velocity, tracks the reference.

A pure PD controller proved insufficient for the surge control loop: the vehicle accelerated with a visible delay and was unable to reach the commanded constant speed, exhibiting a persistent steady-state error.

Table F.1 Final tuning of the surge PID controller in ArduSub. The "**Value**" column gives the value actually used, while the "**Maximum Value**" column gives the largest value selectable in ArduSub without modifying the firmware. Units have been omitted for all parameters except the integral wind-up protection.

Parameter	Value	Maximum Value
Proportional Gain	1.00	6.00
Derivative Gain	0.25	1.00
Integral Gain	0.30	1.00
Integral Windup protection in $\mathrm{cm\,s^{-2}}$	600	4500
Feedforward Gain	3.00	6.00

Adding an integral term with anti-wind-up protection resolved this to some extent. Figure F.9 shows the response with the full PID controller, using the parameter set listed in Table F.1 and the "**Final Value**" values of Table 6.2. With the integral action the AUV tracked the constant-speed phase with only a small steady-state error and a settling time of $T_\mathrm{s} \approx 30\,\mathrm{s}$. The response (blue) was noticeably noisy and the surge dynamics are strongly influenced by the prevailing water current — the vehicle accelerated faster when moving with the current and more slowly when moving against it and vice versa for the deceleration. The maximum achievable acceleration was determined by commanding ramps with progressively shorter rise times until the vehicle could no longer track the reference without significant lag. The shortest rise time t_rise at which the AUV still reliably tracked the ramp up yielded the acceleration constraint

$$a_x^\mathrm{max} = \frac{v_{x,\mathrm{cruise}}}{t_\mathrm{rise}} = \frac{0.4\,\mathrm{m\,s^{-1}}}{35.0\,\mathrm{s}} = 0.011\,\mathrm{m\,s^{-2}}\,. \tag{F.7}$$

The integral action was essential for eliminating the steady-state velocity error. At constant speed the thruster force must exactly balance the hydrodynamic drag, which scales non-linearly with velocity $F_D \propto v_x^2$. A proportional controller alone settles at the operating point where the proportional output matches the drag force, leaving a steady-state error. The integral term accumulates this residual over time and increases the thruster command until the drag is fully compensated. The derivative gain was kept low because the velocity estimate from the EKF contains high-frequency noise, which the derivative term would amplify into thruster chatter.

Conclusion

The surge control loop could still be tuned more aggressively, but the performance was sufficient for the purposes of this work. There remains room for improvement, for instance the settling time T_s and the steady-state error. Further refinement — for instance to improve disturbance rejection in the presence of water currents — is left for future work.

Yaw-Rate Commands

Introduction

The tuning objective for the yaw rate loop differs from that of the surge loop. Whereas the surge controller shall primarily maintain a constant cruising speed without droop, the yaw rate controller should accurately follow rapidly changing commands $\bar{\omega}_z$. The MPPC issues yaw rate corrections that are typically short in duration but change rapidly, so the priority was responsive ramp tracking rather than steady-state accuracy.

Materials and Methods

The same procedure as for the surge commands was followed, with yaw rate ramp commands in place of surge velocity ramps.

Results and Discussion

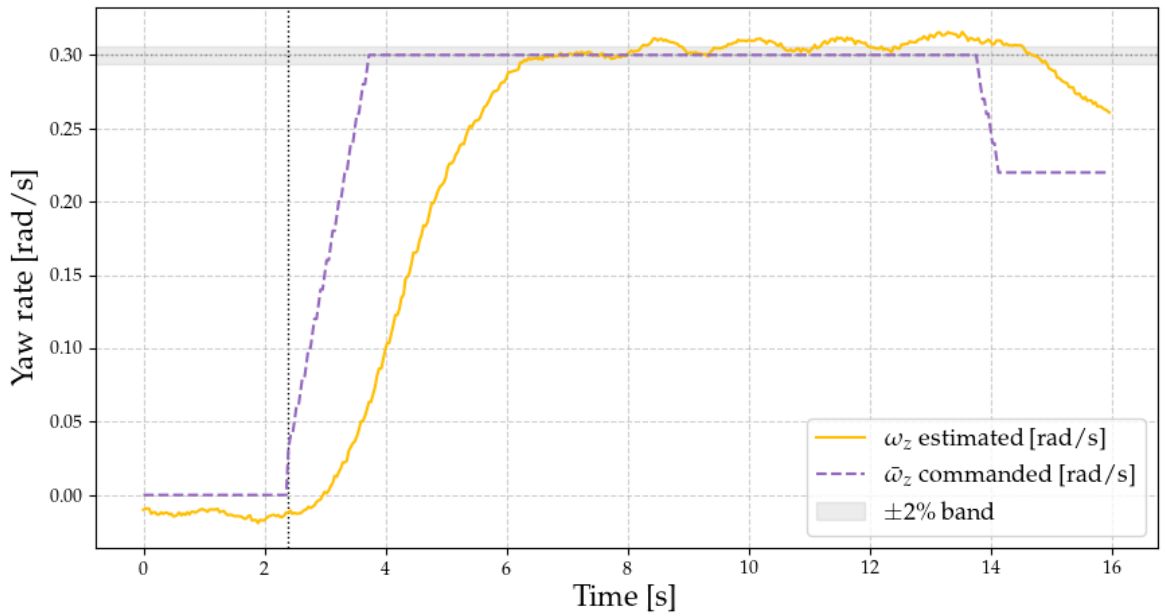


Figure F.10 Yaw rate response for a ramp with rise time $t_{\text{rise}} = 1.5\,\text{s}$. The commanded yaw-rate $\bar{\omega}_z$ (purple) ramps up to the predefined maximum yaw-rate $\bar{\omega}_z^{\max} = 0.3\,\text{rad}\,\text{s}^{-1}$, while the estimated yaw-rate ω_z (yellow), obtained from the EKF as the estimated body-frame yaw rate, tracks the reference.

Table F.2 Final tuning of the yaw PD controller in ArduSub. The "**Value**" column gives the value actually used, while the "**Maximum Value**" column gives the largest value selectable in ArduSub without modifying the firmware. Units have been omitted for all parameters.

Parameter	Value	Maximum Value
Proportional Gain	0.3	0.5
Derivative Gain	0.016	0.02
Integral Gain	0.0	0.05
Feedforward Gain	0.10	0.5

Figure F.10 shows the result of the tuning, with the corresponding PD (not PID) gains listed in Table F.2. The estimated yaw rate ω_z (yellow) followed the commanded ramp $\bar{\omega}_z$ (purple) with a noticeable delay of about 1 s before it began to respond, and reached the commanded rate of $\bar{\omega}_z = 0.30\,\text{rad}\,\text{s}^{-1}$ after a settling time of $T_s \approx 5\,\text{s}$, estimated from the start of the ramp. Thereafter it stayed within the ±2 % band with only a small steady-state error and minor fluctuations. The maximum yaw acceleration was determined using the same approach as for the surge axis, yielding $\dot{\omega}_z^{\max} = \frac{0.1\,\text{rad}\,\text{s}^{-1}}{5.0\,\text{s}} = 0.02\,\text{rad}\,\text{s}^{-2}$.
The integral gain was set to zero, unlike in the surge loop. The commanded yaw rates are smaller in magnitude, and the moment of inertia and hydrodynamic drag about the yaw axis are both lower than in the surge loop, so the residual error without integral action remained acceptable while keeping the loop responsive and free of integral wind-up. A further reason for omitting the integral term was that the yaw-rate estimate is prone to drift, which an integrator would accumulate over time and turn into a slowly growing bias in the thruster command. The tuning priority was therefore placed on a fast, well-damped ramp response rather than on driving the tracking error to zero.

Conclusion

With purely proportional–derivative action the loop cannot fully eliminate the lag during a ramp, since a constant-slope reference produces an unbounded steady-state velocity error that only integral action could remove [60]. This is visible in Figure F.10 as the offset during the ramp-up phase. Because the guidance controller relies on accurate tracking of the commanded yaw-rate ramps, reintroducing a small, carefully tuned integral term to reduce this offset — without provoking overshoot or accumulating drift — is a clear avenue for improvement and is left for future work.

F.6 First Deployment of the Full Autonomy Stack

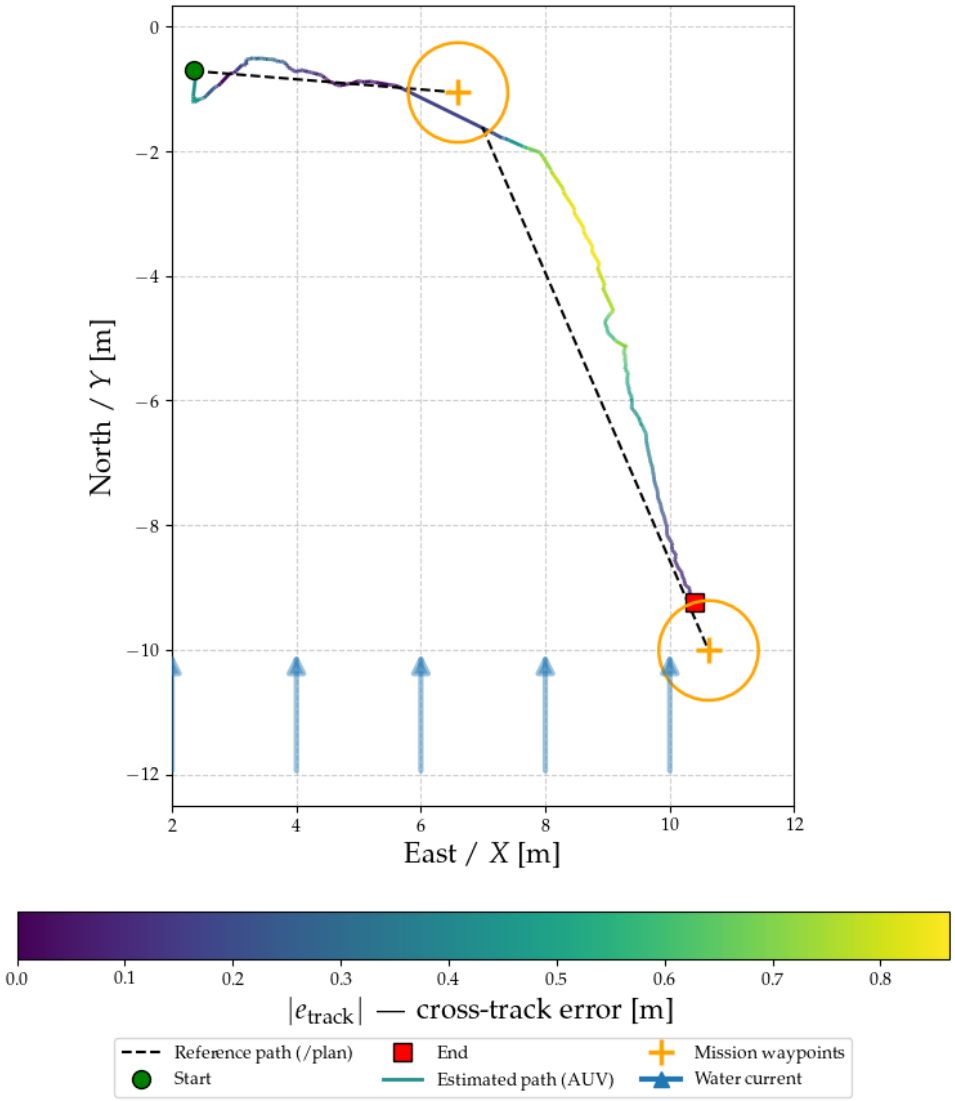


Figure F.11 Top-down $X - Y$ view of the first autonomous completed mission at Lake Zurich. The estimated AUV path (from the EKF) is shown against the online-computed reference paths (dashed) and the predefined mission waypoints, coloured by the cross-track error $|e_{\text{track}}|$, defined as the distance from the AUV to the closest point on the reference path. The blue arrows indicate the direction of the northward water current only and are not scaled to its magnitude.

Figure F.11 shows the result of the first completed autonomous lake mission, consisting of two waypoints. The water current was weak but directed northward. The AUV navigated towards the first waypoint, then drifted forward in a straight line; once the second plan was loaded, the Schmitt trigger activated (not visible in the plot) and the AUV converged asymptotically towards the goal and completed the mission successfully.

This test showed that the architecture works with the current setup, from state estimation through to waypoint loading. A clear area for improvement is the braking behaviour before waypoints as have been discussed in Section 6.3.1.

F.7 Autonomous Grid in Lake Zurich — Further Reading

Hysteresis

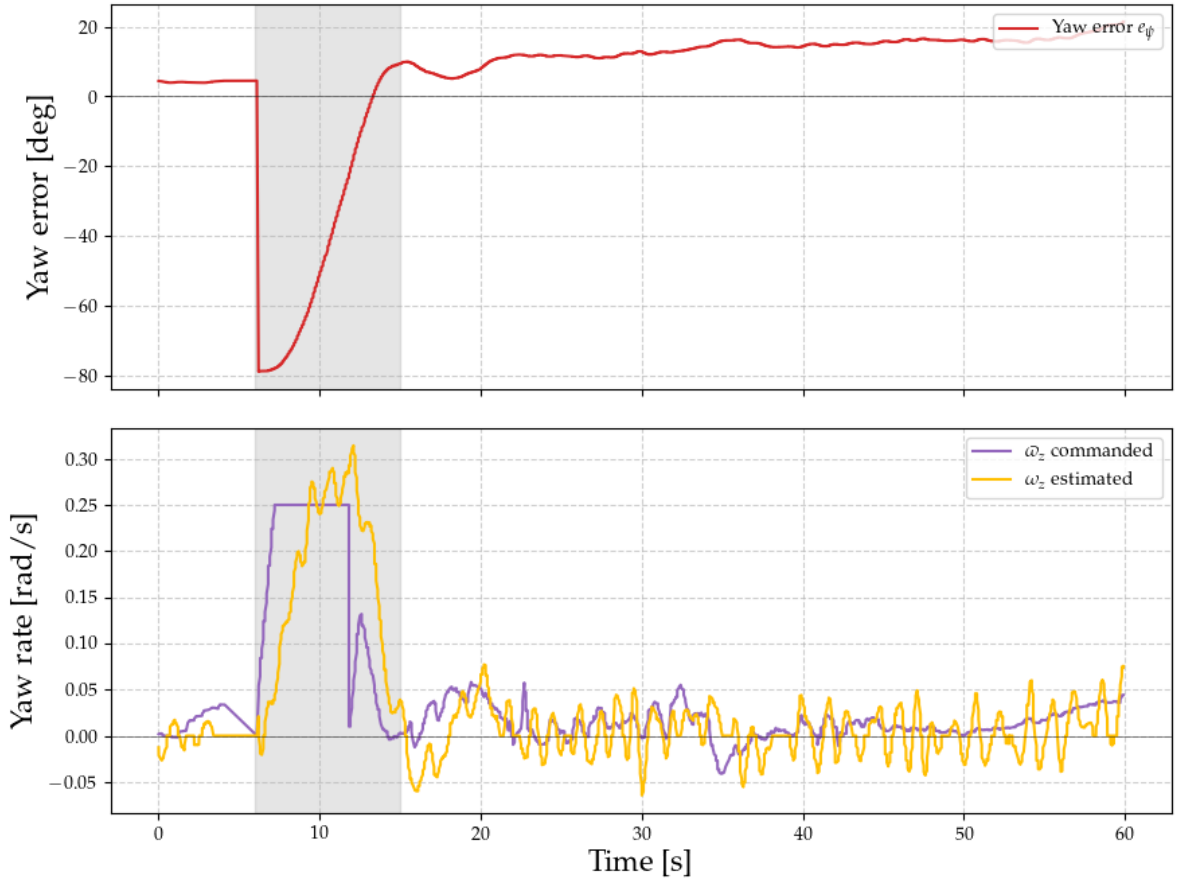


Figure F.12 Yaw error and yaw-rate response during a representative 90° turn of the grid mission. The upper subplot shows the yaw error e_φ, defined as the difference between the current heading and the heading of the straight-line reference path. The lower subplot shows the commanded yaw rate $\bar{\omega}_z$ (purple dashed) and the estimated yaw rate ω_z (yellow). The grey region marks the hysteresis activation window.

The grey-shaded region in Figure F.12 marks the hysteresis activation window during a representative 90° turn. At the onset of the turn, the yaw error jumped abruptly as the new reference path was loaded, and the controller immediately commanded the maximum yaw rate. The error then decreased approximately linearly as the vehicle rotated. However, the hysteresis disengaged too abruptly: the yaw rate command dropped sharply, causing the vehicle to overshoot the desired heading. After the hysteresis phase, the small commanded yaw rate $\bar{\omega}_z$ corrections and the oscillating estimated yaw rate ω_z were insufficient to bring the error e_φ back to zero, resulting in a persistent residual heading offset. This suggests that the hysteresis deactivation logic could be improved. In particular, the maximum yaw acceleration constraints obtained in Section 6.3.2 were only enforced during normal path-tracking mode and were bypassed during the hysteresis phase, leading to the abrupt deceleration and subsequent overshoot.

40 m × 40 m **Grid**

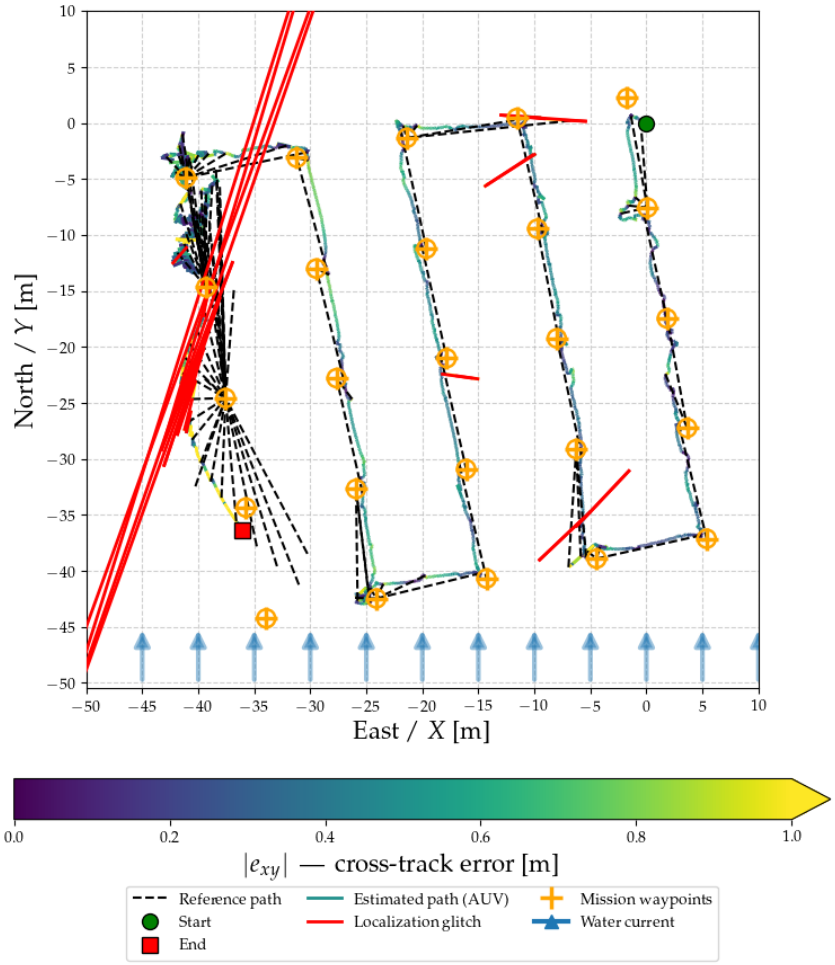


Figure F.13 Top-down $X - Y$ view of the 40 m × 40 m grid mission at Lake Zurich. The estimated AUV path (from the EKF), coloured by the cross-track error $|e_{\text{track}}|$, is shown against the online-computed reference paths (dashed) and the predefined mission waypoints. The start and end points are marked, and the red segments mark localisation glitches. The blue arrows indicate the direction of the northward water current only and are not scaled to its magnitude.

Figure F.13 shows the execution of the 40 m × 40 m grid with a 10 m spacing. The AUV tracked the reference paths accurately and reached each waypoint in sequence. On the final leg, however, the DVL began reporting invalid velocity spikes, causing the dead-reckoning estimate to diverge. The mission was subsequently aborted manually.

APPENDIX G

Ice Measurement Appendix

G.1 St. Moritz Measurement Dataset

Table G.1 lists all validated measurement pairs used in the statistical analysis presented in Section 7.2.

Measurement	Sensor Thickness (cm)	Reference Thickness (cm)
1	17.336	15.750
2	17.447	17.650
3	14.678	14.150
4	13.333	13.750
5	15.245	17.000
6	14.667	15.350
7	20.127	20.250
8	19.994	20.550
9	19.338	18.250
10	20.961	19.750
11	13.655	13.450
12	20.739	20.950
13	18.493	19.550
14	18.826	18.950
15	14.990	16.550
16	12.910	12.750
17	15.212	15.250
18	14.500	15.150
19	16.913	17.000
20	13.644	13.750
21	16.458	16.500
22	18.871	18.000
23	19.327	19.250
24	23.018	22.250
25	22.307	20.250
26	17.258	16.650
27	16.547	16.550
28	20.072	20.000
29	16.702	16.650

Table G.1 Sensor measurements of the estimated ice-thickness and their corresponding reference measurements.

Bibliography

[1] W. Guo, T. Tyrovouzis, and L. E. Kavraki, "Python bindings for a large c++ robotics library: The case of ompl," *arXiv preprint arXiv:2603.04668*, pp. 1–8, 2026.

[2] A. D. Team, "Ardupilot sub," https://ardupilot.org/sub/, accessed: 2025-06-01.

[3] S. Macenski, F. Martin, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[4] C. McQueen, "orca4: ROS-2 AUV based on the BlueROV2, ArduSub and Navigation2," Jun. 2022. [Online]. Available: https://github.com/clydemcqueen/orca4

[5] OpenRobotics, "Gazebo," https://gazebosim.org/home, accessed: 2025-06-01.

[6] QGroundControl, "Qgroundcontrol," https://qgroundcontrol.com/, accessed: 2025-06-01.

[7] Foxglove, "Foxglove," https://app.foxglove.dev/, accessed: 2025-06-01.

[8] Holybro, "Pixhawk 6c," https://holybro.com/products/pixhawk-6c?variant=42783569871037, accessed: 2025-06-01.

[9] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, January 1992.

[10] M. Nitsch, D. Stenger, and D. Abel, "Automated tuning of nonlinear kalman filters for optimal trajectory tracking performance of auvs," 2023. [Online]. Available: https://arxiv.org/abs/2304.03565

[11] T. Moore and D. Stouch, "A generalized extended kalman filter implementation for the robot operating system," in *Intelligent Autonomous Systems 13 (IAS-13)*, ser. Advances in Intelligent Systems and Computing, vol. 302. Cham: Springer, 2016, pp. 335–348.

[12] N. El-Sheimy, H. Hou, and X. Niu, "Analysis and modeling of inertial sensors using Allan variance," *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.

[13] J. J. Magnuson, D. M. Robertson, B. J. Benson, R. H. Wynne, D. M. Livingstone, T. Arai, R. A. Assel, R. G. Barry, V. Card, E. Kuusisto, N. G. Granin, T. D. Prowse, K. M. Stewart, and V. S. Vuglinski, "Historical trends in lake and river ice cover in the northern hemisphere," *Science*, vol. 289, no. 5485, pp. 1743–1746, 2000.

[14] P. Lemke, J. Ren, R. B. Alley, I. Allison, J. Carrasco, G. Flato, Y. Fujii, G. Kaser, P. Mote, R. H. Thomas, and T. Zhang, "Observations: Changes in snow, ice and frozen ground," in *Climate Change 2007: The Physical Science Basis. Contribution of Working Group 1 to the Fourth Assessment Report of the Intergovernmental Panel on Climate Change*, S. Solomon, D. Qin, M. Manning, Z. Chen, M. Marquis, K. B. Averyt, M. Tignor, and H. L. Miller, Eds. Cambridge, United Kingdom and New York, NY, USA: Cambridge University Press, 2007.

[15] J. F. Beckers, J. A. Casey, and C. Haas, "Retrievals of lake ice thickness from great slave lake and great bear lake using cryosat-2," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 7, pp. 3708–3720, 2017.

[16] J. G. Murfitt and C. R. Duguay, "50 years of lake ice research from active microwave remote sensing: Progress and prospects," *Remote Sensing of Environment*, vol. 264, p. 112616, 2021.

[17] A. Belward, M. Bourassa, M. Dowell, S. Briggs, H. Dolman, K. Holmlund, R. Husband, S. Quegan, R. Saunders, A. Simmons, B. Sloyan, D. Stammer, K. Steffen, T. Tanhua, and M. Verstraete, "The global observing system for climate: Implementation needs (gcos 2016 implementation plan)," World Meteorological Organization (WMO), Geneva, Switzerland, Tech. Rep. GCOS-200, 2016.

[18] M. Tom, M. Suetterlin, D. Bouffard, M. Rothermel, S. Wunderle, and E. Baltsavias, "Integrated monitoring of ice in selected swiss lakes. final project report," ETH Zurich / University of Bern / Swiss Federal Institute of Aquatic Science and Technology, Tech. Rep., 2019. [Online]. Available: https://prs.igp.ethz.ch/research/current_projects/integrated-monitoring-of-ice-swiss-lakes.html

[19] A. P. Annan, N. Diamanti, J. D. Redman, and S. R. Jackson, "Ground-penetrating radar for assessing winter roads," *Geophysics*, vol. 81, no. 1, pp. WA101–WA109, 2016.

[20] Bambu Lab, "Bambu Filament Technical Data Sheet V1.0: PETG HF," Bambu Lab,

Technical Data Sheet V1.0, 2024, accessed: 2026-06-04. Contains mechanical and physical properties tested under ISO 527 and ISO 178 standards. [Online]. Available: https://store.bblcdn.com/3a230e260a3a47c2b0db0156e07eef91.pdf
[21] A. K. Van der Vegt and L. E. Govaert, *Polymeren: Van Keten Tot Kunststof*. Twente, The Netherlands: VSSD, 2005, mechanical properties and sub-zero temperature resilience of amorphous co-polyesters.
[22] D. Moreno Nieto, M. Alonso-García, M.-A. Pardo-Vicente, and L. Rodríguez-Parada, "Product design by additive manufacturing for water environments: Study of degradation and absorption behavior of pla and petg," *Polymers*, vol. 13, no. 7, p. 1036, 2021. [Online]. Available: https://www.mdpi.com/2073-4360/13/7/1036
[23] Blueye Robotics AS, "Underwater GPS G2 R100 Blueye Kit," https://www.blueyerobotics.com/products/waterlinked-gps-g2-r100, 2026, accessed: 2026-06-23.
[24] Inside GNSS, "GNSS Solutions: How Well Can GPS Signals Penetrate Avalanche Snow," *Inside GNSS Magazine*, vol. 2, no. 6, pp. 24–26, 2007.
[25] Blue Robotics Inc., "Bar High-Resolution Depth/Pressure Sensors/Technical Details," https://bluerobotics.com/store/sensors-cameras/sensors/bar-depth-pressure-sensor/, accessed: 2026-06-23.
[26] KELLER AG für Druckmesstechnik, "Series 26X High-precision piezoresistive level probe," KELLER AG für Druckmesstechnik, Winterthur, Switzerland, Technical Data Sheet, 2025, accessed: 2026-06-23. [Online]. Available: https://keller-pressure.com/en/products/level-probes/standard-level-probes/series-26x
[27] Movella Inc., "Xsens Sirius IMU Technical Leaflet," Movella Automation & Mobility, Product Specification Sheet, 2025, accessed: 2026-06-23. [Online]. Available: https://www.xsens.com/sensor-modules/xsens-sirius-imu
[28] Water Linked AS, "DVL A50," Water Linked AS, Trondheim, Norway, Technical Data Sheet, 2025, accessed: 2026-06-23. [Online]. Available: https://www.waterlinked.com/shop/dvl-a50-1248#attribute_values=47,52,126
[29] Community, "Mavlink common messages," https://mavlink.io/en/messages/common.html, accessed: 2025-06-01.
[30] ——, "World geodetic system 1984," https://de.wikipedia.org/wiki/World_Geodetic_System_1984#Literatur, accessed: 2025-06-01.
[31] ——, "hysteresis," https://en.wikipedia.org/wiki/Hysteresis, accessed: 2025-06-23.
[32] A. D. Team, "Setting up sitl," https://ardupilot.org/dev/docs/SITL-setup-landingpage.html, accessed: 2025-06-01.
[33] T. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*, 04 2021.
[34] A. M. Lekkas, M. Candeloro, and I. Schjølberg, "Outlier rejection in underwater acoustic position measurements based on prediction errors," *IFAC-PapersOnLine*, vol. 48, no. 2, pp. 82–87, 2015, 4th IFAC Workshop onNavigation, Guidance and Controlof Underwater VehiclesNGCUV 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896315002530
[35] L. Paull, S. Saeedi, M. Seto, and H. Li, "AUV navigation and localization: A review," *IEEE Journal of Oceanic Engineering*, vol. 39, no. 1, pp. 131–149, 2014.
[36] L. L. Whitcomb, D. R. Yoerger, and H. Singh, "Advances in Doppler-based navigation of underwater robotic vehicles," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1999, pp. 399–406.
[37] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
[38] G. L. Smith, S. F. Schmidt, and L. A. McGee, "Application of statistical filter theory to the optimal estimation of position and velocity on board a circumlunar vehicle," NASA, Tech. Rep. TR R-135, 1962.
[39] S. J. Julier and J. K. Uhlmann, "A new extension of the Kalman filter to nonlinear systems," in *Proc. SPIE 3068, Signal Processing, Sensor Fusion, and Target Recognition VI*, 1997, pp. 182–193.
[40] E. A. Wan and R. van der Merwe, "The unscented Kalman filter for nonlinear estimation," in *Proc. IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (AS-SPCC)*, 2000, pp. 153–158.
[41] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, "Novel approach to nonlinear/non-Gaussian Bayesian state estimation," *IEE Proceedings F (Radar and Signal Processing)*, vol. 140, no. 2, pp. 107–113, 1993.
[42] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking," *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp.

174–188, 2002.
[43] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
[44] F. Dellaert and M. Kaess, "Square root SAM: Simultaneous localization and mapping via square root information smoothing," *The International Journal of Robotics Research*, vol. 25, no. 12, pp. 1181–1203, 2006.
[45] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the Bayes tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012.
[46] F. Dellaert and M. Kaess, "Factor graphs for robot perception," *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.
[47] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. John Wiley & Sons, 2001.
[48] W. Meeussen, "REP 105: Coordinate frames for mobile platforms," https://www.ros.org/reps/rep-0105.html, 2010.
[49] A. Chulliat, W. Brown, P. Alken, C. Beggan, M. Nair, G. Cox, A. Woods, S. Macmillan, B. Meyer, and M. Paniccia, "The US/UK world magnetic model for 2020-2025: Technical report," National Centers for Environmental Information, NOAA, Tech. Rep., 2020.
[50] V. Renaudin, M. H. Afzal, and G. Lachapelle, "Complete triaxis magnetometer calibration in the magnetic domain," *Journal of Sensors*, vol. 2010, p. 967245, 2010.
[51] R. K. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Transactions on Automatic Control*, vol. 15, no. 2, pp. 175–184, 1970.
[52] B. Jalving, K. Gade, K. Svartveit, A. Willumsen, and R. Sørhagen, "DVL velocity aiding in the HUGIN 1000 integrated inertial navigation system," *Modeling, Identification and Control*, vol. 25, no. 4, pp. 223–235, 2004.
[53] J. C. Kinsey, R. M. Eustice, and L. L. Whitcomb, "A survey of underwater vehicle navigation: Recent advances and new challenges," in *Proc. IFAC Conference on Manoeuvring and Control of Marine Craft (MCMC)*, Lisbon, Portugal, 2006.
[54] M. Colledanchise and P. Ögren, "Behavior trees in robotics and ai," July 2018. [Online]. Available: http://dx.doi.org/10.1201/9780429489105
[55] A. Lekkas and T. Fossen, in *Line-of-Sight Guidance for Path Following of Marine Vehicles*. LAP Lambert Academic Publishing, 06 2013.
[56] C. Shen, Y. Shi, and B. Buckham, "Model predictive control for an AUV with dynamic path planning," in *2015 54th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE)*, 2015, pp. 475–480.
[57] X. Zhang and K. Jiang, "Backstepping-based adaptive control of underactuated AUV subject to unknown dynamics and zero tracking errors," *Ocean Engineering*, vol. 302, p. 117640, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0029801824009776
[58] W. M. Bessa, M. S. Dutra, and E. Kreuzer, "Depth control of remotely operated underwater vehicles using an adaptive fuzzy sliding mode controller," *Robotics and Autonomous Systems*, vol. 56, no. 8, pp. 670–677, 2008. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889007001728
[59] Polaris, "Polaris - ardusub," https://github.com/aris-space/project-polaris-ardusub, accessed: 2025-06-01.
[60] K. Åström and R. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010. [Online]. Available: https://books.google.ch/books?id=cdG9fNqTDS8C
[61] "Declaration of generative AI usage," generative AI tools (ChatGPT GPT-4o, Gemini 3.5 Flash, and Claude 3.5 Sonnet) were used throughout the entire report for wording, language polishing, grammatical proofreading, LaTeX formatting/tables, and visualisations.